

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ
КОМП'ЮТЕРНИХ СИСТЕМ

КУРСОВИЙ ПРОЄКТ: СТРУКТУРИ ДАНИХ, АЛГОРИТМИ ТА ОБ'ЄКТНО-ОРІЄНТОВАНА ПАРАДИГМА

на тему: Система бронювання номерів в готелі

здобувач вищої освіти II курсу 243-Б групи
спеціальності 121 «Інженерія програмного
забезпечення»
першого (бакалаврського) рівня вищої освіти
Дем'янчук А.М.

Оцінка:

за національною шкалою

(словами)

кількість балів

(цифра)

за шкалою ECTS

(літера)

Чернівці, 2025

ЧЕРНІВЕЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ЮРІЯ ФЕДЬКОВИЧА

Навчально-науковий інститут фізико-технічних та комп'ютерних наук
Кафедра програмного забезпечення комп'ютерних систем

ЗАВДАННЯ
на курсовий проект
здобувачу вищої освіти
першого (бакалаврського) рівня вищої освіти
Дем'янчук Анастасії Михайлівни

- 1) Тема проєкту: Розробка системи бронювання номерів в готелі засобами мови C++.
- 2) Вхідні дані до проєкту: Індивідуальне завдання №24. Необхідно реалізувати облік готелів, номерного фонду, гостей та бронювань.
- 3) Вихідні дані до проєкту:
 - визначені сценарії використання програми (бронювання, пошук вільних номерів, заселення);
 - наведено блок-схеми алгоритмів: перевірки доступності номера, пошуку та фільтрації;
 - задокументовано класи: Hotel, Room, Guest, Booking, BookingManager та інші;
 - реалізовано збереження даних у файли CSV (data_hotels.csv, data_rooms.csv, data_bookings.csv, data_guests.csv, users.txt);
 - проведено тестування функцій пошуку вільних місць та валідації дат;
 - розроблена програмна документація.

Керівники проєкту

(підпис керівника)

Валь О.О.

(підпис керівника)

Дяконенко Б.В.

(підпис керівника)

Комісарчук В.В.

Завдання взяв до виконання

(підпис студента)

Дем'янчук А.М.

РЕФЕРАТ

Курсовий проєкт налічує сторінок, 34 рисунків, таблиць, 8 джерел та 1 додаток

Об'єктами проєктування є програмні сутності системи бронювання готелю:

- Класи – шаблони, що визначають властивості та методи для об'єктів (Hotel, Room, Guest, Booking).
- Об'єкти – екземпляри класів, які представляють конкретні сутності (готелі, номери, клієнти), що зберігаються в базі даних.
- Логіка додатку – набір правил та інструкцій, які визначають взаємодію об'єктів для досягнення поставленої мети (пошук вільних місць, оформлення бронювання).

Метою роботи є розробка програмної системи, яка автоматизує рутинні процеси адміністратора готелю, такі як облік номерного фонду, реєстрація гостей, керування бронюваннями та пошук доступних номерів за критеріями.

Для забезпечення ефективного зберігання даних було обрано формат файлу **CSV** (Comma-Separated Values).

Розроблено консольний додаток засобами середовища **CLion** на мові **C++**, який забезпечує користувачам різні рівні доступу до даних (Адміністратор та Користувач).

Реалізовано функціонал:

- ідентифікації користувача;
- маніпуляції з даними (додавання, редагування, видалення готелів, номерів, гостей);
- перегляду інформації з файлів;
- пошуку вільних номерів за містом, датами та місткістю;
- перевірки доступності номера перед бронюванням (уникнення конфліктів дат).

Дана система дозволить підвищити швидкість обслуговування клієнтів, уникнути помилок при бронюванні та забезпечити надійне централізоване зберігання інформації.

СИСТЕМА БРОНЮВАННЯ, C++, OOP, CSV, КОНСОЛЬНИЙ ДОДАТОК, ГОТЕЛЬ, БРОНЮВАННЯ.

SUMMARY

The course project contains pages, 34 figures, tables, 8 references and 1 appendix.

The objects of design are software entities of the hotel booking system:

- Classes – templates that define properties and methods for objects (Hotel, Room, Guest, Booking).
- Objects – instances of classes representing specific entities (hotels, rooms, clients) stored in the database.
- Application logic – a set of rules and instructions that define object interaction to achieve the goal (searching for available rooms, booking processing).

The aim of the work is to develop a software system that automates routine processes of a hotel administrator, such as room stock accounting, guest registration, booking management, and searching for available rooms by criteria.

The CSV (Comma-Separated Values) file format was chosen to ensure efficient data storage.

A console application was developed using the CLion environment in C++, which provides different levels of data access for users (Administrator and User).

Implemented functionality:

- user identification;
- data manipulation (adding, editing, deleting hotels, rooms, guests);
- viewing information from files;
- searching for available rooms by city, dates, and capacity;
- checking room availability before booking (avoiding date conflicts).

Also, there is a function to view the booking history of a specific guest.

This system will increase the speed of customer service, avoid booking errors, and ensure reliable centralized information storage.

BOOKING SYSTEM, C++, OOP, CSV, CONSOLE APPLICATION, HOTEL, BOOKING.

ЗМІСТ

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАЧЕНЬ	6
РОЗДІЛ 1. СПЕЦИФІКАЦІЯ ПРОЄКТУ	7
1.1. Постановка завдання	7
1.2. Опис вимог до програмної системи	7
РОЗДІЛ 2. ПРОГРАМНА ДОКУМЕНТАЦІЯ	14
2.1. Алгоритм розв’язання задачі	14
2.2. Проєктування програмної системи	20
2.3. Опис програмної системи	24
2.4. Тестування програмної системи	29
2.5. Програмні засоби розробки	35
ВИСНОВКИ	36
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	37
ДОДАТКИ	38
Додаток А. Скролінг програми	38

ПЕРЕЛІК СКОРОЧЕНЬ ТА УМОВНИХ ПОЗНАЧЕНЬ

№	Скорочення	Пояснення
1	ІС	Інформаційна система
2	CSV	Comma-Separated Values, це текстовий формат для зберігання табличних даних, де значення розділені роздільниками (використовує кому)
3	CRUD	Create, Read, Update, Delete (базові операції з даними)
4	ООП	Об'єктно-орієнтоване програмування
5	ID	Унікальний ідентифікатор запису
6	ПЗ	Програмне забезпечення
7	БД	База даних

РОЗДІЛ 1. СПЕЦИФІКАЦІЯ ПРОЄКТУ

1.1. Постановка завдання

Опис предметної області «Система бронювання номерів в готелі» охоплює процеси, пов'язані з обліком готелів, номерного фонду, клієнтської бази та реєстрацією бронювань.

Основними учасниками системи є:

- Неавторизований користувач – має пройти процедуру автентифікації.
- Користувач – має доступ до управління даними готелів, номерів, гостей та бронювань.
- Адміністратор – має повний доступ до системи, включаючи право додавати та видаляти облікові записи інших користувачів.

Доступ до основних функцій програми надається лише після успішної авторизації у додатку.

Система повинна підтримувати:

- 1) функції ідентифікації користувачів під різними ролями;
- 2) Збереження даних у текстові файли формату CSV (data_hotels.csv, data_rooms.csv, data_guests.csv, data_bookings.csv) та їх зчитування при запуску;
- 3) Додавання нової інформації в БД за атрибутами: назва готелю, місто, клас (зірки), тип номера, місткість, паспортні дані гостя, дати бронювання;
- 4) Часткове редагування існуючої інформації (можливість змінити лише окремі поля, залишивши інші без змін);
- 5) Каскадне видалення даних (при видаленні готелю автоматично видаляються всі його номери та пов'язані бронювання для забезпечення цілісності даних);
- 6) Пошук вільних номерів за критеріями: місто, місткість, дати заїзду/виїзду;
- 7) Сортування списків (наприклад, гостей за алфавітом).

1.2. Опис вимог до програмної системи

Вимоги до інтерфейсу користувача

- 1) Робоча мова інтерфейсу – українська.

- 2) Використання термінів, зрозумілих користувачеві.
- 3) Навігаційна панель (меню), яка забезпечує перегляд, редагування, додавання та зчитування даних.
- 4) «Посібник користувача», що містить інструкції з використання програми.
- 5) Повідомлення про некоректно введені дані.
- 6) Колірна гамма повинна наслідувати загальноприйняті рекомендації.

Вимоги до архітектури програми

- 1) Мова програмування – C++.
- 2) Вид програмного продукту – консольний (з наявним меню).
- 3) Дані вводяться користувачем із клавіатури, результати – виводяться у консоль.
- 4) СД – це текстовий файл формату CSV.
- 5) Наявність окремого файлу users.txt для облікових записів у форматі: username:password.
- 6) Наявність користувача «admin» із фіксованим паролем, заданим під час ініціалізації або вбудованим у файл.
- 7) Наявність користувача «admin» із паролем «admin» за замовчуванням.
- 8) Паролі зберігаються у текстовому файлі у відкритому вигляді (наприклад: admin:admin).
- 9) Доступ до основних функцій програми надається лише після успішної авторизації користувача у додатку.
- 10) Використання ООП: класи, успадкування, поліморфізм, абстрактні класи.
- 11) Кожен клас має включати: приватні поля, get/set методи, конструктор за замовчуванням, конструктор з параметрами, копіювальний та переміщувальний конструктори, деструктор.
- 12) Обробка помилок через try-catch блоки.
- 13) Використання констант замість магічних чисел та повторюваних рядків.

Вимоги до функціональності програми

- 1) Збереження даних у CSV файли, їх зчитування, зміна та видалення.
- 2) Обов'язкова перевірка коректності вводу даних та обробка виключень.
- 3) Сортування за різними критеріями.
- 4) Пошук за одним або кількома критеріями.
- 5) Фільтрація записів за заданими умовами.
- 6) Наявність текстового меню з навігацією, яке дозволяє:
 - додавати нові об'єкти;
 - редагувати існуючі;
 - видаляти об'єкти;
 - переглядати списки об'єктів;
 - шукати за одним або кількома критеріями;
 - сортувати дані;
 - фільтрувати записи за заданою умовою;
 - виводити інструкцію користувача.
- 7) Програма повинна підтримувати роботу з обліковими записами користувачів:
 - при запуску програма запитує логін і пароль;
 - адміністратор має доступ до перегляду, додавання та видалення користувачів.

Опис сценаріїв використання програми

Сценарій 1: Авторизація в системі	
Передумова: Програма запущена, відображається логотип та запит авторизації.	
Основний сценарій	
Крок 1	Система виводить запит «Логін: ». Користувач вводить свій логін.
Крок 2	Система виводить запит «Пароль: ». Користувач вводить пароль.
Крок 3	Система перевіряє введені дані через менеджер користувачів.
Крок 4	Якщо дані вірні, виводиться повідомлення «Ви успішно увійшли в систему!», відображається ім'я користувача та його роль (якщо це адміністратор).
Крок 5	Відкривається Головне меню програми.
Альтернативний сценарій	

Крок 1	Користувач вводить неправильний логін або пароль.
Крок 2	Система виводить повідомлення «Невірний логін або пароль! Залишилось спроб: [X]».
Крок 3	Якщо кількість спроб вичерпана, програма завершує роботу або блокує вхід. Якщо спроби є — повернення до Кроку 1.

Сценарій 2: Додавання нового готелю	
Передумова: Користувач знаходиться в підменю «Керування готелями».	
Основний сценарій	
Крок 1	Користувач обирає пункт 1. Додати готель.
Крок 2	Система виводить заголовок «ДОДАТИ ГОТЕЛЬ» і запитує «Назва готелю: ».
Крок 3	Користувач вводить назву.
Крок 4	Система запитує «Місто: ». Користувач вводить місто.
Крок 5	Система запитує «Кількість зірок (1-5): ». Користувач вводить число.
Крок 6	Система створює об'єкт готелю, присвоює йому унікальний ID і зберігає в пам'яті.
Крок 7	Виводиться повідомлення «Готель успішно додано!».
Альтернативний сценарій	
Крок 1	При введенні кількості зірок користувач вводить число поза діапазоном (наприклад, 6) або текст.
Крок 2	Спрацьовує валідація: виводиться повідомлення «Число має бути в діапазоні від 1 до 5» або «Некоректне введення».
Крок 3	Система просить повторити введення некоректного поля.

Сценарій 3: Редагування даних готелю	
Передумова: Користувач знаходиться в підменю «Керування готелями».	
Основний сценарій	
Крок 1	Користувач обирає пункт 2. Редагувати готель.
Крок 2	Система виводить список усіх готелів і просить ввести ID готелю.
Крок 3	Користувач вводить ID.
Крок 4	Система знаходить готель і виводить його поточні дані. Відображається меню вибору поля для редагування (1. Назву, 2. Місто, 3. Зірки, 4. Все, 0. Скасувати).
Крок 5	Користувач обирає, наприклад, «1. Назву».
Крок 6	Система просить ввести нову назву (або натиснути Enter, щоб залишити стару).
Крок 7	Користувач вводить нові дані.
Крок 8	Система оновлює запис і виводить «Готель успішно відредаговано!».
Альтернативний сценарій	
Крок 1	Користувач вводить ID готелю, якого не існує.
Крок 2	Система виводить помилку «Готель не знайдено!» і повертає користувача в меню.

Сценарій 4: Видалення готелю	
Передумова: Користувач знаходиться в підменю «Керування готелями».	
Основний сценарій	

Крок 1	Користувач обирає пункт 3. Видалити готель.
Крок 2	Система виводить список готелів і просить ввести ID.
Крок 3	Користувач вводить ID.
Крок 4	Система показує дані готелю і попередження: «При видаленні готелю будуть також видалені всі його номери та бронювання!».
Крок 5	Система запитує підтвердження (так/ні).
Крок 6	Користувач підтверджує дію.
Крок 7	Система видалляє всі номери, прив'язані до цього готелю, а потім сам готель.
Крок 8	Виводиться повідомлення «Готель успішно видалено! Також видалено номерів: [кількість]».
Альтернативний сценарій	
Крок 1	Користувач на етапі підтвердження вводить «ні».
Крок 2	Система виводить повідомлення «Видалення скасовано» і повертає в меню.

Сценарій 5: Перегляд та пошук готелів	
Передумова: Користувач знаходиться в підменю «Керування готелями».	
Основний сценарій (Перегляд)	
Крок 1	Користувач обирає пункт 4. Переглянути всі готелі.
Крок 2	Система виводить таблицю з усіма готелями.
Крок 3	Очікування натискання клавіші для повернення.
Крок 7	Якщо перетинів немає — бронювання створюється.
Основний сценарій (Пошук)	
Крок 1	Користувач обирає пункт 5. Пошук готелів.
Крок 2	Система пропонує критерії: 1. За назвою, 2. За містом, 3. За кількістю зірок.
Крок 3	Користувач обирає, наприклад, «За містом» і вводить «Київ».
Крок 4	Система фільтрує список і виводить тільки готелі в Києві.
Альтернативний сценарій	
Крок 1	Пошук не дав результатів.
Крок 2	Система виводить повідомлення «Нічого не знайдено!».

Сценарій 6: Сортювання списку готелів	
Передумова: Користувач знаходиться в підменю «Керування готелями».	
Основний сценарій	
Крок 1	Користувач обирає пункт 6. Сортювання готелів.
Крок 2	Система запитує критерій сортювання: 1. За назвою, 2. За містом, 3. За зірками.
Крок 3	Користувач робить вибір.
Крок 4	Система сортує список у пам'яті та виводить повідомлення про успіх (наприклад, «Готелі відсортовано за назвою!»).
Крок 5	Автоматично викликається функція перегляду вже відсортованого списку.
Альтернативний сценарій	
Крок 1	Користувач обирає неіснуючий пункт меню сортювання.
Крок 2	Система повідомляє про невірний вибір.

Сценарій 7: Додавання нового номера	
Передумова: Користувач знаходиться в меню «Керування номерами»	
Основний сценарій	

Крок 1	Користувач обирає опцію «1. Додати номер».
Крок 2	Система виводить список доступних готелів і просить обрати ID.
Крок 3	Користувач вводить ID готелю.
Крок 4	Система запитує тип номера (введіть "Люкс" або "Стандарт").
Крок 5	Система запитує місткість (введіть 2 або 3).
Крок 6	Програма створює новий об'єкт Room, прив'язує його до готелю і зберігає.
Крок 7	Виводиться повідомлення «Номер успішно додано» і оновлений список номерів.
Альтернативний сценарій	
Крок 1	Користувач вводить некоректну місткість (наприклад, 5).
Крок 2	Система виводить повідомлення «Некоректне введення. Будь ласка, введіть число» або валідацію (якщо реалізовано обмеження 2/3).
Крок 3	Повторний запит даних.

Сценарій 8: Редагування номера	
Передумова: Користувач знаходиться в меню «Керування номерами»	
Основний сценарій	
Крок 1	Користувач обирає опцію «2. Редагувати номер».
Крок 2	Система просить ввести ID номера.
Крок 3	Користувач вводить ID існуючого номера.
Крок 4	Система запитує «Новий клас (Enter - залишити без змін): ».
Крок 5	Користувач натискає Enter (пропускає зміну класу).
Крок 6	Система запитує «Нові місця (0 - залишити): ».
Крок 7	Користувач вводить нове число (наприклад, 3).
Крок 8	Система оновлює тільки місткість, залишаючи клас без змін.
Альтернативний сценарій	
Крок 1	Користувач вводить ID номера, якого немає в базі.
Крок 2	Система виводить «Помилка: Номер не знайдено».

Сценарій 9: Видалення номера	
Передумова: Користувач знаходиться в меню «Керування номерами»	
Основний сценарій	
Крок 1	Користувач обирає опцію «3. Видалити номер».
Крок 2	Система просить ввести ID номера.
Крок 3	Користувач вводить ID.
Крок 4	Система автоматично звертається до BookingManager і видалляє всі активні бронювання, пов'язані з цим номером.
Крок 5	Система видалляє сам номер з реєстру.
Крок 6	Виводиться повідомлення про успішне видалення.
Альтернативний сценарій	
Не передбачено (якщо ID існує, видалення відбувається миттєво).	

Сценарій 10: Реєстрація нового гостя	
Передумова: Користувач знаходиться в меню «Керування гостями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «1. Додати гостя».
Крок 2	Система запитує Прізвище клієнта.
Крок 3	Система запитує Ім'я клієнта.
Крок 4	Система запитує Номер телефону.

Крок 5	Користувач вводить дані.
Крок 6	Система створює нового гостя в GuestManager і присвоює йому ID.
Крок 7	Виводиться повідомлення «Гостя успішно додано».
Альтернативний сценарій	
Крок 1	Користувач залишає обов'язкове поле пустим.
Крок 2	Система виводить попередження «Введення не може бути порожнім» і просить повторити ввід.

Сценарій 11: Редагування даних гостя	
Передумова: Користувач знаходиться в меню «Керування гостями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «2. Редагувати гостя».
Крок 2	Вводить ID гостя.
Крок 3	Система показує поточні дані.
Крок 4	Система запитує «Нове Прізвище (Enter - залишити без змін): ».
Крок 5	Користувач вводить нове прізвище або натискає Enter.
Крок 6	Аналогічно для Імені та Телефону.
Крок 7	Система зберігає зміни лише для тих полів, що були введені.
Крок 8	Виводиться повідомлення «Дані оновлено».
Альтернативний сценарій	
Крок 1	Введено ID, якого не існує.
Крок 2	Система виводить «Помилка: Гостя не знайдено» і повертає в меню.

Сценарій 12: Видалення гостя	
Передумова: Користувач знаходиться в меню «Керування гостями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «3. Видалити гостя».
Крок 2	Вводить ID гостя.
Крок 3	Система знаходить гостя і попереджає: «При видаленні гостя будуть також видалені всі його бронювання!».
Крок 4	Користувач підтверджує дію.
Крок 5	BookingManager знаходить всі активні бронювання цього гостя і видаляє їх (номери звільняються).
Крок 6	GuestManager видаляє запис про гостя.
Крок 7	Система зберігає зміни лише для тих полів, що були введені.
Альтернативний сценарій	
Крок 1	Користувач скасовує видалення на етапі підтвердження.
Крок 2	Дані залишаються без змін.

Сценарій 13: Сортювання та Пошук гостей	
Передумова: Користувач знаходиться в меню «Керування Гостями».	
Основний сценарій (Сортювання)	
Крок 1	Користувач обирає опцію «4. Сортювати за іменем».
Крок 2	Система сортує список гостей в алфавітному порядку за Прізвищем/Іменем.
Крок 3	Оновлений список виводиться на екран.
Основний сценарій (Пошук)	
Крок 1	Користувач обирає опцію «5. Пошук гостя».
Крок 2	Вводить критерій (ПІБ або Номер телефону).

Крок 3	Система шукає збіги.
Крок 4	Виводяться дані знайденого гостя або повідомлення «Не знайдено».
Альтернативний сценарій	
Крок 1	Пошук не дав результатів.
Крок 2	Система виводить повідомлення «Нічого не знайдено!».

Сценарій 14: Створення нового бронювання	
Передумова: Користувач знаходиться в меню «Керування бронюваннями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «1. Додати бронювання».
Крок 2	Система запитує бажані дати заїзду та виїзду.
Крок 3	Система аналізує базу даних і виводить список готелів, вказуючи кількість вільних номерів у кожному з них на ці дати.
Крок 4	Користувач обирає ID готелю.
Крок 5	Система виводить список конкретних вільних номерів у цьому готелі (ID, Клас, Місткість).
Крок 6	Користувач вводить ID бажаного номера.
Крок 7	Користувач обирає ID гостя зі списку.
Крок 8	Система створює бронювання і зберігає його.
Альтернативний сценарій	
Крок 1	На вказані дати немає жодного вільного номера в жодному готелі.
Крок 2	Система виводить повідомлення «На жаль, на ці дати немає вільних номерів» і повертає в меню.

Сценарій 15: Редагування бронювання	
Передумова: Користувач знаходиться в меню «Керування бронюваннями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «2. Редагувати бронювання».
Крок 2	Вводить ID бронювання.
Крок 3	Система виводить поточні дані та меню вибору: 1. Змінити гостя, 2. Змінити дати.
Крок 4	Варіант А (Зміна гостя): Користувач обирає нового гостя зі списку.
Крок 5	Варіант Б (Зміна дат): Користувач вводить нові дати заїзду/виїзду. Система перевіряє, чи вільний цей номер у новий період.
Крок 6	Якщо перевірка успішна, дані оновлюються.
Альтернативний сценарій	
Крок 1	При зміні дат виявляється, що номер вже зайнятий у новий період.
Крок 2	Система виводить помилку «Номер зайнятий на ці дати!» і скасовує редагування.

Сценарій 16: Видалення бронювання	
Передумова: Користувач знаходиться в меню «Керування бронюваннями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «3. Видалити бронювання».
Крок 2	Вводить ID бронювання.
Крок 3	Система запитує підтвердження.
Крок 4	Користувач підтверджує дію.
Крок 5	Система видаляє запис про бронювання, номер автоматично стає вільним на ці дати.
Крок 6	Виводиться повідомлення «Бронювання успішно видалено».
Альтернативний сценарій	

Крок 1	Введено невірний ID.
Крок 2	Система повідомляє «Бронювання не знайдено».

Сценарій 17: Перегляд зайнятості по готелю	
Передумова: Користувач знаходиться в меню «Керування бронюваннями»	
Основний сценарій	
Крок 1	Користувач обирає опцію «4. Зайнятість по готелю».
Крок 2	Обирає ID готелю зі списку.
Крок 3	Система виводить список усіх номерів цього готелю.
Крок 4	Для кожного номера вказується його статус: «Вільно» або дати активного бронювання.
Крок 5	Користувач переглядає звіт.
Альтернативний сценарій	
Не передбачено.	

Сценарій 18: Пошук історії бронювань	
Передумова: Користувач знаходиться в Головному меню	
Основний сценарій	
Крок 1	Користувач обирає пункт «5. Пошук бронювань».
Крок 2	Система пропонує критерії пошуку: 1. За ID гостя, 2. За ID номера, 3. За датою заїзду, 4. За датою виїзду.
Крок 3	Користувач обирає, наприклад, «1. За ID гостя» і обирає гостя зі списку.
Крок 4	BookingManager виконує пошук (searchByGuestId) і повертає список бронювань.
Крок 5	Результати виводяться на екран у вигляді таблиці.
Альтернативний сценарій	
Крок 1	Користувач обирає гостя, у якого немає активних бронювань.
Крок 2	Система виводить повідомлення «У цього гостя немає бронювань».

Сценарій 19: Перегляд довідки (Допомога)	
Передумова: Користувач знаходиться в Головному меню	
Основний сценарій	
Крок 1	Користувач обирає пункт «6. Допомога».
Крок 2	На екран виводиться детальна інструкція: опис функцій, правила вводу даних (формат дат, числових полів), пояснення щодо навігації.
Крок 3	Користувач натискає Enter для повернення в меню.
Альтернативний сценарій	
Не передбачено.	

Сценарій 20: Керування обліковими записами	
Передумова: Користувач авторизований як Адміністратор і знаходиться в Головному меню	
Основний сценарій	
Крок 1	Користувач обирає пункт «7. Адміністрування».
Крок 2	Відкривається підменю: 1. Додати користувача, 2. Видалити користувача, 3. Переглянути всіх.
Крок 3	Адміністратор обирає «1. Додати користувача».
Крок 4	Вводить новий логін. Система перевіряє його унікальність (userExists).

Крок 5	Вводить пароль та підтверджує права адміністратора.
Крок 6	Новий користувач додається до системи.
Альтернативний сценарій	
Крок 1	Адміністратор намагається видалити користувача з логіном admin або самого себе.
Крок 2	Система блокує дію і виводить повідомлення: «Не можна видалити адміністратора!» або «Не можна видалити себе!».

Сценарій 21: Збереження даних та вихід	
Передумова: Користувач знаходиться в Головному меню	
Основний сценарій	
Крок 1	Користувач обирає пункт «8. Вихід та збереження» (або 0).
Крок 2	Викликається функція saveAllData().
Крок 3	Дані з усіх менеджерів (HotelManager, RoomManager тощо) послідовно записуються у відповідні файли .csv.
Крок 4	Виводиться повідомлення «До побачення! Дані збережено».
Крок 5	Програма завершує виконання.
Альтернативний сценарій	
Крок 1	Виникає помилка доступу до файлу при запису (наприклад, файл відкритий в іншій програмі).
Крок 2	Система перехоплює виняток (catch std::exception) і виводить повідомлення про помилку: «Не вдалося зберегти дані у файл».

РОЗДІЛ 2. ПРОГРАМНА ДОКУМЕНТАЦІЯ

2.1. Алгоритм розв'язання задачі

Узагальнені блок-схеми алгоритмів додавання об'єкту, редагування об'єкту та видалення об'єкту наведено на рис.1-3.

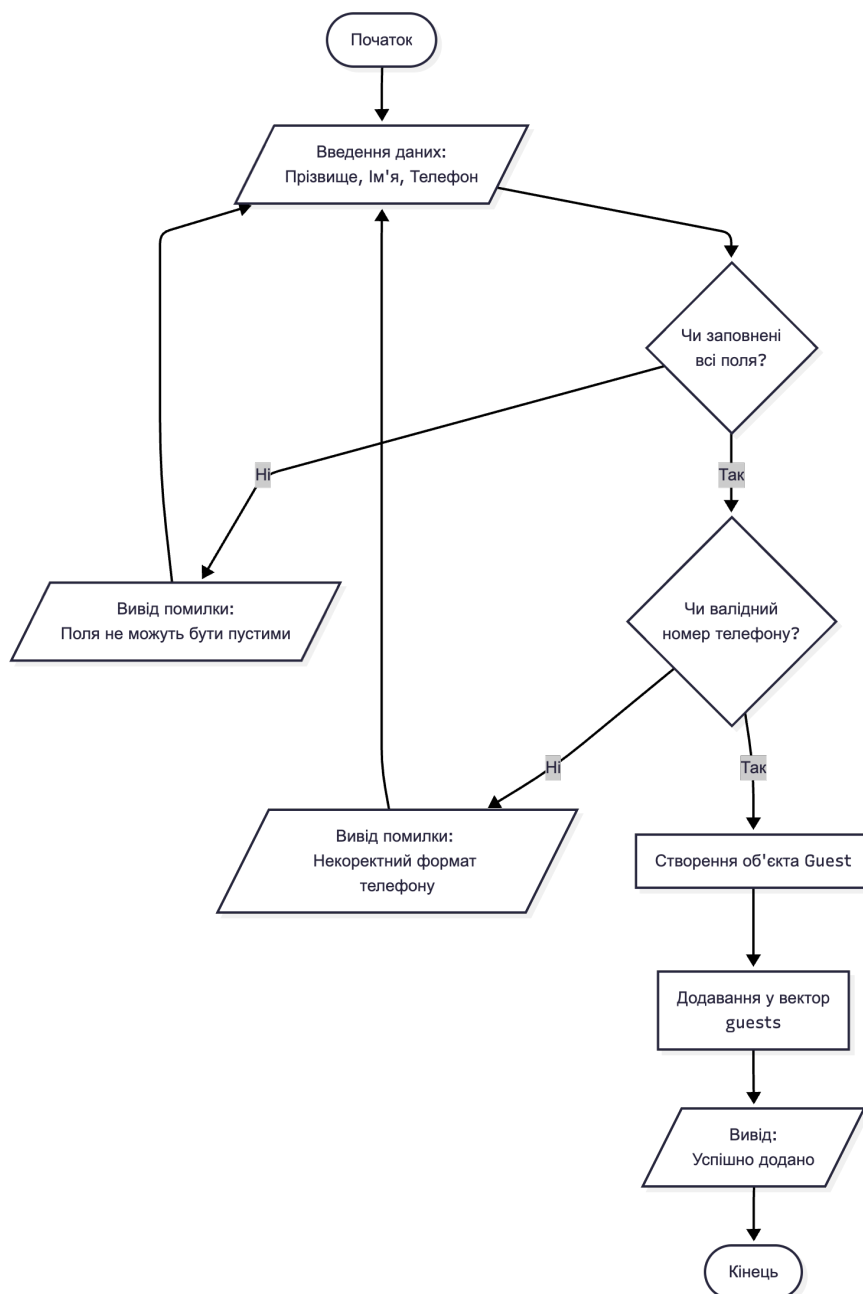


Рисунок 1 – Блок-схема алгоритму додавання Гостя

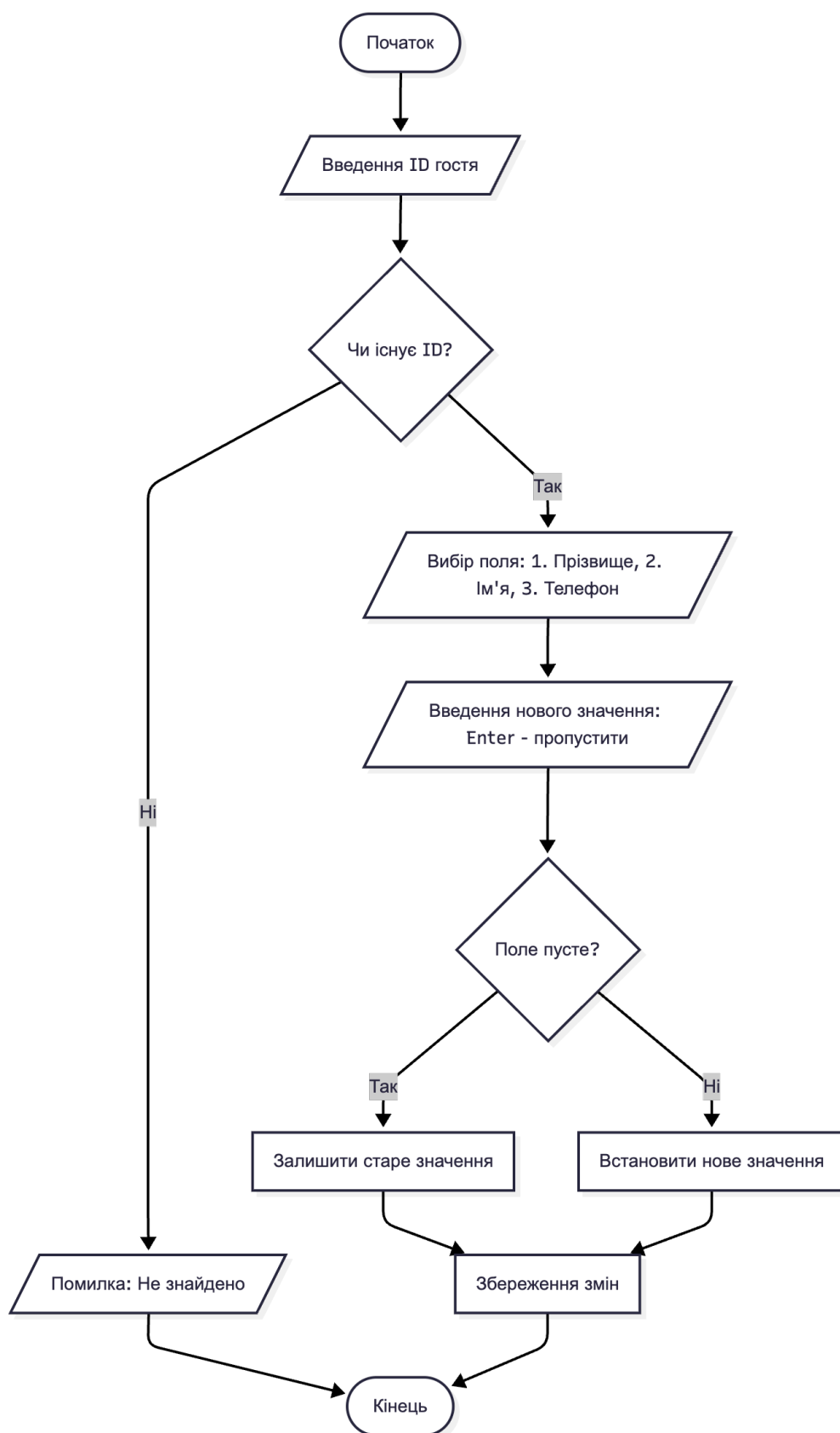


Рисунок 2 – Блок-схема алгоритму редагування Гостя

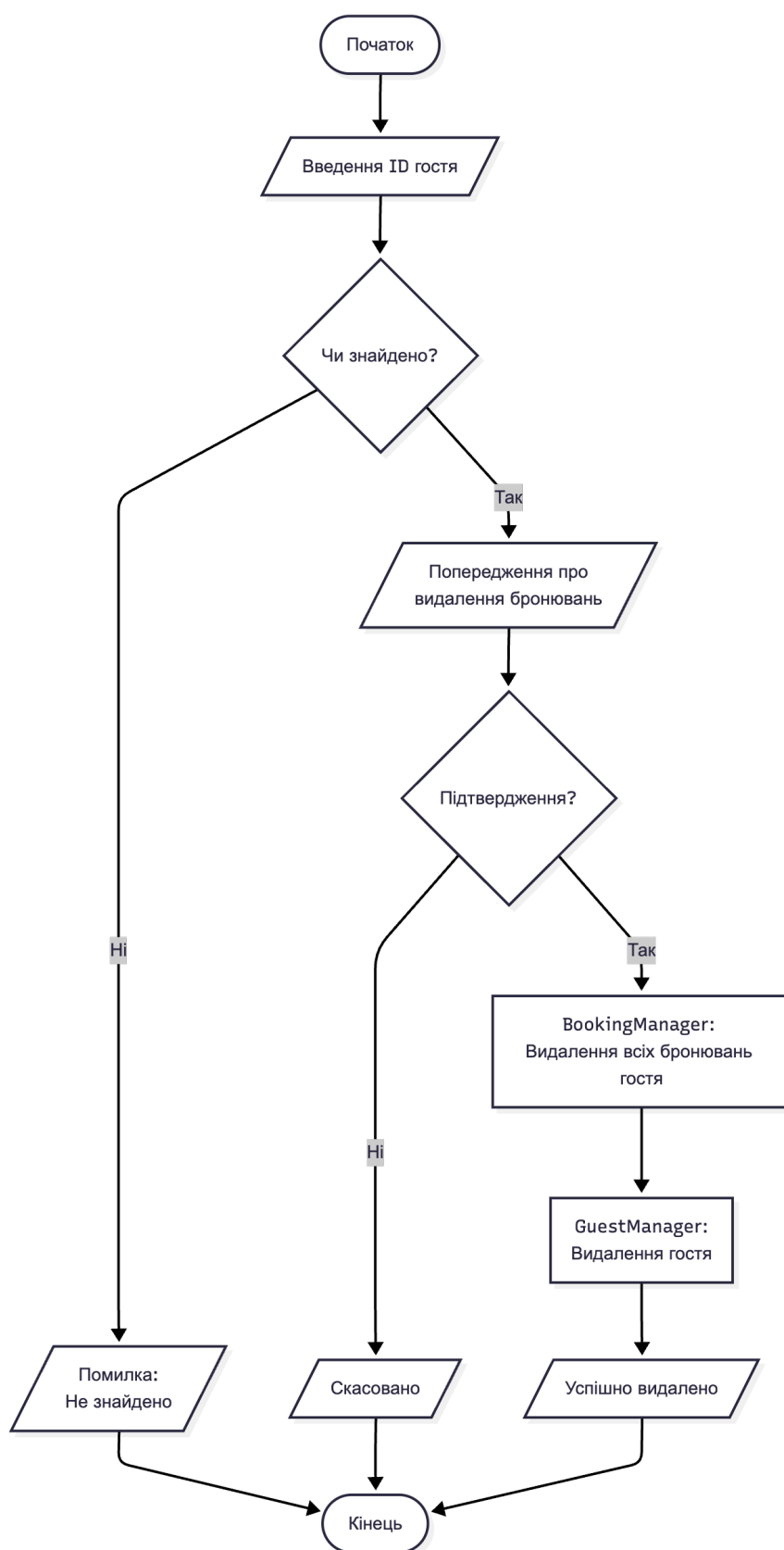


Рисунок 3 – Блок-схема алгоритму видалення Гостя

Узагальнені блок-схеми алгоритмів пошуку об'єкта, перегляду доданих об'єктів та їх сортування, а також перевірки доступності номера наведено на рис. 4-6

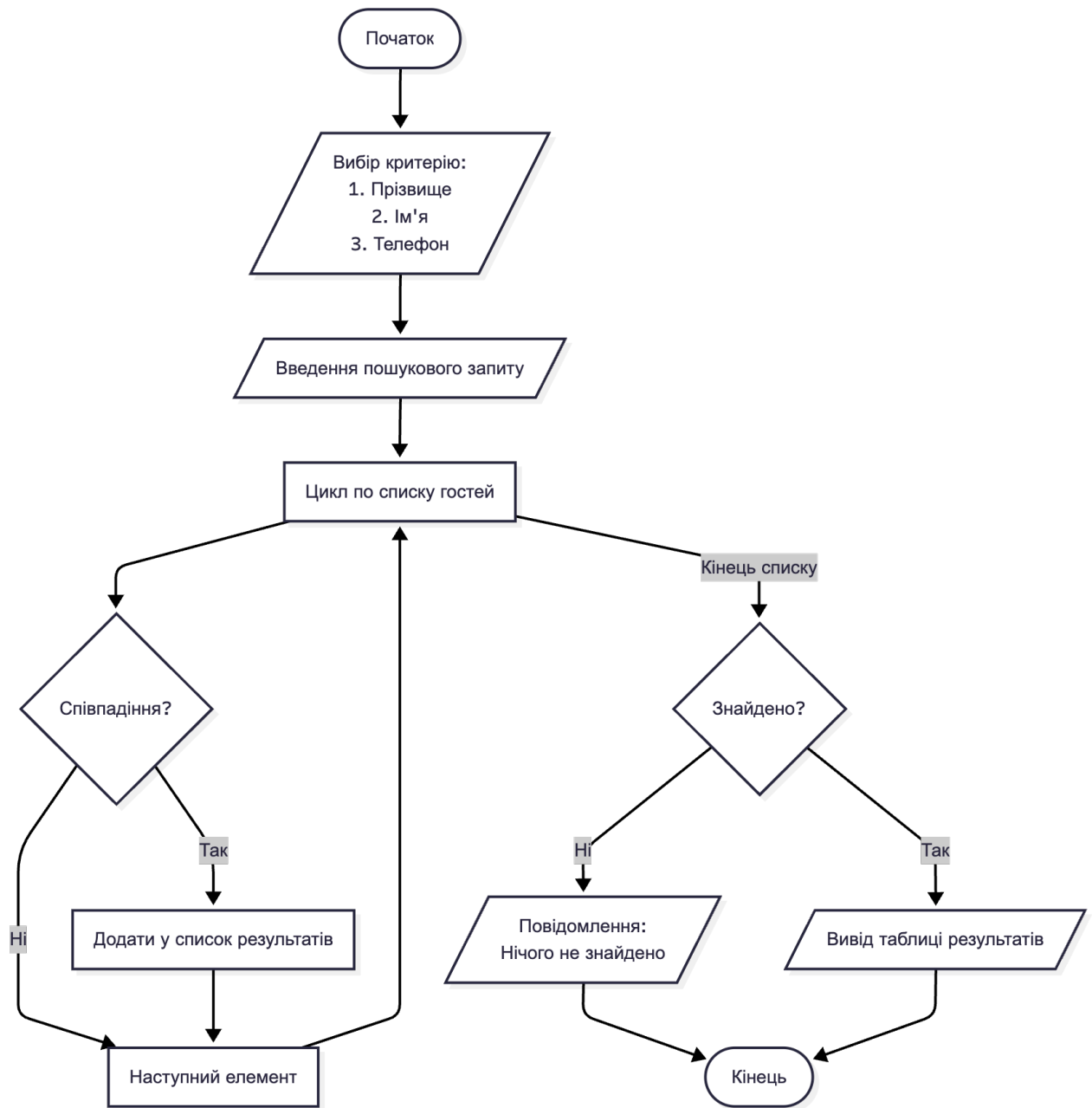


Рисунок 4 – Узагальнена блок-схема алгоритму пошуку Гостя

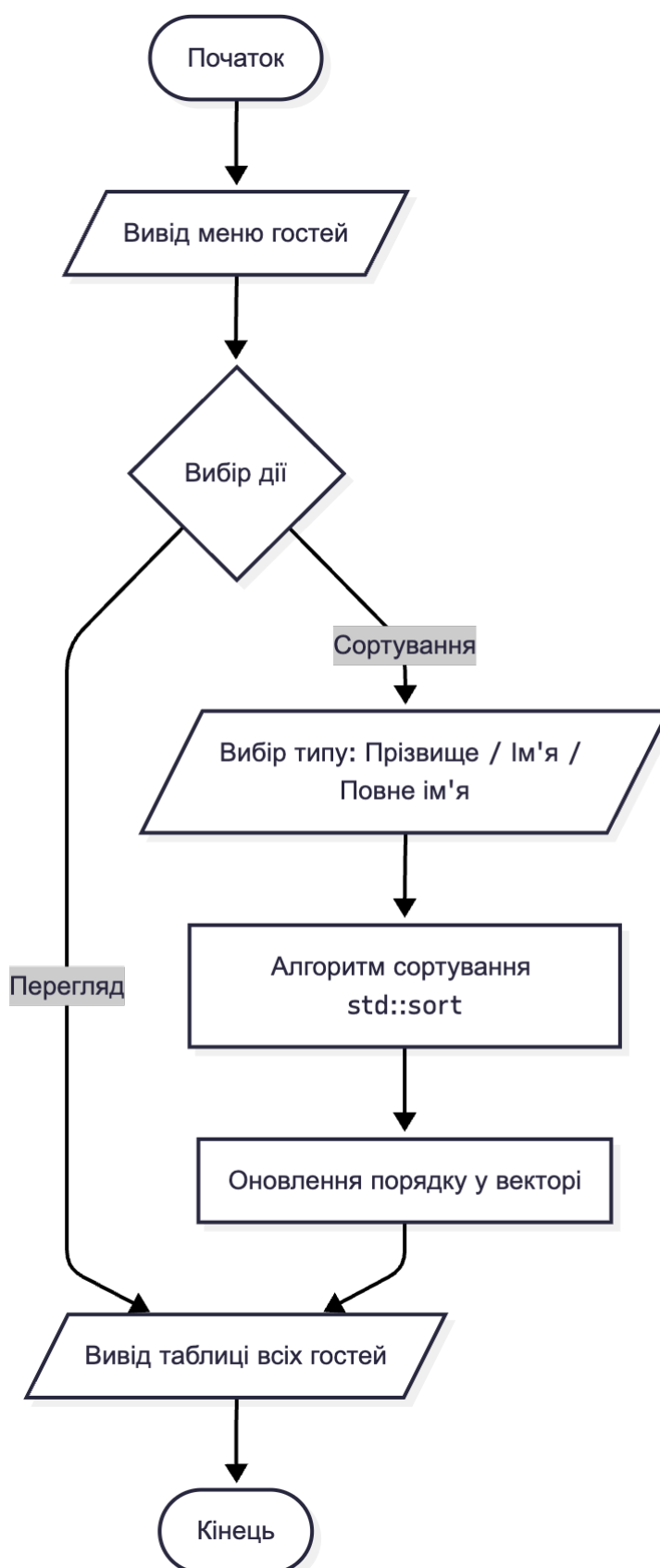


Рисунок 5 – Блок-схема алгоритму перегляду доданих Гостей та їх сортування

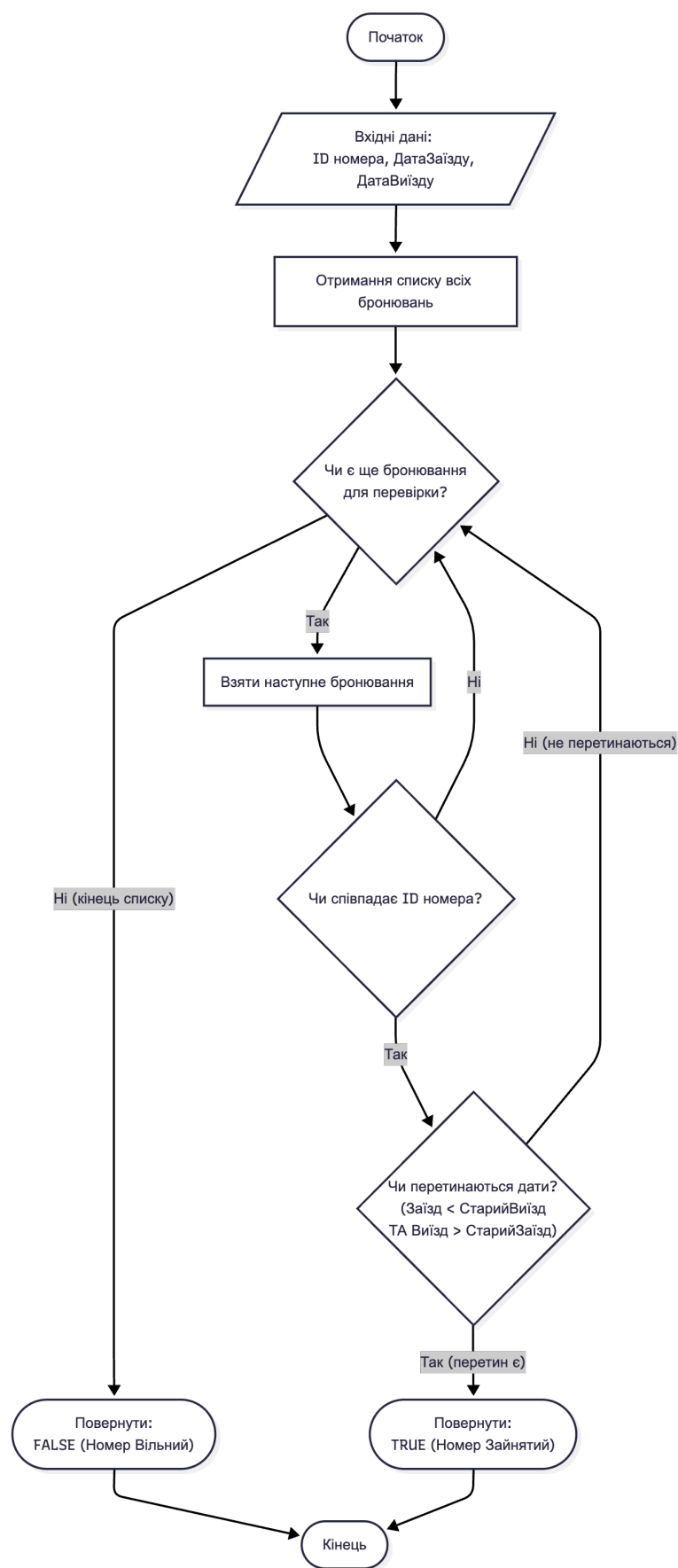


Рисунок 6 – Блок-схема алгоритму перевірки доступності номера

2.2. Проектування програмної системи

Робота розробленого програмного забезпечення реалізується наступними класами:

- 1) Hotel.
- 2) HotelManager.
- 3) Room.
- 4) LuxuryRoom.
- 5) StandardRoom.
- 6) RoomManager.
- 7) Guest.
- 8) GuestManager.
- 9) Booking.
- 10) BookingManager.
- 11) User.
- 12) UserManager.
- 13) Menu.

В табл. 1-13 подано документування класів.

Таблиця 1 – Документування класу *Hotel*

Характеристика		Опис
Назва класу		Hotel
Опис класу		Клас для представлення готелю в системі бронювання. Зберігає інформацію про назву, місто розташування та кількість зірок готелю.
Залежності від інших класів		Використовується класами HotelManager для управління колекцією готелів та RoomManager для зв'язку номерів з готелями
Опис атрибутів (полів)		id -- унікальний ідентифікатор готелю name -- назва готелю city -- місто розташування stars -- кількість зірок (1-5) nextId -- статичне поле для генерації унікальних ID
Опис методів		
1	Hotel() -- конструктор за замовчуванням	
2	Hotel(name, city, stars) -- конструктор з параметрами	
3	Hotel(id, name, city, stars) -- конструктор з усіма параметрами	

4	Hotel(const Hotel& other) -- копіювальний конструктор
5	Hotel(Hotel&& other) -- переміщувальний конструктор
6	~Hotel() -- деструктор з повідомленням про знищення
7	getId() -- повертає ID готелю
8	getName() -- повертає назву готелю
9	getCity() -- повертає місто
10	getStars() -- повертає кількість зірок
11	setId(int) -- встановлює ID
12	setName(string) -- встановлює назву
13	setCity(string) -- встановлює місто
14	setStars(int) -- встановлює кількість зірок з валідацією
15	display() -- виводить інформацію про готель
16	validate() -- перевіряє валідність даних
17	toCSV() -- конвертує об'єкт у рядок CSV
18	fromCSV(string) -- створює об'єкт з рядка CSV
19	updateNextId(int) -- оновлює лічильник ID

Таблиця 2 – Документування класу *HotelManager*

Характеристика		Опис
Назва класу		HotelManager
Опис класу		Клас-менеджер для управління колекцією готелів. Забезпечує функціонал додавання, редагування, видалення, пошуку та сортування готелів. Відповідає за збереження та завантаження даних з CSV файлу.
Залежності від інших класів		Управляє колекцією об'єктів Hotel. Використовується Menu для виконання операцій з готелями
Опис атрибутів (полів)		hotels -- вектор об'єктів Hotel filename -- ім'я CSV файлу для збереження даних
Опис методів		
1	HotelManager() -- конструктор за замовчуванням (використовує "data_hotels.csv")	
2	HotelManager() -- конструктор за замовчуванням (використовує "data_hotels.csv")	
3	HotelManager() -- конструктор за замовчуванням (використовує "data_hotels.csv")	
4	HotelManager(HotelManager&&) -- переміщувальний конструктор	
5	~HotelManager() -- деструктор	
6	addHotel(Hotel) -- додає новий готель до колекції	
7	editHotel(id, Hotel) -- редагує готель за ID	
8	deleteHotel(id) -- видаляє готель за ID	
9	findHotelById(id) -- знаходить готель за ID	

10	searchByName(string) -- пошук готелів за назвою
11	searchByCity(string) -- пошук готелів за містом
12	searchByStars(int) -- пошук готелів за кількістю зірок
13	getAllHotels() -- повертає всі готелі
14	getHotelCount() -- повертає кількість готелів
15	hotelExists(id) -- перевіряє чи існує готель
16	sortByName() -- сортує готелі за назвою
17	sortByCity() -- сортує готелі за містом
18	sortByStars() -- сортує готелі за кількістю зірок
19	displayAll() -- виводить всі готелі
20	loadFromFile() -- завантажує дані з CSV файлу
21	saveToFile() -- зберігає дані у CSV файл
22	clear() -- очищає всі дані

Таблиця 3 – Документування класу *Room*

Характеристика		Опис
Назва класу		Room
Опис класу		Абстрактний базовий клас для представлення номера в готелі. Містить чисті віртуальні методи для реалізації поліморфізму. Визначає загальні властивості та поведінку для всіх типів номерів.
Залежності від інших класів		Батьківський клас для LuxuryRoom та StandardRoom. Використовується RoomManager для управління колекцією номерів
Опис атрибутів (полів)		id -- унікальний ідентифікатор номера hotelId -- ID готелю до якого належить номер capacity -- кількість місць (2 або 3) isOccupied -- статус зайнятості номера nextId -- статичне поле для генерації ID
Опис методів		
1	Room() -- конструктор за замовчуванням	
2	Room(hotelId, capacity) -- конструктор з параметрами	
3	Room(id, hotelId, capacity, isOccupied) -- конструктор з усіма параметрами	
4	Room(const Room&) -- копіювальний конструктор	
5	Room(Room&&) -- переміщувальний конструктор	
6	virtual ~Room() -- віртуальний деструктор	
7	getId() -- повертає ID номера	
8	getHotelId() -- повертає ID готелю	
9	getCapacity() -- повертає кількість місць	

10	getIsOccupied() -- повертає статус зайнятості
11	setId(int) -- встановлює ID
12	setHotelId(int) -- встановлює ID готелю
13	setCapacity(int) -- встановлює кількість місць з валідацією
14	setIsOccupied(bool) -- встановлює статус зайнятості
15	virtual getType() -- чисто віртуальний метод отримання типу номера
16	virtual getTypeName() -- чисто віртуальний метод отримання назви типу
17	virtual display() -- чисто віртуальний метод виведення інформації
18	virtual calculatePrice() -- чисто віртуальний метод розрахунку вартості
19	validate() -- перевіряє валідність даних
20	toCSV() -- конвертує об'єкт у рядок CSV
21	static fromCSV(string) -- створює відповідний тип номера з CSV
22	static updateNextId(int) -- оновлює лічильник ID

Таблиця 4 – Документування класу *LuxuryRoom*

Характеристика		Опис
Назва класу		LuxuryRoom
Опис класу		Клас для представлення номера типу "Люкс". Успадковується від абстрактного класу Room. Реалізує специфічну поведінку для люкс-номерів, включаючи розрахунок підвищеної вартості.
Залежності від інших класів		Успадковується від Room. Використовується RoomManager для створення та управління люкс-номерами
Опис атрибутів (полів)		BASE_PRICE -- статична константа базової ціни люкс-номера (2500 грн)
Опис методів		
1	LuxuryRoom() -- конструктор за замовчуванням	
2	LuxuryRoom(hotelId, capacity) -- конструктор з параметрами	
3	LuxuryRoom(id, hotelId, capacity, isOccupied) -- конструктор з усіма параметрами	
4	LuxuryRoom(const LuxuryRoom&) -- копіювальний конструктор	
5	LuxuryRoom(LuxuryRoom&&) -- переміщувальний конструктор	
6	~LuxuryRoom() -- деструктор	
7	getType() -- повертає RoomType::LUXURY	
8	getTypeName() -- повертає "Люкс"	
9	display() -- виводить інформацію про люкс-номер з іконками	
10	calculatePrice() -- розраховує вартість (базова ціна + доплата за додаткове місце)	

Таблиця 5 – Документування класу *StandardRoom*

Характеристика		Опис
Назва класу		StandardRoom
Опис класу		Клас для представлення номера типу "Стандарт". Успадковується від абстрактного класу Room. Реалізує специфічну поведінку для стандартних номерів з базовою вартістю.
Залежності від інших класів		Успадковується від Room. Використовується RoomManager для створення та управління стандартними номерами
Опис атрибутів (полів)		BASE_PRICE -- статична константа базової ціни стандартного номера (1200 грн)
Опис методів		
1	StandardRoom() -- конструктор за замовчуванням	
2	StandardRoom(hotelId, capacity) -- конструктор з параметрами	
3	StandardRoom(id, hotelId, capacity, isOccupied) -- конструктор з усіма параметрами	
4	StandardRoom(const StandardRoom&) -- копіювальний конструктор	
5	StandardRoom(StandardRoom&&) -- переміщувальний конструктор	
6	~StandardRoom() -- деструктор	
7	getType() -- повертає RoomType::STANDARD	
8	getTypeName() -- повертає "Стандарт"	
9	display() -- виводить інформацію про стандартний номер	
10	calculatePrice() -- розраховує вартість (базова ціна + доплата за додаткове місце)	

Таблиця 6 – Документування класу *RoomManager*

Характеристика		Опис
Назва класу		RoomManager
Опис класу		Клас-менеджер для управління колекцією номерів. Працює з вказівниками на об'єкти Room для реалізації поліморфізму. Забезпечує каскадне видалення номерів при видаленні готелю.
Залежності від інших класів		Управляє вказівниками на Room (LuxuryRoom, StandardRoom). Використовується Menu та взаємодіє з HotelManager
Опис атрибутів (полів)		rooms -- вектор вказівників на Room filename -- ім'я CSV файлу для збереження даних

Опис методів	
1	RoomManager() -- конструктор за замовчуванням
2	RoomManager(filename) -- конструктор з параметром
3	RoomManager(const RoomManager&) -- копіювальний конструктор з глибоким копіюванням
4	RoomManager(RoomManager&&) -- переміщувальний конструктор
5	~RoomManager() -- деструктор з очищенням пам'яті
6	addRoom(Room*) -- додає новий номер
7	editRoom(id, Room*) -- редагує номер за ID
8	deleteRoom(id) -- видаляє номер за ID
9	deleteRoomsByHotelId(hotelId) -- каскадне видалення номерів готелю
10	findRoomById(id) -- знаходить номер за ID
11	searchByHotelId(hotelId) -- пошук номерів за ID готелю
12	searchByType(RoomType) -- пошук номерів за типом
13	searchByCapacity(int) -- пошук номерів за кількістю місць
14	searchFreeRooms() -- пошук всіх вільних номерів
15	searchFreeRoomsByHotelId(hotelId) -- пошук вільних номерів у готелі
16	getAllRooms() -- повертає всі номери
17	getRoomCount() -- повертає загальну кількість номерів
18	getRoomCountByHotelId(hotelId) -- повертає кількість номерів у готелі
19	getFreeRoomCountByHotelId(hotelId) -- повертає кількість вільних номерів у готелі
20	roomExists(id) -- перевіряє чи існує номер
21	sortByHotelId() -- сортує номери за ID готелю
22	sortByType() -- сортує номери за типом
23	sortByCapacity() -- сортує номери за кількістю місць
24	displayAll() -- виводить всі номери
25	displayByHotelId(hotelId) -- виводить номери конкретного готелю
26	loadFromFile() -- завантажує дані з CSV
27	saveToFile() -- зберігає дані у CSV
28	clear() -- очищає всі дані та звільняє пам'ять

Таблиця 7 – Документування класу *Guest*

Характеристика	Опис
Назва класу	Guest
Опис класу	Клас для представлення гостя готелю. Зберігає персональну інформацію про гостя включаючи ПІБ та контактний телефон.

Залежності від інших класів	Використовується GuestManager для управління колекцією гостей та BookingManager для створення бронювань
Опис атрибутів (полів)	id -- унікальний ідентифікатор гостя lastName -- прізвище гостя firstName -- ім'я гостя phone -- номер телефону nextId -- статичне поле для генерації ID
Опис методів	
1	Guest() -- конструктор за замовчуванням
2	Guest(lastName, firstName, phone) -- конструктор з параметрами
3	Guest(id, lastName, firstName, phone) -- конструктор з усіма параметрами
4	Guest(const Guest&) -- копіювальний конструктор
5	Guest(Guest&&) -- переміщувальний конструктор
6	~Guest() -- деструктор
7	getId() -- повертає ID гостя
8	getLastName() -- повертає прізвище
9	getFirstName() -- повертає ім'я
10	getFullName() -- повертає повне ім'я (прізвище + ім'я)
11	getPhone() -- повертає номер телефону
12	setId(int) -- встановлює ID
13	setLastName(string) -- встановлює прізвище
14	setFirstName(string) -- встановлює ім'я
15	setPhone(string) -- встановлює телефон з валідацією
16	display() -- виводить інформацію про гостя
17	validate() -- перевіряє валідність даних
18	static isValidPhone(string) -- перевіряє валідність номера телефону
19	toCSV() -- конвертує об'єкт у рядок CSV
20	static fromCSV(string) -- створює об'єкт з рядка CSV
21	static updateNextId(int) -- оновлює лічильник ID
22	operator<(const Guest&) -- оператор порівняння для сортування за ім'ям
23	operator==(const Guest&) -- оператор перевірки рівності

Таблиця 8 – Документування класу *GuestManager*

Характеристика	Опис
Назва класу	GuestManager
Опис класу	Клас-менеджер для управління колекцією гостей. Забезпечує функціонал додавання, редагування, видалення, пошуку та сортування гостей за іменем (згідно вимог проєкту).

Залежності від інших класів	Управляє колекцією об'єктів Guest. Використовується Menu та BookingManager
Опис атрибутів (полів)	guests -- вектор об'єктів Guest filename -- ім'я CSV файлу для збереження даних
Опис методів	
1	GuestManager() -- конструктор за замовчуванням
2	GuestManager(filename) -- конструктор з параметром
3	GuestManager(const GuestManager&) -- копіювальний конструктор
4	GuestManager(GuestManager&&) -- переміщувальний конструктор
5	~GuestManager() -- деструктор
6	addGuest(Guest) -- додає нового гостя
7	editGuest(id, Guest) -- редагує дані гостя
8	deleteGuest(id) -- видаляє гостя
9	findGuestById(id) -- знаходить гостя за ID
10	searchByLastName(string) -- пошук гостей за прізвищем
11	searchByFirstName(string) -- пошук гостей за ім'ям
12	searchByPhone(string) -- пошук гостя за номером телефону
13	getAllGuests() -- повертає всіх гостей
14	getGuestCount() -- повертає кількість гостей
15	guestExists(id) -- перевіряє чи існує гість
16	sortByLastName() -- сортує гостей за прізвищем
17	sortByFirstName() -- сортує гостей за ім'ям
18	sortByFullName() -- сортує гостей за повним ім'ям (прізвище + ім'я)
19	displayAll() -- виводить всіх гостей
20	loadFromFile() -- завантажує дані з CSV
21	saveToFile() -- зберігає дані у CSV
22	clear() -- очищає всі дані

Таблиця 9 – Документування класу *Booking*

Характеристика	Опис
Назва класу	Booking
Опис класу	Клас для представлення бронювання номера. Зберігає інформацію про гостя, номер, дати заїзду та виїзду. Забезпечує валідацію дат та розрахунок тривалості проживання.
Залежності від інших класів	Використовується BookingManager для управління бронюваннями. Пов'язаний з Guest (через guestId) та Room (через roomId)
Опис атрибутів (полів)	id -- унікальний ідентифікатор бронювання guestId -- ID гостя roomId -- ID номера checkInDate -- дата заїзду (формат YYYY-

	MM-DD) checkOutDate -- дата виїзду (формат YYYY-MM-DD) nextId -- статичне поле для генерації ID
Опис методів	
1	Booking() -- конструктор за замовчуванням
2	Booking(guestId, roomId, checkInDate, checkOutDate) -- конструктор з параметрами
3	Booking(id, guestId, roomId, checkInDate, checkOutDate) -- конструктор з усіма параметрами
4	Booking(const Booking&) -- копіювальний конструктор
5	Booking(Booking&&) -- переміщувальний конструктор
6	~Booking() -- деструктор
7	getId() -- повертає ID бронювання
8	getGuestId() -- повертає ID гостя
9	getRoomId() -- повертає ID номера
10	getCheckInDate() -- повертає дату заїзду
11	getCheckOutDate() -- повертає дату виїзду
12	setId(int) -- встановлює ID
13	setGuestId(int) -- встановлює ID гостя з валідацією
14	setRoomId(int) -- встановлює ID номера з валідацією
15	setCheckInDate(string) -- встановлює дату заїзду з валідацією
16	setCheckOutDate(string) -- встановлює дату виїзду з валідацією
17	display() -- виводить інформацію про бронювання
18	validate() -- перевіряє валідність всіх даних
19	static isValidDate(string) -- перевіряє валідність формату дати
20	areDatesValid() -- перевіряє що дата виїзду пізніше дати заїзду
21	calculateDays() -- розраховує кількість днів проживання
22	toCSV() -- конвертує об'єкт у рядок CSV
23	static fromCSV(string) -- створює об'єкт з рядка CSV
24	static updateNextId(int) -- оновлює лічильник ID
25	static compareDates(string, string) -- порівнює дві дати

Таблиця 10 – Документування класу *BookingManager*

Характеристика	Опис
Назва класу	BookingManager
Опис класу	Клас-менеджер для управління колекцією бронювань. Забезпечує перевірку доступності номерів на конкретні дати, каскадне видалення бронювань при видаленні гостя або номера, пошук та сортування бронювань.

Залежності від інших класів		Управляє колекцією об'єктів Booking. Взаємодіє з GuestManager та RoomManager для валідації зв'язків
Опис атрибутів (полів)		bookings -- вектор об'єктів Booking filename -- ім'я CSV файлу для збереження даних
Опис методів		
1	BookingManager()	-- конструктор за замовчуванням
2	BookingManager(filename)	-- конструктор з параметром
3	BookingManager(const BookingManager&)	-- копіювальний конструктор
4	BookingManager(BookingManager&&)	-- переміщувальний конструктор
5	~BookingManager()	-- деструктор
6	addBooking(Booking)	-- додає нове бронювання
7	editBooking(id, Booking)	-- редагує бронювання
8	deleteBooking(id)	-- видаляє бронювання
9	deleteBookingsByGuestId(guestId)	-- каскадне видалення бронювань гостя
10	deleteBookingsByRoomId(roomId)	-- каскадне видалення бронювань номера
11	findBookingById(id)	-- знаходить бронювання за ID
12	searchByGuestId(guestId)	-- пошук бронювань гостя
13	searchByRoomId(roomId)	-- пошук бронювань номера
14	searchByCheckInDate(string)	-- пошук за датою заїзду
15	searchByCheckOutDate(string)	-- пошук за датою виїзду
16	isRoomOccupied(roomId, checkIn, checkOut)	-- перевіряє зайнятість номера в період
17	getAllBookings()	-- повертає всі бронювання
18	getBookingCount()	-- повертає кількість бронювань
19	bookingExists(id)	-- перевіряє чи існує бронювання
20	sortByCheckInDate()	-- сортує за датою заїзду
21	sortByCheckOutDate()	-- сортує за датою виїзду
22	sortByGuestId()	-- сортує за ID гостя
23	displayAll()	-- виводить всі бронювання
24	displayByGuestId(guestId)	-- виводить бронювання гостя
25	displayByRoomId(roomId)	-- виводить бронювання номера
26	loadFromFile()	-- завантажує дані з CSV
27	saveToFile()	-- зберігає дані у CSV
28	clear()	-- очищає всі дані

Таблиця 11 – Документування класу *User*

Характеристика	Опис
----------------	------

Назва класу	User
Опис класу	Клас для представлення користувача системи. Забезпечує функціонал авторизації та управління правами доступу (звичайний користувач або адміністратор).
Залежності від інших класів	Використовується UserManager для управління обліковими записами користувачів
Опис атрибутів (полів)	username -- логін користувача password -- пароль користувача isAdmin -- прапорець статусу адміністратора
Опис методів	
1	User() -- конструктор за замовчуванням
2	User(username, password, isAdmin) -- конструктор з параметрами
3	User(const User&) -- копіювальний конструктор
4	User(User&&) -- переміщувальний конструктор
5	~User() -- деструктор
6	getUsername() -- повертає логін
7	getPassword() -- повертає пароль
8	getIsAdmin() -- повертає статус адміністратора
9	setUsername(string) -- встановлює логін з валідацією
10	setPassword(string) -- встановлює пароль з валідацією
11	setIsAdmin(bool) -- встановлює статус адміністратора
12	display() -- виводить інформацію про користувача
13	validate() -- перевіряє валідність даних
14	static isValidUsername(string) -- перевіряє валідність логіну (мінімум 3 символи)
15	static isValidPassword(string) -- перевіряє валідність пароля (мінімум 4 символи)
16	checkPassword(string) -- перевіряє чи співпадає пароль
17	toString() -- конвертує об'єкт у рядок для файлу (username:password:isAdmin)
18	static fromString(string) -- створює об'єкт з рядка
19	operator==(const User&) -- оператор перевірки рівності

Таблиця 12 – Документування класу *UserManager*

Характеристика	Опис
Назва класу	UserManager
Опис класу	Клас-менеджер для управління обліковими записами користувачів. Забезпечує авторизацію, створення користувача-адміністратора за замовчуванням, управління правами доступу.

Залежності від інших класів	Управляє колекцією об'єктів User. Використовується Menu для перевірки прав доступу
Опис атрибутів (полів)	users -- вектор об'єктів User filename -- ім'я текстового файлу (users.txt) currentUser -- вказівник на поточного авторизованого користувача
Опис методів	
1	UserManager() -- конструктор за замовчуванням, створює адміністратора
2	UserManager(filename) -- конструктор з параметром
3	UserManager(const UserManager&) -- копіювальний конструктор
4	UserManager(UserManager&&) -- переміщувальний конструктор
5	~UserManager() -- деструктор
6	initializeDefaultAdmin() -- створює користувача admin:admin якщо не існує
7	login(username, password) -- авторизує користувача
8	logout() -- виходить з системи
9	getCurrentUser() -- повертає поточного користувача
10	isLoggedIn() -- перевіряє чи користувач авторизований
11	isCurrentUserAdmin() -- перевіряє чи поточний користувач адміністратор
12	addUser(User) -- додає нового користувача (тільки для адміна)
13	deleteUser(username) -- видаляє користувача (не можна видалити admin та себе)
14	findUserByUsername(username) -- знаходить користувача за логіном
15	userExists(username) -- перевіряє чи існує користувач
16	getAllUsers() -- повертає всіх користувачів
17	getUserCount() -- повертає кількість користувачів
18	displayAll() -- виводить всіх користувачів
19	loadFromFile() -- завантажує дані з users.txt
20	saveToFile() -- зберігає дані у users.txt
21	clear() -- очищає всі дані та відновлює адміністратора

Таблиця 12 – Документування класу *Menu*

Характеристика	Опис
Назва класу	Menu
Опис класу	Клас для управління інтерфейсом користувача системи бронювання. Забезпечує консольне меню з навігацією, авторизацію, всі операції з даними через відповідні Manager класи.
Залежності від інших класів	Використовує всі Manager класи (HotelManager, RoomManager, GuestManager,

	BookingManager, UserManager) для виконання операцій. Взаємодіє з Utils для введення/виведення
Опис атрибутів (полів)	hotelManager -- посилання на HotelManager roomManager -- посилання на RoomManager guestManager -- посилання на GuestManager bookingManager -- посилання на BookingManager userManager -- посилання на UserManager isRunning -- прапорець роботи програми
Опис методів	
1	Menu(...) -- конструктор з посиланнями на всі Manager класи
2	~Menu() -- деструктор
3	run() -- запускає головний цикл програми
4	showLoginScreen() -- показує екран авторизації
5	showMainMenu() -- показує головне меню (7 пунктів)
6	hotelMenu() -- меню керування готелями
7	addHotel() -- додати готель
8	editHotel() -- редагувати готель
9	deleteHotel() -- видалити готель з каскадним видаленням номерів
10	viewAllHotels() -- переглянути всі готелі
11	searchHotels() -- пошук готелів
12	sortHotels() -- сортування готелів
13	roomMenu() -- меню керування номерами
14	addRoom() -- додати номер (Люкс/Стандарт)
15	editRoom() -- редагувати номер
16	deleteRoom() -- видалити номер з каскадним видаленням бронювань
17	viewAllRooms() -- переглянути всі номери
18	viewRoomsByHotel() -- переглянути номери конкретного готелю
19	searchFreeRooms() -- пошук вільних номерів
20	sortRooms() -- сортування номерів
21	guestMenu() -- меню керування гостями
22	addGuest() -- додати гостя
23	editGuest() -- редагувати гостя
24	deleteGuest() -- видалити гостя з каскадним видаленням бронювань
25	viewAllGuests() -- переглянути всіх гостей
26	searchGuests() -- пошук гостей
27	sortGuests() -- сортування гостей за іменем
28	bookingMenu() -- меню керування бронюваннями
29	addBooking() -- додати бронювання з перевіркою доступності
30	editBooking() -- редагувати бронювання
31	deleteBooking() -- видалити бронювання
32	viewAllBookings() -- переглянути всі бронювання
33	searchBookings() -- пошук бронювань

34	sortBookings() -- сортування бронювань
35	showHelp() -- показати довідку
36	adminMenu() -- меню адміністрування (тільки для адміна)
37	addUser() -- додати користувача
38	deleteUser() -- видалити користувача
39	viewAllUsers() -- переглянути всіх користувачів
40	saveAllData() -- зберегти всі дані у файли
41	loadAllData() -- завантажити всі дані з файлів
42	selectHotel() -- допоміжний метод вибору готелю зі списку
43	selectRoom() -- допоміжний метод вибору номера зі списку
44	selectGuest() -- допоміжний метод вибору гостя зі списку

2.3. Опис програмної системи

Взаємодія з користувачем відбувається через консольний інтерфейс. При запуску програми спочатку відбувається ініціалізація менеджерів та завантаження даних з файлів. Після цього на екран виводиться логотип системи та вікно авторизації.

1. Авторизація в системі

Для початку роботи користувач повинен пройти процедуру автентифікації. Система запитує логін та пароль. Введені дані перевіряються через клас UserManager. Якщо дані вірні, користувач отримує доступ до головного меню. У разі помилки виводиться повідомлення «Невірний логін або пароль!» і кількість спроб зменшується (рис. 27).

```

=====
                        АВТОРИЗАЦІЯ
=====
Логін:  admin
Пароль:  admin

✅ Ви успішно увійшли в систему!

Вітаємо, admin!
Роль: Адміністратор

Натисніть Enter для продовження...

```

Рисунок 7 – Екран авторизації користувача

Після успішного входу система вітає користувача та відображає його роль (наприклад, «Роль: Адміністратор»).

2. Головне меню

Головне меню програми є центральним вузлом навігації і містить наступні пункти (рис. 8):

1. Керування готелями.
2. Керування номерами.
3. Керування гостями.
4. Керування бронюваннями.
5. Пошук бронювань.
6. Допомога.
7. Адміністрування (відображається лише для адміністратора).

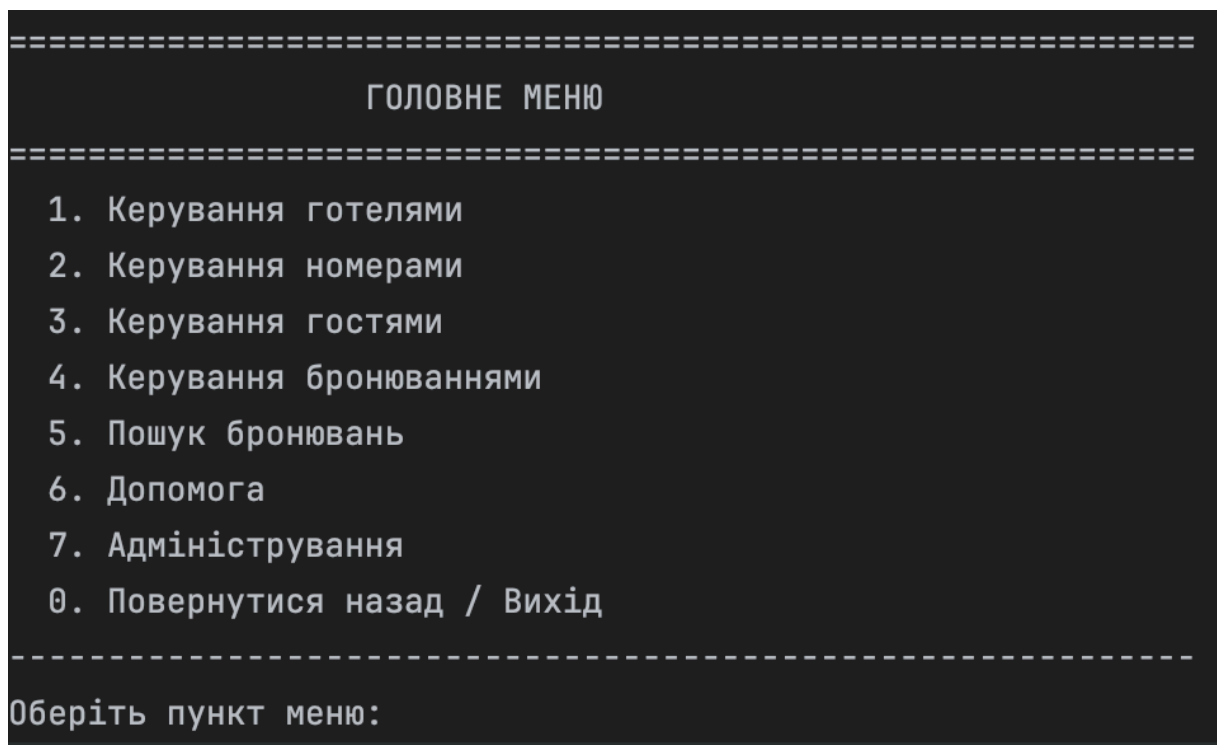


Рисунок 8 – Головне меню програми

3. Підсистема керування готелями

Обравши пункт «Керування готелями», користувач потрапляє у відповідне підменю (рис. 9). Тут доступні опції додавання, редагування, видалення, перегляду, пошуку та сортування готелів.

При виборі опції «Переглянути всі готелі» (пункт 4) на екран виводиться таблиця з інформацією про кожен готель: ID, назва, місто та рейтинг у зірках (рис. 10).

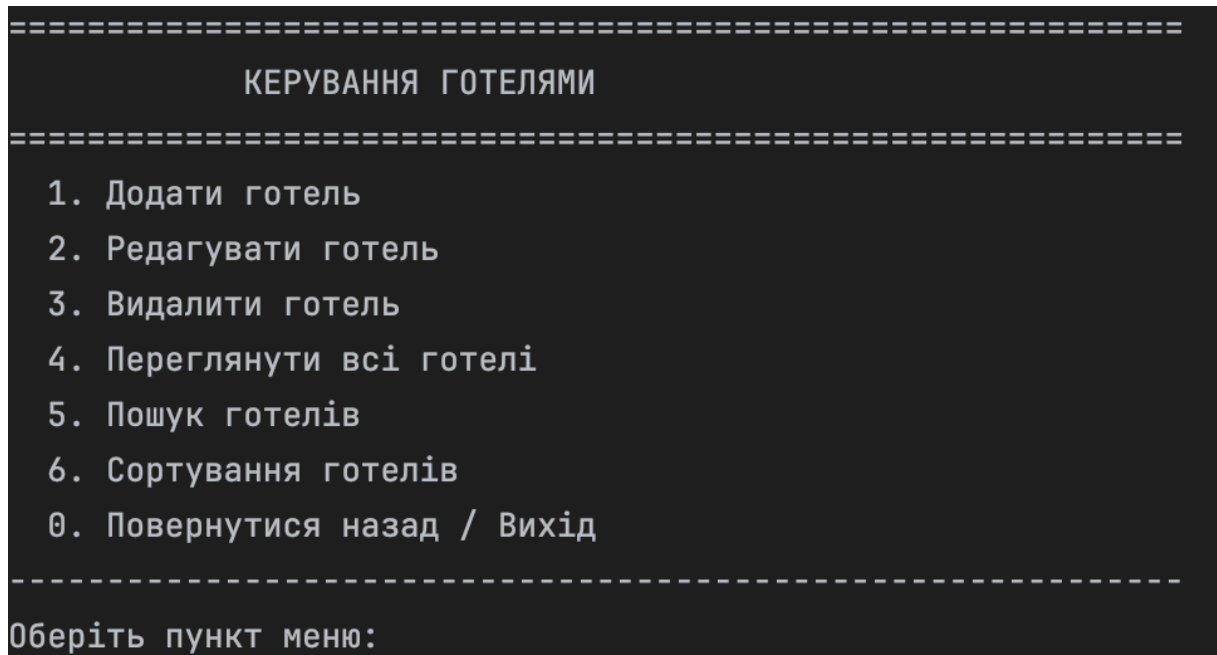


Рисунок 9 – Меню керування готелями



Рисунок 10 – Перегляд списку готелів

Функція «Додати готель» (пункт 1) запитує у користувача назву, місто та кількість зірок, після чого валідує введені дані (наприклад, кількість зірок має бути від 1 до 5).

4. Підсистема керування номерами та пошук вільних місць

У меню «Керування номерами» (рис. 11) користувач може адмініструвати номерний фонд. Особливістю цього розділу є функція «Пошук вільних номерів» (пункт 6).

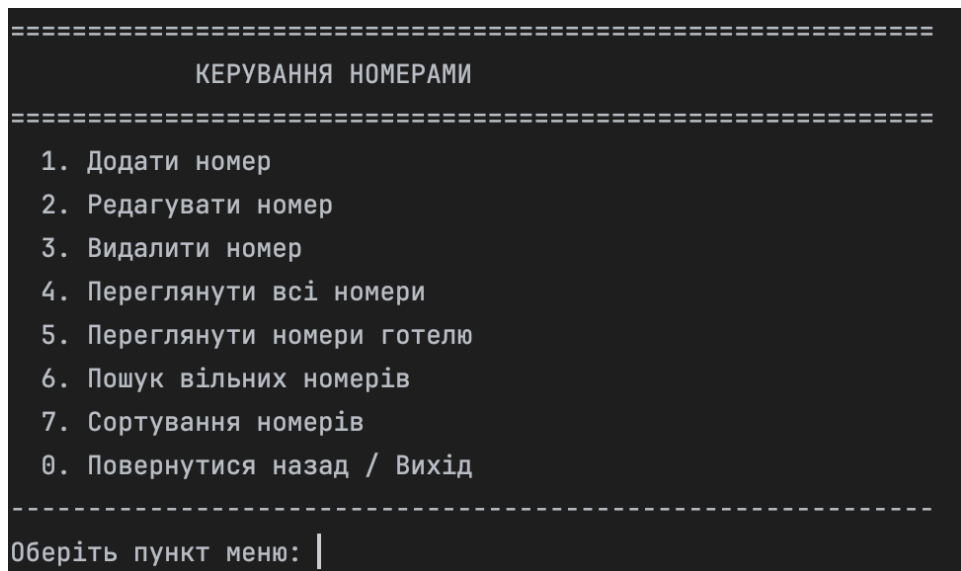


Рисунок 11 – Меню керування номерами

При виборі цього пункту система пропонує шукати у всіх готелях або в конкретному. Після цього відображається список номерів, які не мають активних бронювань (статус `isOccupied` є `false` або немає перетину дат) (рис. 12).


```

=====
                          ПОШУК ВІЛЬНИХ НОМЕРІВ
=====
Шукати в:
1. Всіх готелях
2. Конкретному готелі
0. Скасувати
Ваш вибір: 1

=====
                          ВІЛЬНІ НОМЕРИ (5)
=====

┌────────────────────────────────────────────────────────────────────────────────┐
│                                ЛЮКС НОМЕР                                    │
├────────────────────────────────────────────────────────────────────────────────┤
│ ID номера: 3                                                              │
│ ID готелю: 4                                                              │
│ Місць: 2                                                                  │
│ Статус: Вільний                                                           │
│ Ціна: 2500 грн/доба                                                       │
└────────────────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────────────────┐
│                                СТАНДАРТНИЙ НОМЕР                        │
├────────────────────────────────────────────────────────────────────────────────┤
│ ID номера: 2                                                              │
│ ID готелю: 1                                                              │
│ Місць: 2                                                                  │
│ Статус: Вільний                                                           │
│ Ціна: 1200 грн/доба                                                       │
└────────────────────────────────────────────────────────────────────────────────┘

```

Рисунок 12 – Результат пошуку вільних номерів

Також тут можна переглянути список номерів конкретного готелю (пункт 5), де для кожного номера вказано його тип (Люкс/Стандарт), місткість та статус (рис. 13).

```

=====
                                НОМЕРИ ГОТЕЛЮ
=====

=====
                                ОБЕРІТЬ ГОТЕЛЬ
=====

Список готелів:

1. Hilton (Київ) - ★★★★★
2. Bristol (Одеса) - ★★★★★
3. Hotel Dnipro (Київ) - ★★★★
4. Bukovyna (Чернівці) - ★★★★
5. Turist (Чернівці) - ★★★

Оберіть готель (0 для скасування): 1

=====
НОМЕРИ ГОТЕЛЮ (ID: 1, Всього: 2)
=====

┌────────────────────────────────────────────────────────────────────────────────┐
│                                ЛЮКС НОМЕР                                │
├────────────────────────────────────────────────────────────────────────────────┤
ID номера: 1
ID готелю: 1
Місце: 3
Статус: Зайнятий
Ціна: 3000 грн/доба
└────────────────────────────────────────────────────────────────────────────────┘

```

Рисунок 13 – Список номерів конкретного готелю

5. Підсистема «Керування гостями» (рис. 14)

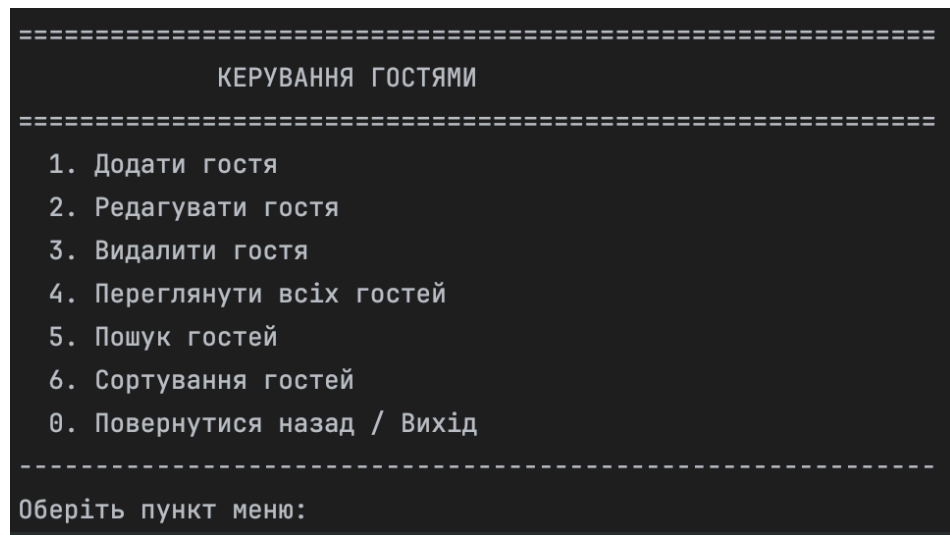


Рисунок 14 – Меню керування гостями

Дозволяє вести базу клієнтів. Функціонал включає:

- Додавання гостя: Введення Прізвища, Імені та Телефону (з валідацією формату номера).
- Перегляд гостей: Можливість переглянути список всіх гостей і інформацію про них (рис. 15)

```
=====
СПИСОК ГОСТЕЙ (Всього: 4)
=====

[
  ІНФОРМАЦІЯ ПРО ГОСТЯ
]
ID: 1
Прізвище: Дем'янчук
Ім'я: Настя
Телефон: 0989853936

[
  ІНФОРМАЦІЯ ПРО ГОСТЯ
]
ID: 3
Прізвище: Дорофтей
Ім'я: Катя
Телефон: 0990128525

[
  ІНФОРМАЦІЯ ПРО ГОСТЯ
]
ID: 4
Прізвище: Кирилюк
Ім'я: Макс
Телефон: 0956124946
```

Рисунок 16 – Результат пошуку гостя (за прізвищем)

- Пошук гостей: Можливість знайти клієнта за прізвищем, ім'ям або номером телефону (рис. 16).

```
=====
                                ПОШУК ГОСТЕЙ
                                =====
Шукати за:
1. Прізвищем
2. Ім'ям
3. Номером телефону
0. Скасувати
Ваш вибір: 1
Введіть прізвище: Дем'янчук

=====
                        РЕЗУЛЬТАТИ ПОШУКУ (1)
                        =====

┌────────────────────────────────────────────────────────────────────────────────┐
│                                ІНФОРМАЦІЯ ПРО ГОСТЯ                                │
├────────────────────────────────────────────────────────────────────────────────┤
│ ID: 1                                                                    │
│ Прізвище: Дем'янчук                                                       │
│ Ім'я: Настя                                                                │
│ Телефон: 0989853936                                                       │
└────────────────────────────────────────────────────────────────────────────────┘
```

Рисунок 16 – Результат пошуку гостя (за прізвищем)

- Сортування: Впорядкування списку гостей за алфавітом (за прізвищем, ім'ям або повним ім'ям) (рис. 17)



Рисунок 17 – Результат сортування гостей (за прізвищем)

6. Підсистема керування бронюваннями (рис. 18)

Це основний робочий інструмент менеджера.

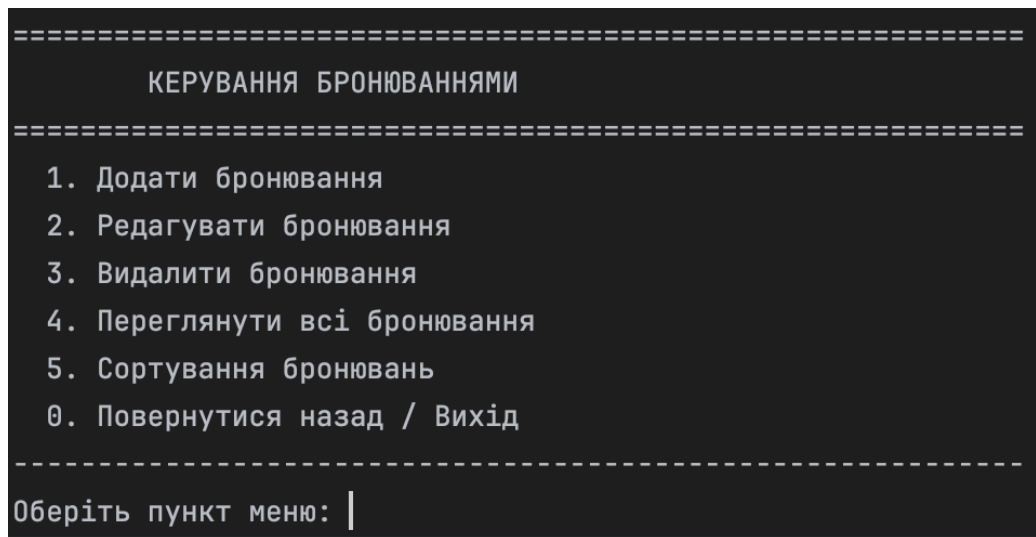


Рисунок 18 – Меню керування бронюваннями

- Додати бронювання: Процес складається з кількох етапів: система показує вільні готелі - користувач обирає гостя - обирає вільний номер - вводить дати. Система автоматично перевіряє конфлікти дат.
- Редагування та Видалення: Можливість змінити параметри існуючої броні або скасувати її.
- Сортування: Перегляд бронювань, впорядкованих за датою заїзду/виїзду або за ID гостя (рис. 19).

```

✓ Бронювання відсортовано за ID гостя!

=====
СПИСОК БРОНЮВАНЬ (Всього: 3)
=====

┌────────────────────────────────────────────────────────────────────────────────┐
│                                ІНФОРМАЦІЯ ПРО БРОНЮВАННЯ                                │
├────────────────────────────────────────────────────────────────────────────────┤
│ ID бронювання: 1                                                            │
│ ID гостя: 1                                                                  │
│ ID номера: 1                                                                │
│ Дата заїзду: 2025-12-02                                                    │
│ Дата виїзду: 2025-12-20                                                    │
│ Кількість днів: 18                                                          │
└────────────────────────────────────────────────────────────────────────────────┘

┌────────────────────────────────────────────────────────────────────────────────┐
│                                ІНФОРМАЦІЯ ПРО БРОНЮВАННЯ                                │
├────────────────────────────────────────────────────────────────────────────────┤
│ ID бронювання: 3                                                            │
│ ID гостя: 2                                                                  │
│ ID номера: 7                                                                │
│ Дата заїзду: 2026-01-04                                                    │
│ Дата виїзду: 2026-01-10                                                    │
│ Кількість днів: 6                                                          │
└────────────────────────────────────────────────────────────────────────────────┘

```

Рисунок 19 – Результат сортування бронювань за ID гостя

7. Допомога

Пункт «6. Допомога» виводить коротку довідку з описом функцій програми (рис. 20).

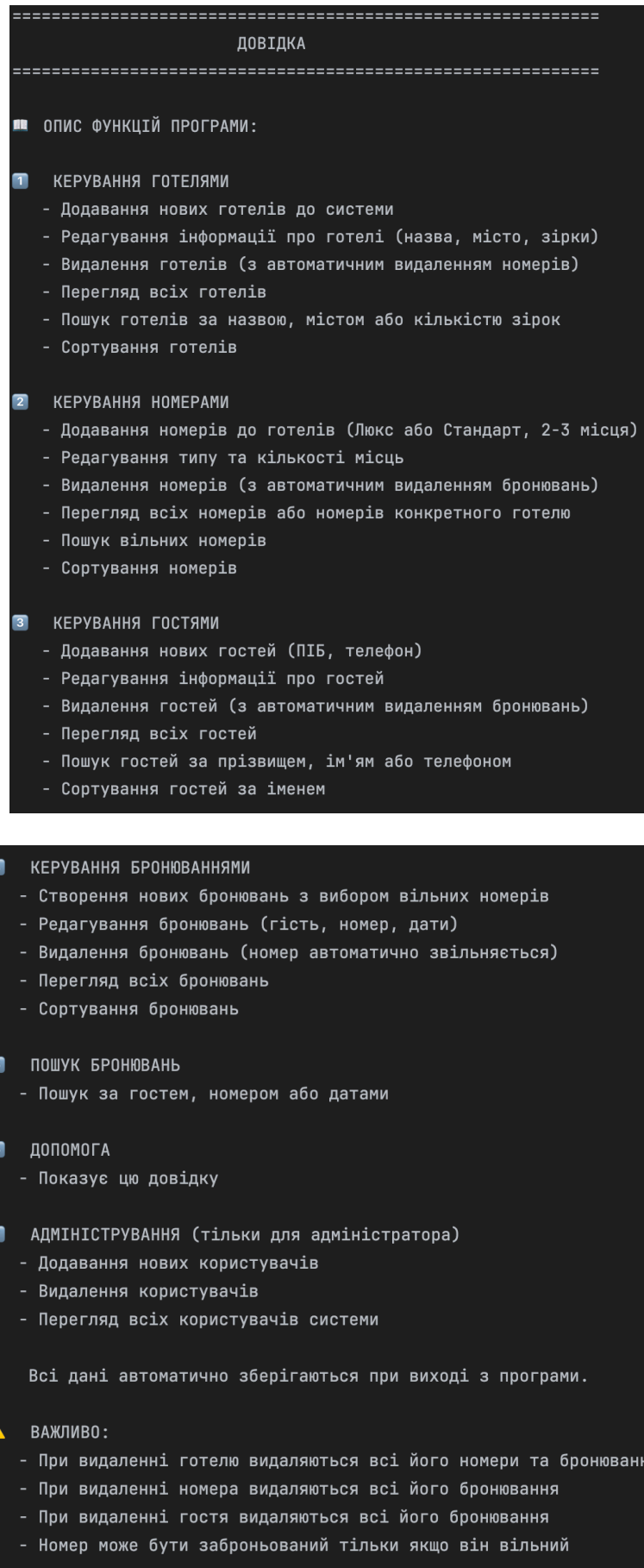


Рисунок 20 – Результат довідки

8. Адміністрування (рис. 21)

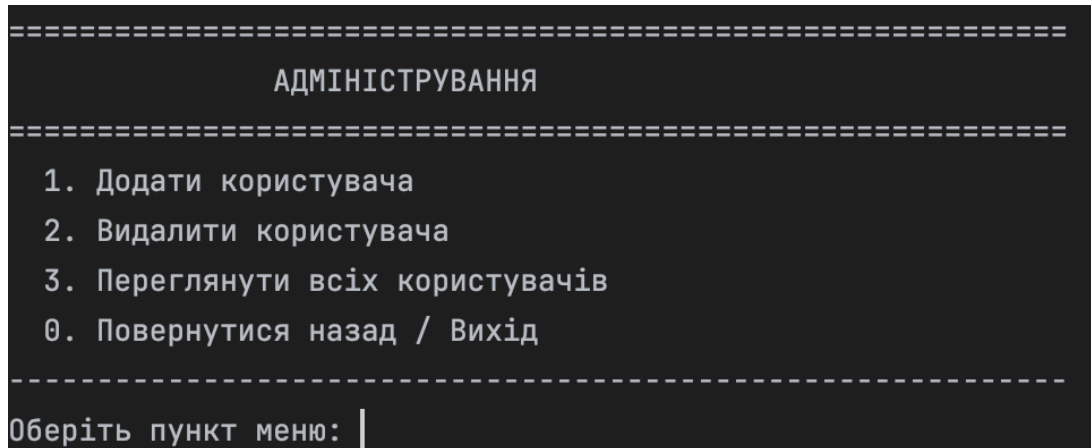


Рисунок 21 – Меню адміністрування

Дозволяє додавати нових користувачів системи, видаляти їх та переглядати список облікових записів (рис. 22).

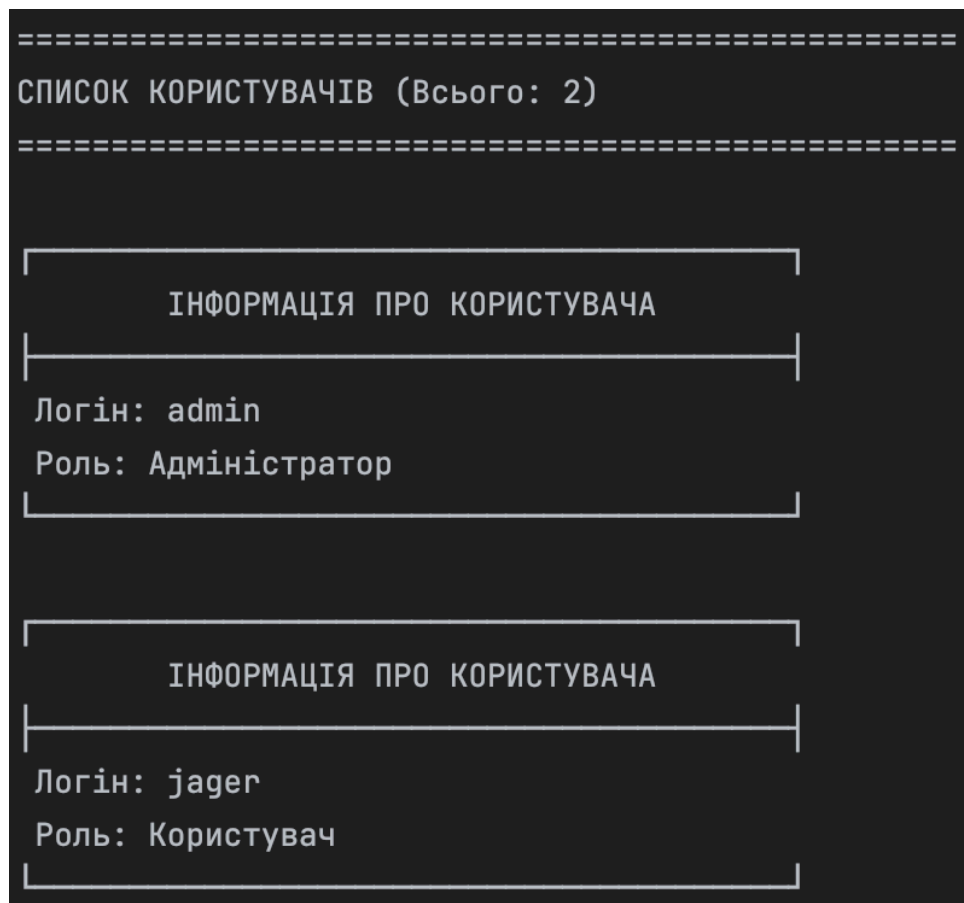


Рисунок 22 – Список користувачів

2.4. Тестування програмної системи

В цьому розділі описано процес функціонального тестування, яке перевіряє, як програма виконує свої функції в умовах коректних та некоректних дій користувача.

Тест авторизації користувача: Перевірити, чи можуть авторизовані користувачі увійти в систему та отримати доступ до переліку функціональних можливостей додатку.

Кроки тестування:

- Ввести коректні дані для авторизації (логін та пароль) та спробувати увійти в систему.

```
=====
                        АВТОРИЗАЦІЯ
=====
Логін:  admin
Пароль:  admin

✅ Ви успішно увійшли в систему!

Вітаємо, admin!
Роль: Адміністратор

Натисніть Enter для продовження...|
```

Рисунок 23

```
=====
                        ГОЛОВНЕ МЕНЮ
=====
1. Керування готелями
2. Керування номерами
3. Керування гостями
4. Керування бронюваннями
5. Пошук бронювань
6. Допомога
7. Адміністрування
0. Повернутися назад / Вихід
-----
Оберіть пункт меню: |
```

Рисунок 24

– Ввести некоректні дані для авторизації (логін та пароль) та спробувати увійти в систему. Навести exception-и, які видає система.

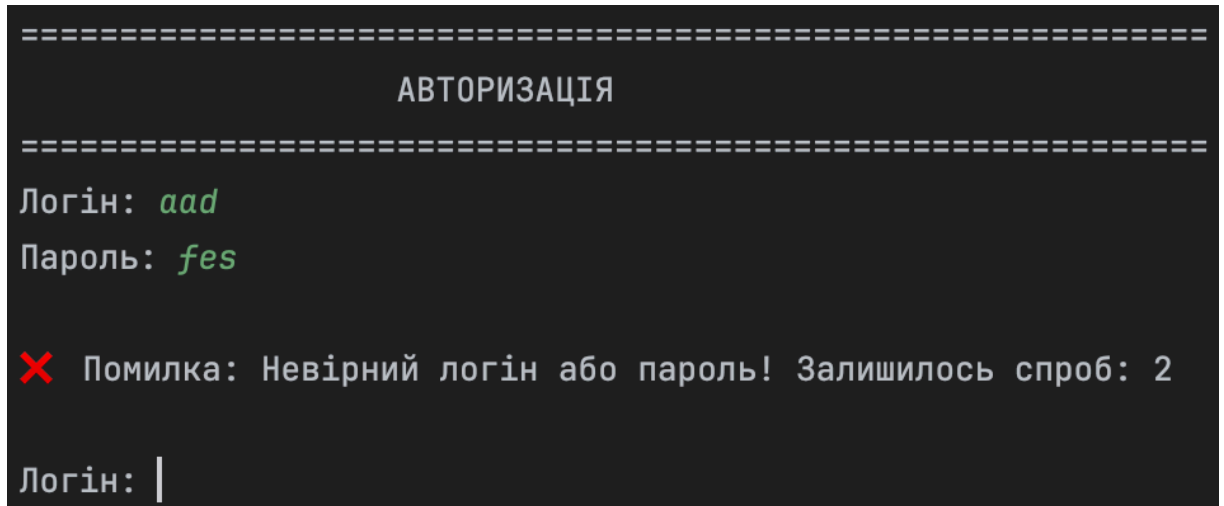


Рисунок 25

Тест на додавання нового об'єкта: Перевірити, чи може користувач додати новий об'єкт.

Кроки тестування:

- Перейти до вікна додавання нового об'єкта.
- Заповнити всі необхідні поля та зберегти.

```

=====
                        КЕРУВАННЯ ГОТЕЛЯМИ
=====

1. Додати готель
2. Редагувати готель
3. Видалити готель
4. Переглянути всі готелі
5. Пошук готелів
6. Сортування готелів
0. Повернутися назад / Вихід

-----
Оберіть пункт меню: 1

=====
                        ДОДАТИ ГОТЕЛЬ
=====

Назва готелю: VIVA Hotel
Місто: Харків
Кількість зірок (1-5): 5

✅ Готель успішно додано!

Натисніть Enter для продовження...|

```

Рисунок 26

```

┌
ID: 6
Назва: VIVA Hotel
Місто: Харків
Зірок: ★★★★★
└

```

Рисунок 27

– Не заповнити деякі необхідні поля і натиснути зберегти. Навести exception-и, які видає система.

```
=====
                        КЕРУВАННЯ ГОТЕЛЯМИ
=====

1. Додати готель
2. Редагувати готель
3. Видалити готель
4. Переглянути всі готелі
5. Пошук готелів
6. Сортування готелів
0. Повернутися назад / Вихід

-----
Оберіть пункт меню: 1

=====
                        ДОДАТИ ГОТЕЛЬ
=====

Назва готелю:

✗ Помилка: Рядок не може бути порожнім!

Назва готелю: |
```

Рисунок 28

Тест на редагування даних об'єкта: Перевірити, чи може користувач змінювати інформацію про об'єкт.

Кроки тестування:

- Перейти до вікна редагування об'єкта.
- Змінити деяке поле, ввівши коректні дані, і зберегти зміни.

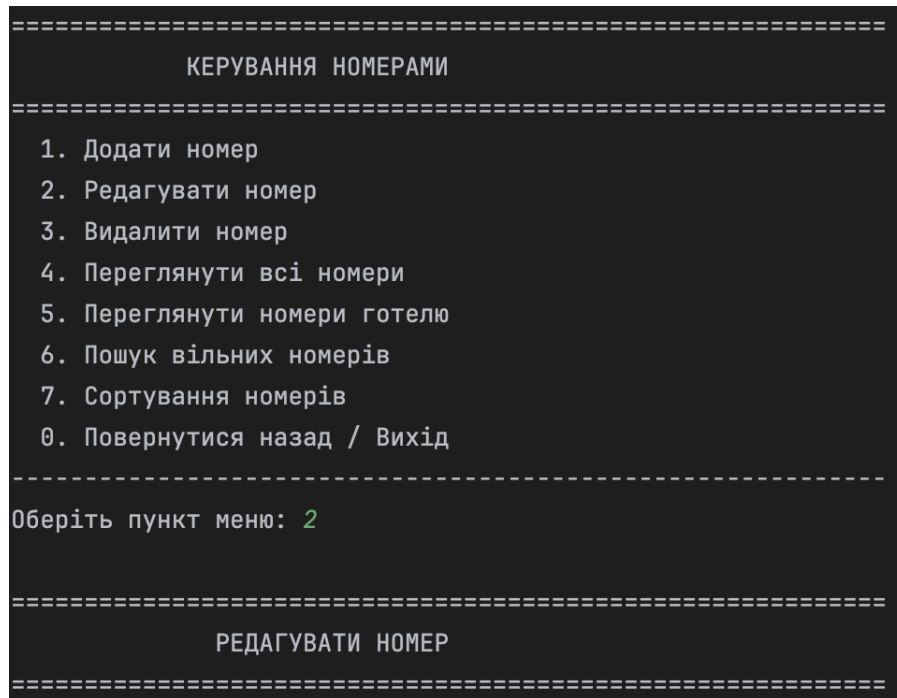


Рисунок 29

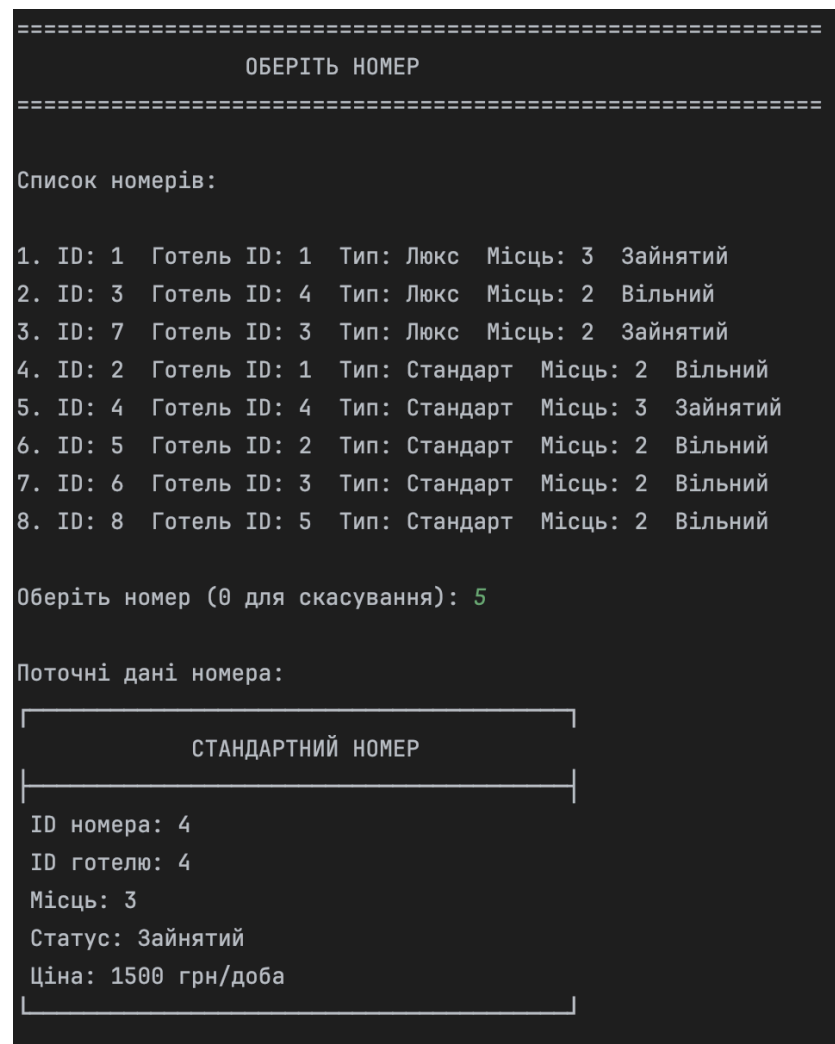


Рисунок 30

```
Що редагувати?  
1. Тип номера  
2. Кількість місць  
3. Все  
0. Скасувати  
Ваш вибір: 2  
Нова кількість місць (2 або 3): 2  
  
✅ Номер успішно відредаговано!  
  
Натисніть Enter для продовження...
```

Рисунок 31

– Внести в деяке поле некоректне значення і натиснути зберегти.
Навести exception-и, які видає система.


```

=====
                                ОБЕРІТЬ НОМЕР
=====

Список номерів:

1. ID: 1  Готель ID: 1  Тип: Люкс  Місце: 3  Зайнятий
2. ID: 3  Готель ID: 4  Тип: Люкс  Місце: 2  Вільний
3. ID: 7  Готель ID: 3  Тип: Люкс  Місце: 2  Зайнятий
4. ID: 2  Готель ID: 1  Тип: Стандарт  Місце: 2  Вільний
5. ID: 4  Готель ID: 4  Тип: Стандарт  Місце: 2  Вільний
6. ID: 5  Готель ID: 2  Тип: Стандарт  Місце: 2  Вільний
7. ID: 6  Готель ID: 3  Тип: Стандарт  Місце: 2  Вільний
8. ID: 8  Готель ID: 5  Тип: Стандарт  Місце: 2  Вільний

Оберіть номер (0 для скасування): 9

❌ Помилка: Число має бути в діапазоні від 0 до 8

Оберіть номер (0 для скасування): |

```

Рисунок 32

Тест на видалення об'єкта: Перевірити, чи може користувач видаляти об'єкт.
Кроки тестування:

- Вибрати об'єкт для видалення та підтвердити видалення.

```

=====
                        КЕРУВАННЯ ГОСТЯМИ
=====

1. Додати гостя
2. Редагувати гостя
3. Видалити гостя
4. Переглянути всіх гостей
5. Пошук гостей
6. Сортювання гостей
0. Повернутися назад / Вихід

-----
Оберіть пункт меню: 3

=====
                        ВИДАЛИТИ ГОСТЯ
=====

=====
                        ОБЕРІТЬ ГОСТЯ
=====

Список гостей:

1. Дем'янчук Настя (Тел: 0989853936)
2. Дорофтей Катя (Тел: 0990128525)
3. Кирилюк Макс (Тел: 0956124946)
4. Мельник Ваня (Тел: 0955898692)

Оберіть гостя (0 для скасування): 3

```

Рисунок 33

```

Гість, якого буде видалено:

┌────────────────────────────────────────────────────────────────────────────────┐
│                                ІНФОРМАЦІЯ ПРО ГОСТЯ                                │
└────────────────────────────────────────────────────────────────────────────────┘
ID: 4
Прізвище: Кирилюк
Ім'я: Макс
Телефон: 0956124946

⚠ Попередження: При видаленні гостя будуть також видалені всі його бронювання!

Ви впевнені, що хочете видалити цього гостя? (так/ні): так

✅ Гостя успішно видалено! Також видалено бронювань: 0

Натисніть Enter для продовження...

```

Рисунок 34

2.5. Програмні засоби розробки

Розробка програмної системи «Система бронювання номерів в готелі» була реалізована засобами інтегрованого середовища розробки (IDE) CLion від компанії JetBrains. Це середовище було обрано завдяки його потужним інструментам для аналізу коду, зручному налагоджувачу та нативній підтримці системи збірки CMake. Робота над проектом велася в операційній системі MacOS.

В якості мови програмування обрано C++ (стандарт C++17). Цей стандарт надає сучасні можливості для написання безпечного та ефективного коду, зберігаючи повну підтримку принципів об'єктно-орієнтованого програмування (інкапсуляція, спадкування, поліморфізм).

Для автоматизації процесу компіляції та збірки проекту використано систему CMake. Вона дозволяє гнучко налаштовувати параметри збірки та забезпечує кросплатформність коду (проект може бути скомпільований як на macOS, так і на Windows чи Linux).

Для реалізації функціоналу системи широко використовувалася Стандартна бібліотека шаблонів (STL):

- Контейнери: `std::vector` для динамічного зберігання списків готелів, номерів, гостей та бронювань.
- Ввід/Вивід: Бібліотеки `<iostream>` та `<iomanip>` для організації зручного консольного інтерфейсу та форматowanego виводу таблиць.
- Робота з файлами: Бібліотека `<fstream>` для реалізації механізму збереження та зчитування даних у форматі CSV (`data_hotels.csv` тощо).
- Алгоритми: Бібліотека `<algorithm>` для реалізації сортування (`std::sort`), пошуку та фільтрації даних.
- Рядки: Клас `std::string` та потоки `<sstream>` для обробки текстових даних та парсингу CSV-рядків.

Для забезпечення коректного відображення інтерфейсу (очищення екрана) було реалізовано кросплатформну функцію, що використовує відповідні системні команди (`clear` для macOS/Linux).

ВИСНОВКИ

У ході виконання курсового проєкту було розроблено програмну систему для автоматизації бронювання номерів у готелях. Робота виконувалася з використанням мови програмування C++ та об'єктно-орієнтованого підходу.

У результаті роботи було виконано наступні завдання:

1. Проаналізовано предметну область та сформульовано вимоги до системи. Визначено ключові сутності (готель, номер, гість, бронювання) та зв'язки між ними.
2. Спроектовано архітектуру програми, яка базується на взаємодії класів-сутностей та класів-менеджерів. Використано принципи інкапсуляції, спадкування.
3. Реалізовано механізм збереження даних у текстові файли формату CSV, що забезпечує збереження інформації між запусками програми.
4. Розроблено консольний інтерфейс користувача, який забезпечує зручну навігацію через систему меню та підтримує українську мову.
5. Реалізовано ключовий функціонал:
 - Автентифікація користувачів із розмежуванням прав доступу (Адміністратор/Користувач).
 - Повний набір CRUD-операцій (створення, читання, оновлення, видалення) для всіх типів даних.
 - Реалізовано механізм часткового редагування, що дозволяє змінювати лише окремі поля об'єктів.
 - Впроваджено каскадне видалення даних для забезпечення цілісності бази (при видаленні готелю автоматично видаляються пов'язані номери та бронювання).
 - Розроблено алгоритм пошуку вільних номерів за складними критеріями (місто, місткість, діапазон дат) з автоматичною перевіркою перетинів у календарі бронювань.
6. Забезпечено надійність роботи програми шляхом валідації введених даних (перевірка числових полів, форматів дат, номерів телефонів) та обробки виключних ситуацій.

Проведене функціональне тестування підтвердило коректність роботи всіх модулів системи. Розроблений програмний продукт повністю відповідає поставленому технічному завданню.

Дана розробка у майбутньому може бути розширена шляхом додавання графічного інтерфейсу, інтеграції з платіжними системами або переведенням бази даних на SQL-сервер.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Microsoft Learn: <https://learn.microsoft.com/en-us/cpp>
- 2) cppreference.com: <https://en.cppreference.com/>
- 3) Doxygen Manual: <https://www.doxygen.nl/manual/index.html>
- 4) STL documentation: <https://cplusplus.com/reference>
- 5) Bjarne Stroustrup, "The C++ Programming Language," 4th Edition, Addison-Wesley, 2013.
- 6) Scott Meyers, "Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14," O'Reilly Media, 2014.
- 7) Щербаков О. В. Основи об'єктно-орієнтованого програмування: навч. посіб. – Харків: ХНЕУ ім. С. Кузнеця, 2019.
- 8) Комісарчук В. В. Методичні рекомендації до виконання курсового проєкту з дисципліни «Структури даних, алгоритми та об'єктно-орієнтована парадигма» / Укладачі: В. В. Комісарчук, О. О. Валь. – Чернівці: ЧНУ, 2025.

ДОДАТКИ

ДОДАТОК А. Скролінг (текст) програми

Програмний модуль «Main»

– Вміст Main.cpp

```

/*
 * @file main.cpp
 * @brief Головний файл програми - Система бронювання номерів в готелях
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Constants.h"
#include "../include/HotelManager.h"
#include "../include/RoomManager.h"
#include "../include/GuestManager.h"
#include "../include/BookingManager.h"
#include "../include/UserManager.h"
#include "../include/Menu.h"
#include <iostream>

/**
 * @brief Головна функція програми
 * @return Код завершення програми (0 - успішно)
 */
int main() {
    try {
        // Ініціалізація менеджерів
        HotelManager hotelManager(Constants::Files::HOTELS);
        RoomManager roomManager(Constants::Files::ROOMS);
        GuestManager guestManager(Constants::Files::GUESTS);
        BookingManager bookingManager(Constants::Files::BOOKINGS);
        UserManager userManager(Constants::Files::USERS);

        // Створення та запуск меню
        Menu menu(hotelManager, roomManager, guestManager, bookingManager, userManager);
        menu.run();

        return 0;
    } catch (const std::exception& e) {
        std::cerr << "Критична помилка: " << e.what() << std::endl;
        return 1;
    }
}

```

Програмний модуль «Константи та Утиліти»

– Вміст Constants.h

```

/**
 * @file Constants.h
 * @brief Файл з константами для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef CONSTANTS_H
#define CONSTANTS_H

#include <string>

/**
 * @namespace Constants
 * @brief Простір імен для констант програми
 */
namespace Constants {

    // ===== ШЛЯХИ ДО ФАЙЛІВ =====
    // (використовуються у багатьох Manager класах)

```

```

namespace Files {
    const std::string USERS = "users.txt";
    const std::string HOTELS = "data_hotels.csv";
    const std::string ROOMS = "data_rooms.csv";
    const std::string GUESTS = "data_guests.csv";
    const std::string BOOKINGS = "data_bookings.csv";
}

// ===== МЕЖИ ТА ОБМЕЖЕННЯ =====
// (повторюються у валідації у різних класах)

namespace Limits {
    const int MIN_STARS = 1;
    const int MAX_STARS = 5;
    const int MIN_CAPACITY = 2;
    const int MAX_CAPACITY = 3;
    const int MIN_USERNAME_LEN = 3;
    const int MIN_PASSWORD_LEN = 4;
    const int MIN_PHONE_DIGITS = 10;
    const int DATE_LENGTH = 10;
    const int MAX_LOGIN_ATTEMPTS = 3;
}

// ===== ЦІНИ (базові значення) =====

namespace Prices {
    const double LUXURY_BASE = 2500.0;
    const double STANDARD_BASE = 1200.0;
    const double LUXURY_EXTRA_PERSON = 500.0;
    const double STANDARD_EXTRA_PERSON = 300.0;
}

// ===== ПОВТОРЮВАНІ ПОВІДОМЛЕННЯ =====
// (ті що зустрічаються у ДЕКІЛЬКОХ місцях коду)

namespace Messages {
    // Помилки які повторюються
    const std::string ERR_FILE_OPEN = "Не вдалося відкрити файл: ";
    const std::string ERR_FILE_SAVE = "Не вдалося відкрити файл для запису: ";
    const std::string ERR_LOAD_DATA = "Помилка при завантаженні даних: ";
    const std::string ERR_SAVE_DATA = "Помилка при збереженні даних: ";
    const std::string ERR_CSV_PARSE = "Помилка парсингу CSV: ";
    const std::string ERR_INVALID_DATA = "Некоректні дані";

    // Формати дат та роздільники
    const std::string DATE_FORMAT = "YYYY-MM-DD";
    const char CSV_DELIMITER = ',';
    const char USER_DELIMITER = ':';
}

// ===== ТИПИ НОМЕРІВ =====

namespace RoomType {
    const std::string LUXURY = "Люкс";
    const std::string STANDARD = "Стандарт";
}

// ===== КОРИСТУВАЧ ЗА ЗАМОВЧУВАННЯМ =====

namespace DefaultUser {
    const std::string ADMIN_LOGIN = "admin";
    const std::string ADMIN_PASSWORD = "admin";
}

} // namespace Constants

#endif // CONSTANTS_H

```

– Вміст файлу Utils.h

```

/**
 * @file Utils.h
 * @brief Заголовковий файл допоміжних функцій для роботи програми
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef UTILS_H
#define UTILS_H

#include <string>
#include <iostream>
#include <limits>

/**
 * @namespace Utils
 * @brief Простір імен для допоміжних функцій
 */
namespace Utils {

    /**
     * @brief Очистити екран консолі
     */
    void clearScreen();

    /**
     * @brief Пауза - чекати натискання Enter
     * @param message Повідомлення для користувача
     */
    void pause(const std::string& message = "Намисніть Enter для продовження...");

    /**
     * @brief Вивести заголовок
     * @param title Текст заголовку
     */
    void printHeader(const std::string& title);

    /**
     * @brief Вивести розділювач
     * @param length Довжина розділювача
     */
    void printSeparator(int length = 50);

    /**
     * @brief Безпечне введення цілого числа
     * @param prompt Текст запити
     * @param min Мінімальне значення
     * @param max Максимальне значення
     * @return Введене число
     */
    int getIntInput(const std::string& prompt, int min = 0,
                    int max = std::numeric_limits<int>::max());

    /**
     * @brief Безпечне введення рядка
     * @param prompt Текст запити
     * @param allowEmpty Чи дозволяти порожній рядок
     * @return Введений рядок
     */
    std::string getStringInput(const std::string& prompt, bool allowEmpty = false);

    /**
     * @brief Запитати підтвердження (так/ні)
     * @param prompt Текст запити
     * @return true якщо користувач підтвердив
     */
    bool getConfirmation(const std::string& prompt);

```



```

/**
 * @brief Перевірити валідність дати
 * @param date Дата у форматі YYYY-MM-DD
 * @return true якщо дата валідна
 */
bool isValidDate(const std::string& date);

/**
 * @brief Отримати дату від користувача
 * @param prompt Текст запиту
 * @return Дата у форматі YYYY-MM-DD
 */
std::string getDateInput(const std::string& prompt);

/**
 * @brief Очистити буфер введення
 */
void clearInputBuffer();

/**
 * @brief Вивести повідомлення про успіх
 * @param message Текст повідомлення
 */
void printSuccess(const std::string& message);

/**
 * @brief Вивести повідомлення про помилку
 * @param message Текст повідомлення
 */
void printError(const std::string& message);

/**
 * @brief Вивести попередження
 * @param message Текст повідомлення
 */
void printWarning(const std::string& message);

/**
 * @brief Вивести інформаційне повідомлення
 * @param message Текст повідомлення
 */
void printInfo(const std::string& message);

/**
 * @brief Перевести рядок у нижній регістр
 * @param str Вхідний рядок
 * @return Рядок у нижньому регістрі
 */
std::string toLowerCase(const std::string& str);

/**
 * @brief Перевести рядок у верхній регістр
 * @param str Вхідний рядок
 * @return Рядок у верхньому регістрі
 */
std::string toUpperCase(const std::string& str);

/**
 * @brief Обрізати пробіли з початку та кінця рядка
 * @param str Вхідний рядок
 * @return Обрізаний рядок
 */
std::string trim(const std::string& str);

/**
 * @brief Отримати поточну дату у форматі YYYY-MM-DD
 * @return Поточна дата
 */
std::string getCurrentDate();

```

```

/**
 * @brief Вивести ASCII лого системи
 */
void printLogo();

/**
 * @brief Вивести меню вибору (з можливістю вибору пункту)
 * @param title Заголовок меню
 * @param options Варіанти вибору
 * @return Номер обраного пункту (1-based) або 0 для виходу
 */
int displayMenu(const std::string& title, const std::vector<std::string>& options);

} // namespace Utils

#endif // UTILS_H

```

– Вміст файлу Utils.cpp

```

/**
 * @file Utils.cpp
 * @brief Реалізація допоміжних функцій для роботи програми
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Utils.h"
#include <algorithm>
#include <cctype>
#include <sstream>
#include <ctime>

#ifdef _WIN32
#include <windows.h>
#endif

namespace Utils {

/**
 * @brief Очистити екран консолі
 */
void clearScreen() {
#ifdef _WIN32
system("cls");
#else
// macOS, Linux та інші Unix системи
system("clear");
#endif
}

/**
 * @brief Пауза - чекати натискання Enter
 */
void pause(const std::string& message) {
std::cout << "\n" << message;
clearInputBuffer();
std::cin.get();
}

/**
 * @brief Вивести заголовок
 */
void printHeader(const std::string& title) {
int width = 60;
int padding = (width - title.length() - 2) / 2;

std::cout << "\n" << std::string(width, '=') << std::endl;
std::cout << std::string(padding, ' ') << " " << title << std::endl;
std::cout << std::string(width, '=') << std::endl;
}
}

```

```

/**
 * @brief Вивести розділювач
 */
void printSeparator(int length) {
    std::cout << std::string(length, '-') << std::endl;
}

/**
 * @brief Безпечне введення цілого числа
 */
int getIntInput(const std::string& prompt, int min, int max) {
    int value;
    while (true) {
        std::cout << prompt;

        if (std::cin >> value) {
            if (value >= min && value <= max) {
                clearInputBuffer();
                return value;
            } else {
                printError("Число має бути в діапазоні від " +
                    std::to_string(min) + " до " + std::to_string(max));
            }
        } else {
            printError("Некоректне введення! Введіть число.");
            std::cin.clear();
            clearInputBuffer();
        }
    }
}

/**
 * @brief Безпечне введення рядка
 */
std::string getStringInput(const std::string& prompt, bool allowEmpty) {
    std::string input;
    while (true) {
        std::cout << prompt;
        std::getline(std::cin, input);
        input = trim(input);

        if (!input.empty() || allowEmpty) {
            return input;
        } else {
            printError("Рядок не може бути порожнім!");
        }
    }
}

/**
 * @brief Запитати підтвердження
 */
bool getConfirmation(const std::string& prompt) {
    std::string input;
    std::cout << prompt << " (так/ні): ";
    std::getline(std::cin, input);
    input = toLowerCase(trim(input));

    return (input == "так" || input == "yes" || input == "y" ||
        input == "м" || input == "I");
}

/**
 * @brief Перевірити валідність дати
 */
bool isValidDate(const std::string& date) {
    if (date.length() != 10) return false;
    if (date[4] != '-' || date[7] != '-') return false;
}

```

```

try {
    int year = std::stoi(date.substr(0, 4));
    int month = std::stoi(date.substr(5, 2));
    int day = std::stoi(date.substr(8, 2));

    if (year < 2020 || year > 2100) return false;
    if (month < 1 || month > 12) return false;
    if (day < 1 || day > 31) return false;

    // Перевірка днів у місяці
    if (month == 2) {
        bool isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
        if (day > (isLeap ? 29 : 28)) return false;
    } else if (month == 4 || month == 6 || month == 9 || month == 11) {
        if (day > 30) return false;
    }

    return true;
} catch (...) {
    return false;
}
}

/**
 * @brief Отримати дату від користувача
 */
std::string getDateInput(const std::string& prompt) {
    std::string date;
    while (true) {
        std::cout << prompt << " (формат: YYYY-MM-DD): ";
        std::getline(std::cin, date);
        date = trim(date);

        if (isValidDate(date)) {
            return date;
        } else {
            printError("Некоректна дата! Використовуйте формат YYYY-MM-DD");
        }
    }
}

/**
 * @brief Очистити буфер введення
 */
void clearInputBuffer() {
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}

/**
 * @brief Вивести повідомлення про успіх
 */
void printSuccess(const std::string& message) {
    std::cout << "\n✅" << message << "\n" << std::endl;
}

/**
 * @brief Вивести повідомлення про помилку
 */
void printError(const std::string& message) {
    std::cout << "\n❌ Помилка: " << message << "\n" << std::endl;
}

/**
 * @brief Вивести попередження
 */
void printWarning(const std::string& message) {
    std::cout << "\n⚠️ Попередження: " << message << "\n" << std::endl;
}
}

```

```

/**
 * @brief Вивести інформаційне повідомлення
 */
void printInfo(const std::string& message) {
    std::cout << "\n❗ " << message << "\n" << std::endl;
}

/**
 * @brief Перевести рядок у нижній регістр
 */
std::string toLowerCase(const std::string& str) {
    std::string result = str;
    std::transform(result.begin(), result.end(), result.begin(), ::tolower);
    return result;
}

/**
 * @brief Перевести рядок у верхній регістр
 */
std::string toUpperCase(const std::string& str) {
    std::string result = str;
    std::transform(result.begin(), result.end(), result.begin(), ::toupper);
    return result;
}

/**
 * @brief Обрізати пробіли з початку та кінця рядка
 */
std::string trim(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\n\r");
    if (first == std::string::npos) return "";

    size_t last = str.find_last_not_of(" \t\n\r");
    return str.substr(first, (last - first + 1));
}

/**
 * @brief Отримати поточну дату
 */
std::string getCurrentDate() {
    time_t now = time(0);
    tm* ltm = localtime(&now);

    std::ostringstream oss;
    oss << (1900 + ltm->tm_year) << "-"
        << (ltm->tm_mon + 1 < 10 ? "0" : "") << (ltm->tm_mon + 1) << "-"
        << (ltm->tm_mday < 10 ? "0" : "") << ltm->tm_mday;

    return oss.str();
}

/**
 * @brief Вивести ASCII лого системи
 */
void printLogo() {
    std::cout << R"(


СИСТЕМА БРОНЮВАННЯ НОМЕРІВ В ГОТЕЛЯХ


)" << std::endl;
}

/**
 * @brief Вивести меню вибору
 */
int displayMenu(const std::string& title, const std::vector<std::string>& options) {
    printHeader(title);

```

```

    for (size_t i = 0; i < options.size(); i++) {
        std::cout << " " << (i + 1) << ". " << options[i] << std::endl;
    }
    std::cout << " 0. Повернутися назад / Вихід" << std::endl;

    printSeparator(60);

    int choice = getIntInput("Оберіть пункт меню: ", 0, options.size());
    return choice;
}

} // namespace Utils

```

Програмний модуль «Hotel»

– Вміст Hotel.h

```

/**
 * @file Hotel.h
 * @brief Заголовковий файл класу Hotel для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef HOTEL_H
#define HOTEL_H

#include <string>
#include <iostream>

/**
 * @class Hotel
 * @brief Клас для представлення готелю
 *
 * Цей клас зберігає інформацію про готель: назву, місто та кількість зірок.
 * Підтримує операції додавання, редагування та видалення готелів.
 */
class Hotel {
private:
    int id;           ///< Унікальний ідентифікатор готелю
    std::string name;  ///< Назва готелю
    std::string city;  ///< Місто розташування
    int stars;         ///< Кількість зірок (1-5)

    static int nextId;  ///< Лічильник для генерації ID

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    Hotel();

    /**
     * @brief Конструктор з параметрами
     * @param name Назва готелю
     * @param city Місто розташування
     * @param stars Кількість зірок
     */
    Hotel(const std::string& name, const std::string& city, int stars);

    /**
     * @brief Конструктор з усіма параметрами (включно з ID)
     * @param id Ідентифікатор готелю
     * @param name Назва готелю
     * @param city Місто розташування
     * @param stars Кількість зірок
     */
    Hotel(int id, const std::string& name, const std::string& city, int stars);
}

```

```

* @brief Копіювальний конструктор
* @param other Інший об'єкт Hotel для копіювання
*/
Hotel(const Hotel& other);

/**
* @brief Переміщувальний конструктор
* @param other Інший об'єкт Hotel для переміщення
*/
Hotel(Hotel&& other) noexcept;

/**
* @brief Деструктор
*/
~Hotel();

/**
* @brief Оператор присвоєння копіюванням
* @param other Інший об'єкт Hotel
* @return Посилання на поточний об'єкт
*/
Hotel& operator=(const Hotel& other);

/**
* @brief Оператор присвоєння переміщенням
* @param other Інший об'єкт Hotel
* @return Посилання на поточний об'єкт
*/
Hotel& operator=(Hotel&& other) noexcept;

// Геттери
/**
* @brief Отримати ID готелю
* @return ID готелю
*/
int getId() const;

/**
* @brief Отримати назву готелю
* @return Назва готелю
*/
std::string getName() const;

/**
* @brief Отримати місто
* @return Місто розташування
*/
std::string getCity() const;

/**
* @brief Отримати кількість зірок
* @return Кількість зірок
*/
int getStars() const;

// Сеттери
/**
* @brief Встановити ID готелю
* @param id Новий ID
*/
void setId(int id);

/**
* @brief Встановити назву готелю
* @param name Нова назва
*/
void setName(const std::string& name);

/**
* @brief Встановити місто

```

```

    * @param city Нове місто
    */
    void setCity(const std::string& city);

    /**
    * @brief Встановити кількість зірок
    * @param stars Нова кількість зірок (1-5)
    * @throw std::invalid_argument якщо кількість зірок поза межами 1-5
    */
    void setStars(int stars);

    // Методи
    /**
    * @brief Вивести інформацію про готель
    */
    void display() const;

    /**
    * @brief Перевірити валідність даних готелю
    * @return true якщо дані валідні, false інакше
    */
    bool validate() const;

    /**
    * @brief Конвертувати об'єкт у рядок CSV формату
    * @return Рядок у форматі CSV
    */
    std::string toCSV() const;

    /**
    * @brief Створити об'єкт Hotel з рядка CSV формату
    * @param csvLine Рядок у форматі CSV
    * @return Об'єкт Hotel
    */
    static Hotel fromCSV(const std::string& csvLine);

    /**
    * @brief Оновити наступний доступний ID
    * @param id Значення ID
    */
    static void updateNextId(int id);
};

#endif // HOTEL_H

```

– Вміст Hotel.cpp

```

/**
 * @file Hotel.cpp
 * @brief Реалізація класу Hotel для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Hotel.h"
#include "../include/Constants.h"
#include <sstream>
#include <stdexcept>
#include <iomanip>

// Ініціалізація статичної змінної
int Hotel::nextId = 1;

/**
 * @brief Конструктор за замовчуванням
 */
Hotel::Hotel() : id(0), name(""), city(""), stars(0) {
    // Порожній конструктор
}

```



```

/**
 * @brief Конструктор з параметрами
 */
Hotel::Hotel(const std::string& name, const std::string& city, int stars)
: id(nextId++), name(name), city(city), stars(stars) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані готелю");
    }
}

/**
 * @brief Конструктор з усіма параметрами
 */
Hotel::Hotel(int id, const std::string& name, const std::string& city, int stars)
: id(id), name(name), city(city), stars(stars) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані готелю");
    }
}

/**
 * @brief Копіювальний конструктор
 */
Hotel::Hotel(const Hotel& other)
: id(other.id), name(other.name), city(other.city), stars(other.stars) {
    // Конструктор копіювання
}

/**
 * @brief Переміщувальний конструктор
 */
Hotel::Hotel(Hotel&& other) noexcept
: id(other.id), name(std::move(other.name)),
  city(std::move(other.city)), stars(other.stars) {
    other.id = 0;
    other.stars = 0;
}

/**
 * @brief Деструктор
 */
Hotel::~Hotel() {
    // Деструктор - виведення повідомлення про знищення
    if (id != 0) {
        // std::cout << "Готель " << name << " (ID: " << id << ") знищено\n";
    }
}

/**
 * @brief Оператор присвоєння копіюванням
 */
Hotel& Hotel::operator=(const Hotel& other) {
    if (this != &other) {
        id = other.id;
        name = other.name;
        city = other.city;
        stars = other.stars;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
Hotel& Hotel::operator=(Hotel&& other) noexcept {
    if (this != &other) {
        id = other.id;
        name = std::move(other.name);
        city = std::move(other.city);
        stars = other.stars;
    }
}

```

```

        other.id = 0;
        other.stars = 0;
    }
    return *this;
}

// Геммепу
int Hotel::getId() const {
    return id;
}

std::string Hotel::getName() const {
    return name;
}

std::string Hotel::getCity() const {
    return city;
}

int Hotel::getStars() const {
    return stars;
}

// Сеттер
void Hotel::setId(int id) {
    this->id = id;
}

void Hotel::setName(const std::string& name) {
    this->name = name;
}

void Hotel::setCity(const std::string& city) {
    this->city = city;
}

void Hotel::setStars(int stars) {
    if (stars < Constants::Limits::MIN_STARS || stars > Constants::Limits::MAX_STARS) {
        throw std::invalid_argument("Кількість зірок має бути від 1 до 5");
    }
    this->stars = stars;
}

/**
 * @brief Вивести інформацію про готель
 */
void Hotel::display() const {
    std::cout << "┌──────────────────────────────────────────────────────────────────────────────────┐ \n";
    std::cout << "ID: " << std::left << std::setw(35) << id << "\n";
    std::cout << "Назва: " << std::left << std::setw(32) << name << "\n";
    std::cout << "Місто: " << std::left << std::setw(32) << city << "\n";
    std::cout << "Зірок: ";
    for (int i = 0; i < stars; i++) {
        std::cout << "★";
    }
    for (int i = stars; i < 5; i++) {
        std::cout << "☆";
    }
    std::cout << std::setw(32 - stars * 3) << " " << "\n";
    std::cout << "└──────────────────────────────────────────────────────────────────────────────────┘ \n";
}

/**
 * @brief Перевірити валідність даних готелю
 */
bool Hotel::validate() const {
    if (name.empty()) {
        return false;
    }
}

```

```

    if (city.empty()) {
        return false;
    }
    if (stars < Constants::Limits::MIN_STARS || stars > Constants::Limits::MAX_STARS) {
        return false;
    }
    return true;
}

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 */
std::string Hotel::toCSV() const {
    std::ostringstream oss;
    oss << id << "," << name << "," << city << "," << stars;
    return oss.str();
}

/**
 * @brief Створити об'єкт Hotel з рядка CSV формату
 */
Hotel Hotel::fromCSV(const std::string& csvLine) {
    std::istringstream iss(csvLine);
    std::string token;
    int id, stars;
    std::string name, city;

    try {
        // Читаємо ID
        std::getline(iss, token, ',');
        id = std::stoi(token);

        // Читаємо назву
        std::getline(iss, name, ',');

        // Читаємо місто
        std::getline(iss, city, ',');

        // Читаємо зірки
        std::getline(iss, token, ',');
        stars = std::stoi(token);

        return Hotel(id, name, city, stars);
    } catch (const std::exception& e) {
        throw std::runtime_error("Помилка парсингу CSV: " + std::string(e.what()));
    }
}

/**
 * @brief Оновити наступний доступний ID
 */
void Hotel::updateNextId(int id) {
    if (id >= nextId) {
        nextId = id + 1;
    }
}
}

```

Програмний модуль «Room»

– Вміст Room.h

```

/**
 * @file Room.h
 * @brief Заголовковий файл класів Room (абстрактний), LuxuryRoom, StandardRoom
 * @author Дем'янчук Анастасія
 * @date 2025
 */

```

```

#ifndef ROOM_H
#define ROOM_H

#include <string>
#include <iostream>

/**
 * @enum RoomType
 * @brief Перелік типів номерів
 */
enum class RoomType {
    LUXURY,    ///< Люкс
    STANDARD  ///< Стандарт
};

/**
 * @class Room
 * @brief Абстрактний базовий клас для представлення номера в готелі
 *
 * Цей клас є базовим для всіх типів номерів.
 * Містить чисті віртуальні методи для реалізації в класах-нащадках.
 */
class Room {
protected:
    int id;           ///< Унікальний ідентифікатор номера
    int hotelId;      ///< ID готелю, до якого належить номер
    int capacity;     ///< Кількість місць (2 або 3)
    bool isOccupied;  ///< Чи зайнятий номер

    static int nextId;  ///< Лічильник для генерації ID

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    Room();

    /**
     * @brief Конструктор з параметрами
     * @param hotelId ID готелю
     * @param capacity Кількість місць
     */
    Room(int hotelId, int capacity);

    /**
     * @brief Конструктор з усіма параметрами
     * @param id ID номера
     * @param hotelId ID готелю
     * @param capacity Кількість місць
     * @param isOccupied Статус зайнятості
     */
    Room(int id, int hotelId, int capacity, bool isOccupied);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт Room
     */
    Room(const Room& other);

    /**
     * @brief Переміщувальний конструктор
     * @param other Інший об'єкт Room
     */
    Room(Room&& other) noexcept;

    /**
     * @brief Віртуальний деструктор
     */
    virtual ~Room();

```

```

/**
 * @brief Оператор присвоєння копіюванням
 */
Room& operator=(const Room& other);

/**
 * @brief Оператор присвоєння переміщенням
 */
Room& operator=(Room&& other) noexcept;

// Геттери
int getId() const;
int getHotelId() const;
int getCapacity() const;
bool getIsOccupied() const;

// Сеттери
void setId(int id);
void setHotelId(int hotelId);
void setCapacity(int capacity);
void setIsOccupied(bool occupied);

/**
 * @brief Чисто віртуальний метод отримання типу номера
 * @return Тип номера
 */
virtual RoomType getType() const = 0;

/**
 * @brief Чисто віртуальний метод отримання назви типу
 * @return Назва типу номера
 */
virtual std::string getTypeName() const = 0;

/**
 * @brief Чисто віртуальний метод виведення інформації про номер
 */
virtual void display() const = 0;

/**
 * @brief Чисто віртуальний метод розрахунку вартості
 * @return Вартість номера за добу
 */
virtual double calculatePrice() const = 0;

/**
 * @brief Перевірити валідність даних номера
 * @return true якщо дані валідні
 */
virtual bool validate() const;

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 * @return Рядок у форматі CSV
 */
virtual std::string toCSV() const;

/**
 * @brief Оновити наступний доступний ID
 * @param id Значення ID
 */
static void updateNextId(int id);

/**
 * @brief Створити об'єкт Room з рядка CSV
 * @param csvLine Рядок CSV
 * @return Вказівник на створений об'єкт Room
 */
static Room* fromCSV(const std::string& csvLine);
};

```

```

/**
 * @class LuxuryRoom
 * @brief Клас для представлення номера типу "Люкс"
 *
 * Успадковується від абстрактного класу Room.
 * Реалізує специфічну поведінку для люкс-номерів.
 */
class LuxuryRoom : public Room {
private:
    static const double BASE_PRICE; ///< Базова ціна люкс-номера

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    LuxuryRoom();

    /**
     * @brief Конструктор з параметрами
     * @param hotelId ID готелю
     * @param capacity Кількість місць (2 або 3)
     */
    LuxuryRoom(int hotelId, int capacity);

    /**
     * @brief Конструктор з усіма параметрами
     * @param id ID номера
     * @param hotelId ID готелю
     * @param capacity Кількість місць
     * @param isOccupied Статус зайнятості
     */
    LuxuryRoom(int id, int hotelId, int capacity, bool isOccupied);

    /**
     * @brief Копіювальний конструктор
     */
    LuxuryRoom(const LuxuryRoom& other);

    /**
     * @brief Переміщувальний конструктор
     */
    LuxuryRoom(LuxuryRoom&& other) noexcept;

    /**
     * @brief Деструктор
     */
    ~LuxuryRoom() override;

    /**
     * @brief Отримати тип номера
     * @return RoomType::LUXURY
     */
    RoomType getType() const override;

    /**
     * @brief Отримати назву типу
     * @return "Люкс"
     */
    std::string getTypeName() const override;

    /**
     * @brief Вивести інформацію про люкс-номер
     */
    void display() const override;

    /**
     * @brief Розрахувати вартість люкс-номера
     * @return Вартість за добу
     */

```

```

    double calculatePrice() const override;
};

/**
 * @class StandardRoom
 * @brief Клас для представлення номера типу "Стандарт"
 *
 * Успадковується від абстрактного класу Room.
 * Реалізує специфічну поведінку для стандартних номерів.
 */
class StandardRoom : public Room {
private:
    static const double BASE_PRICE; ///< Базова ціна стандартного номера

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    StandardRoom();

    /**
     * @brief Конструктор з параметрами
     * @param hotelId ID готелю
     * @param capacity Кількість місць (2 або 3)
     */
    StandardRoom(int hotelId, int capacity);

    /**
     * @brief Конструктор з усіма параметрами
     * @param id ID номера
     * @param hotelId ID готелю
     * @param capacity Кількість місць
     * @param isOccupied Статус зайнятості
     */
    StandardRoom(int id, int hotelId, int capacity, bool isOccupied);

    /**
     * @brief Копіювальний конструктор
     */
    StandardRoom(const StandardRoom& other);

    /**
     * @brief Переміщувальний конструктор
     */
    StandardRoom(StandardRoom&& other) noexcept;

    /**
     * @brief Деструктор
     */
    ~StandardRoom() override;

    /**
     * @brief Отримати тип номера
     * @return RoomType::STANDARD
     */
    RoomType getType() const override;

    /**
     * @brief Отримати назву типу
     * @return "Стандарт"
     */
    std::string getTypeName() const override;

    /**
     * @brief Вивести інформацію про стандартний номер
     */
    void display() const override;

    /**
     * @brief Розрахувати вартість стандартного номера

```

```

    * @return Вартість за добу
    */
    double calculatePrice() const override;
};

#endif // ROOM_H

```

– Вміст Room.cpp

```

/**
 * @file Room.cpp
 * @brief Реалізація класів Room, LuxuryRoom та StandardRoom
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Room.h"
#include "../include/Constants.h"
#include <sstream>
#include <stdexcept>
#include <iomanip>

// Ініціалізація статичних змінних
int Room::nextId = 1;
const double LuxuryRoom::BASE_PRICE = Constants::Prices::LUXURY_BASE;
const double StandardRoom::BASE_PRICE = Constants::Prices::STANDARD_BASE;

/**
 * @brief Конструктор за замовчуванням
 */
Room::Room() : id(0), hotelId(0), capacity(2), isOccupied(false) {}

/**
 * @brief Конструктор з параметрами
 */
Room::Room(int hotelId, int capacity)
    : id(nextId++), hotelId(hotelId), capacity(capacity), isOccupied(false) {}

/**
 * @brief Конструктор з усіма параметрами
 */
Room::Room(int id, int hotelId, int capacity, bool isOccupied)
    : id(id), hotelId(hotelId), capacity(capacity), isOccupied(isOccupied) {}

/**
 * @brief Копіювальний конструктор
 */
Room::Room(const Room& other)
    : id(other.id), hotelId(other.hotelId),
      capacity(other.capacity), isOccupied(other.isOccupied) {}

/**
 * @brief Переміщувальний конструктор
 */
Room::Room(Room&& other) noexcept
    : id(other.id), hotelId(other.hotelId),
      capacity(other.capacity), isOccupied(other.isOccupied) {
    other.id = 0;
    other.hotelId = 0;
    other.capacity = 0;
    other.isOccupied = false;
}

/**
 * @brief Віртуальний деструктор
 */

```



```

Room::~~Room() {
    // Деструктор базового класу
}

/**
 * @brief Оператор присвоєння копіюванням
 */
Room& Room::operator=(const Room& other) {
    if (this != &other) {
        id = other.id;
        hotelId = other.hotelId;
        capacity = other.capacity;
        isOccupied = other.isOccupied;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
Room& Room::operator=(Room&& other) noexcept {
    if (this != &other) {
        id = other.id;
        hotelId = other.hotelId;
        capacity = other.capacity;
        isOccupied = other.isOccupied;

        other.id = 0;
        other.hotelId = 0;
        other.capacity = 0;
        other.isOccupied = false;
    }
    return *this;
}

// Геттери
int Room::getId() const { return id; }
int Room::getHotelId() const { return hotelId; }
int Room::getCapacity() const { return capacity; }
bool Room::getIsOccupied() const { return isOccupied; }

// Сеттери
void Room::setId(int id) { this->id = id; }
void Room::setHotelId(int hotelId) { this->hotelId = hotelId; }

void Room::setCapacity(int capacity) {
    if (capacity != Constants::Limits::MIN_CAPACITY &&
        capacity != Constants::Limits::MAX_CAPACITY) {
        throw std::invalid_argument("Кількість місць має бути 2 або 3");
    }
    this->capacity = capacity;
}

void Room::setIsOccupied(bool occupied) {
    this->isOccupied = occupied;
}

/**
 * @brief Перевірити валідність даних номера
 */
bool Room::validate() const {
    if (hotelId <= 0) return false;
    if (capacity != Constants::Limits::MIN_CAPACITY &&
        capacity != Constants::Limits::MAX_CAPACITY) return false;
    return true;
}

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 */

```

```

std::string Room::toCSV() const {
    std::ostringstream oss;
    oss << id << "," << hotelId << "," << getTypeName() << ","
        << capacity << "," << (isOccupied ? "1" : "0");
    return oss.str();
}

/**
 * @brief Оновити наступний доступний ID
 */
void Room::updateNextId(int id) {
    if (id >= nextId) {
        nextId = id + 1;
    }
}

/**
 * @brief Створити об'єкт Room з рядка CSV
 */
Room* Room::fromCSV(const std::string& csvLine) {
    std::istringstream iss(csvLine);
    std::string token;
    int id, hotelId, capacity;
    std::string type;
    bool isOccupied;

    try {
        // Читаємо ID
        std::getline(iss, token, ',');
        id = std::stoi(token);

        // Читаємо ID готелю
        std::getline(iss, token, ',');
        hotelId = std::stoi(token);

        // Читаємо тип
        std::getline(iss, token, ',');
        type = token;

        // Читаємо кількість місць
        std::getline(iss, token, ',');
        capacity = std::stoi(token);

        // Читаємо статус зайнятості
        std::getline(iss, token, ',');
        isOccupied = (token == "1");

        // Створюємо відповідний тип номера
        if (type == "Люкс") {
            return new LuxuryRoom(id, hotelId, capacity, isOccupied);
        } else {
            return new StandardRoom(id, hotelId, capacity, isOccupied);
        }
    } catch (const std::exception& e) {
        throw std::runtime_error("Помилка парсингу CSV: " + std::string(e.what()));
    }
}

//
//
//
// РЕАЛІЗАЦІЯ КЛАСУ LUXURYROOM (Люкс)
//
//
//
//
/**
 * @brief Конструктор за замовчуванням
 */
LuxuryRoom::LuxuryRoom() : Room() {
}

```

```

/**
 * @brief Конструктор з параметрами
 */
LuxuryRoom::LuxuryRoom(int hotelId, int capacity)
    : Room(hotelId, capacity) {
}

/**
 * @brief Конструктор з усіма параметрами
 */
LuxuryRoom::LuxuryRoom(int id, int hotelId, int capacity, bool isOccupied)
    : Room(id, hotelId, capacity, isOccupied) {
}

/**
 * @brief Копіювальний конструктор
 */
LuxuryRoom::LuxuryRoom(const LuxuryRoom& other) : Room(other) {
}

/**
 * @brief Переміщувальний конструктор
 */
LuxuryRoom::LuxuryRoom(LuxuryRoom&& other) noexcept : Room(std::move(other)) {
}

/**
 * @brief Деструктор
 */
LuxuryRoom::~LuxuryRoom() {
    // Деструктор люкс-номера
}

/**
 * @brief Отримати тип номера
 */
RoomType LuxuryRoom::getType() const {
    return RoomType::LUXURY;
}

/**
 * @brief Отримати назву типу
 */
std::string LuxuryRoom::getTypeName() const {
    return Constants::RoomType::LUXURY;
}

/**
 * @brief Вивести інформацію про люкс-номер
 */
void LuxuryRoom::display() const {
    std::cout << "
    std::cout << "
    std::cout << "
    std::cout << " ID номера: " << std::left << std::setw(27) << id << "\n";
    std::cout << " ID готелю: " << std::left << std::setw(27) << hotelId << "\n";
    std::cout << " Місць: " << std::left << std::setw(31) << capacity << "\n";
    std::cout << " Статус: " << std::left << std::setw(30)
        << (isOccupied ? "Зайнятий" : "Вільний") << "\n";
    std::cout << " Ціна: " << std::left << std::setw(32)
        << (std::to_string((int)calculatePrice()) + " грн/доба") << "\n";
    std::cout << "
}

/**
 * @brief Розрахувати вартість люкс-номера
 */
double LuxuryRoom::calculatePrice() const {
    return BASE_PRICE + (capacity > Constants::Limits::MIN_CAPACITY ?

```

```

        Constants::Prices::LUXURY_EXTRA_PERSON : 0.0);
    }

//
=====
// РЕАЛІЗАЦІЯ КЛАСУ STANDARDROOM (Стандарт)
//
=====

/**
 * @brief Конструктор за замовчуванням
 */
StandardRoom::StandardRoom() : Room() {
}

/**
 * @brief Конструктор з параметрами
 */
StandardRoom::StandardRoom(int hotelId, int capacity)
    : Room(hotelId, capacity) {
}

/**
 * @brief Конструктор з усіма параметрами
 */
StandardRoom::StandardRoom(int id, int hotelId, int capacity, bool isOccupied)
    : Room(id, hotelId, capacity, isOccupied) {
}

/**
 * @brief Копіювальний конструктор
 */
StandardRoom::StandardRoom(const StandardRoom& other) : Room(other) {
}

/**
 * @brief Переміщувальний конструктор
 */
StandardRoom::StandardRoom(StandardRoom&& other) noexcept : Room(std::move(other)) {
}

/**
 * @brief Деструктор
 */
StandardRoom::~StandardRoom() {
    // Деструктор стандартного номера
}

/**
 * @brief Отримати тип номера
 */
RoomType StandardRoom::getType() const {
    return RoomType::STANDARD;
}

/**
 * @brief Отримати назву типу
 */
std::string StandardRoom::getTypeName() const {
    return Constants::RoomType::STANDARD;
}

/**
 * @brief Вивести інформацію про стандартний номер
 */
void StandardRoom::display() const {
    std::cout << "
    std::cout << "

```

```

std::cout << " | _____ | \n";
std::cout << " ID номера: " << std::left << std::setw(27) << id << "\n";
std::cout << " ID готелю: " << std::left << std::setw(27) << hotelId << "\n";
std::cout << " Місць: " << std::left << std::setw(31) << capacity << "\n";
std::cout << " Статус: " << std::left << std::setw(30)
    << (isOccupied ? "Зайнятий" : "Вільний") << "\n";
std::cout << " Ціна: " << std::left << std::setw(32)
    << (std::to_string((int)calculatePrice()) + " грн/доба") << "\n";
std::cout << " | _____ | \n";
}

/**
 * @brief Розрахувати вартість стандартного номера
 */
double StandardRoom::calculatePrice() const {
    return BASE_PRICE + (capacity > Constants::Limits::MIN_CAPACITY ?
        Constants::Prices::STANDARD_EXTRA_PERSON : 0.0);
}

```

Програмний модуль «Guest»

– Вміст Guest.h

```

/**
 * @file Guest.h
 * @brief Заголовковий файл класу Guest для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef GUEST_H
#define GUEST_H

#include <string>
#include <iostream>

/**
 * @class Guest
 * @brief Клас для представлення гостя готелю
 *
 * Цей клас зберігає інформацію про гостя: прізвище, ім'я та номер телефону.
 * Підтримує операції додавання, редагування та видалення гостей.
 */
class Guest {
private:
    int id;           ///< Унікальний ідентифікатор гостя
    std::string lastName; ///< Прізвище гостя
    std::string firstName; ///< Ім'я гостя
    std::string phone;  ///< Номер телефону

    static int nextId;  ///< Лічильник для генерації ID

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    Guest();

    /**
     * @brief Конструктор з параметрами
     * @param lastName Прізвище гостя
     * @param firstName Ім'я гостя
     * @param phone Номер телефону
     */
    Guest(const std::string& lastName, const std::string& firstName,
        const std::string& phone);

    /**
     * @brief Конструктор з усіма параметрами (включно з ID)
     */

```

```

* @param id Ідентифікатор гостя
* @param lastName Прізвище гостя
* @param firstName Ім'я гостя
* @param phone Номер телефону
*/
Guest(int id, const std::string& lastName, const std::string& firstName,
      const std::string& phone);

/**
* @brief Копіювальний конструктор
* @param other Інший об'єкт Guest для копіювання
*/
Guest(const Guest& other);

/**
* @brief Переміщувальний конструктор
* @param other Інший об'єкт Guest для переміщення
*/
Guest(Guest&& other) noexcept;

/**
* @brief Деструктор
*/
~Guest();

/**
* @brief Оператор присвоєння копіюванням
* @param other Інший об'єкт Guest
* @return Посилання на поточний об'єкт
*/
Guest& operator=(const Guest& other);

/**
* @brief Оператор присвоєння переміщенням
* @param other Інший об'єкт Guest
* @return Посилання на поточний об'єкт
*/
Guest& operator=(Guest&& other) noexcept;

// Геттери
/**
* @brief Отримати ID гостя
* @return ID гостя
*/
int getId() const;

/**
* @brief Отримати прізвище гостя
* @return Прізвище гостя
*/
std::string getLastName() const;

/**
* @brief Отримати ім'я гостя
* @return Ім'я гостя
*/
std::string getFirstName() const;

/**
* @brief Отримати повне ім'я гостя
* @return Повне ім'я (Прізвище Ім'я)
*/
std::string getFullName() const;

/**
* @brief Отримати номер телефону
* @return Номер телефону
*/
std::string getPhone() const;

```

```

// Сеттери
/**
 * @brief Встановити ID гостя
 * @param id Новий ID
 */
void setId(int id);

/**
 * @brief Встановити прізвище гостя
 * @param lastName Нове прізвище
 */
void setLastName(const std::string& lastName);

/**
 * @brief Встановити ім'я гостя
 * @param firstName Нове ім'я
 */
void setFirstName(const std::string& firstName);

/**
 * @brief Встановити номер телефону
 * @param phone Новий номер телефону
 */
void setPhone(const std::string& phone);

// Методи
/**
 * @brief Вивести інформацію про гостя
 */
void display() const;

/**
 * @brief Перевірити валідність даних гостя
 * @return true якщо дані валідні, false інакше
 */
bool validate() const;

/**
 * @brief Перевірити валідність номера телефону
 * @param phone Номер телефону для перевірки
 * @return true якщо номер валідний, false інакше
 */
static bool isValidPhone(const std::string& phone);

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 * @return Рядок у форматі CSV
 */
std::string toCSV() const;

/**
 * @brief Створити об'єкт Guest з рядка CSV формату
 * @param csvLine Рядок у форматі CSV
 * @return Об'єкт Guest
 */
static Guest fromCSV(const std::string& csvLine);

/**
 * @brief Оновити наступний доступний ID
 * @param id Значення ID
 */
static void updateNextId(int id);

/**
 * @brief Оператор порівняння за ім'ям (для сортування)
 * @param other Інший гість
 * @return true якщо поточний гість менший за іншого
 */
bool operator<(const Guest& other) const;

```

```

/**
 * @brief Оператор порівняння на рівність
 * @param other Інший гість
 * @return true якщо гості рівні
 */
bool operator==(const Guest& other) const;
};

#endif // GUEST_H

```

– Вміст Guest.cpp

```

/**
 * @file Guest.cpp
 * @brief Реалізація класу Guest для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Guest.h"
#include "../include/Constants.h"
#include <sstream>
#include <stdexcept>
#include <iomanip>
#include <algorithm>
#include <cctype>

// Ініціалізація статичної змінної
int Guest::nextId = 1;

/**
 * @brief Конструктор за замовчуванням
 */
Guest::Guest() : id(0), lastName(""), firstName(""), phone("") {
}

/**
 * @brief Конструктор з параметрами
 */
Guest::Guest(const std::string& lastName, const std::string& firstName,
             const std::string& phone)
    : id(nextId++), lastName(lastName), firstName(firstName), phone(phone) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані гостя");
    }
}

/**
 * @brief Конструктор з усіма параметрами
 */
Guest::Guest(int id, const std::string& lastName, const std::string& firstName,
             const std::string& phone)
    : id(id), lastName(lastName), firstName(firstName), phone(phone) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані гостя");
    }
}

/**
 * @brief Копіювальний конструктор
 */
Guest::Guest(const Guest& other)
    : id(other.id), lastName(other.lastName),
      firstName(other.firstName), phone(other.phone) {
}

/**
 * @brief Переміщувальний конструктор
 */
Guest::Guest(Guest&& other) noexcept

```



```

        : id(other.id), lastName(std::move(other.lastName)),
        firstName(std::move(other.firstName)), phone(std::move(other.phone)) {
        other.id = 0;
    }

    /**
     * @brief Деструктор
     */
    Guest::~Guest() {
        // Деструктор - виведення повідомлення про знищення
        if (id != 0) {
            // std::cout << "Гість " << getFullName() << " (ID: " << id << ") знищено\n";
        }
    }

    /**
     * @brief Оператор присвоєння копіюванням
     */
    Guest& Guest::operator=(const Guest& other) {
        if (this != &other) {
            id = other.id;
            lastName = other.lastName;
            firstName = other.firstName;
            phone = other.phone;
        }
        return *this;
    }

    /**
     * @brief Оператор присвоєння переміщенням
     */
    Guest& Guest::operator=(Guest&& other) noexcept {
        if (this != &other) {
            id = other.id;
            lastName = std::move(other.lastName);
            firstName = std::move(other.firstName);
            phone = std::move(other.phone);

            other.id = 0;
        }
        return *this;
    }

    // Геттери
    int Guest::getId() const {
        return id;
    }

    std::string Guest::getLastName() const {
        return lastName;
    }

    std::string Guest::getFirstName() const {
        return firstName;
    }

    std::string Guest::getFullName() const {
        return lastName + " " + firstName;
    }

    std::string Guest::getPhone() const {
        return phone;
    }

    // Сеттери
    void Guest::setId(int id) {
        this->id = id;
    }

    void Guest::setLastName(const std::string& lastName) {

```

```

    if (lastName.empty()) {
        throw std::invalid_argument("Прізвище не може бути порожнім");
    }
    this->lastName = lastName;
}

void Guest::setFirstName(const std::string& firstName) {
    if (firstName.empty()) {
        throw std::invalid_argument("Ім'я не може бути порожнім");
    }
    this->firstName = firstName;
}

void Guest::setPhone(const std::string& phone) {
    if (!IsValidPhone(phone)) {
        throw std::invalid_argument("Некоректний формат номера телефону");
    }
    this->phone = phone;
}

/**
 * @brief Вивести інформацію про гостя
 */
void Guest::display() const {
    std::cout << "
    std::cout << "          ІНФОРМАЦІЯ ПРО ГОСТЯ          \n";
    std::cout << "
    std::cout << "ID: " << std::left << std::setw(36) << id << "\n";
    std::cout << "Прізвище: " << std::left << std::setw(30) << lastName << "\n";
    std::cout << "Ім'я: " << std::left << std::setw(34) << firstName << "\n";
    std::cout << "Телефон: " << std::left << std::setw(31) << phone << "\n";
    std::cout << "
}

/**
 * @brief Перевірити валідність даних гостя
 */
bool Guest::validate() const {
    if (lastName.empty() || firstName.empty()) {
        return false;
    }
    if (!IsValidPhone(phone)) {
        return false;
    }
    return true;
}

/**
 * @brief Перевірити валідність номера телефону
 */
bool Guest::IsValidPhone(const std::string& phone) {
    if (phone.empty()) {
        return false;
    }

    // Перевіряємо чи містить номер лише цифри, пробіли, дужки, + та -
    for (char c : phone) {
        if (!std::isdigit(c) && c != ' ' && c != '+' &&
            c != '-' && c != '(' && c != ')') {
            return false;
        }
    }

    // Підраховуємо кількість цифр
    int digitCount = 0;
    for (char c : phone) {
        if (std::isdigit(c)) {
            digitCount++;
        }
    }
}

```

```

// Мінімум 10 цифр для валідного номера
return digitCount >= Constants::Limits::MIN_PHONE_DIGITS;
}

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 */
std::string Guest::toCSV() const {
    std::ostringstream oss;
    oss << id << "," << lastName << "," << firstName << "," << phone;
    return oss.str();
}

/**
 * @brief Створити об'єкт Guest з рядка CSV формату
 */
Guest Guest::fromCSV(const std::string& csvLine) {
    std::istringstream iss(csvLine);
    std::string token;
    int id;
    std::string lastName, firstName, phone;

    try {
        // Читаємо ID
        std::getline(iss, token, ',');
        id = std::stoi(token);

        // Читаємо прізвище
        std::getline(iss, lastName, ',');

        // Читаємо ім'я
        std::getline(iss, firstName, ',');

        // Читаємо телефон
        std::getline(iss, phone, ',');

        return Guest(id, lastName, firstName, phone);
    } catch (const std::exception& e) {
        throw std::runtime_error("Помилка парсингу CSV: " + std::string(e.what()));
    }
}

/**
 * @brief Оновити наступний доступний ID
 */
void Guest::updateNextId(int id) {
    if (id >= nextId) {
        nextId = id + 1;
    }
}

/**
 * @brief Оператор порівняння за ім'ям (для сортування)
 */
bool Guest::operator<(const Guest& other) const {
    // Спочатку порівнюємо за прізвищем
    if (lastName != other.lastName) {
        return lastName < other.lastName;
    }
    // Якщо прізвища однакові, порівнюємо за ім'ям
    return firstName < other.firstName;
}

/**
 * @brief Оператор порівняння на рівність
 */
bool Guest::operator==(const Guest& other) const {
    return id == other.id;
}

```

Програмний модуль «Booking»

– Вміст Booking.h

```

/**
 * @file Booking.h
 * @brief Заголовковий файл класу Booking для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef BOOKING_H
#define BOOKING_H

#include <string>
#include <iostream>
#include <ctime>

/**
 * @class Booking
 * @brief Клас для представлення бронювання номера в готелі
 *
 * Цей клас зберігає інформацію про бронювання: гостя, номер, дати заїзду/виїзду.
 * Підтримує операції додавання, редагування та видалення бронювань.
 */
class Booking {
private:
    int id;           ///< Унікальний ідентифікатор бронювання
    int guestId;      ///< ID гостя
    int roomId;       ///< ID номера
    std::string checkInDate; ///< Дата заїзду (формат: YYYY-MM-DD)
    std::string checkOutDate; ///< Дата виїзду (формат: YYYY-MM-DD)

    static int nextId; ///< Лічильник для генерації ID

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    Booking();

    /**
     * @brief Конструктор з параметрами
     * @param guestId ID гостя
     * @param roomId ID номера
     * @param checkInDate Дата заїзду
     * @param checkOutDate Дата виїзду
     */
    Booking(int guestId, int roomId, const std::string& checkInDate,
            const std::string& checkOutDate);

    /**
     * @brief Конструктор з усіма параметрами (включно з ID)
     * @param id Ідентифікатор бронювання
     * @param guestId ID гостя
     * @param roomId ID номера
     * @param checkInDate Дата заїзду
     * @param checkOutDate Дата виїзду
     */
    Booking(int id, int guestId, int roomId, const std::string& checkInDate,
            const std::string& checkOutDate);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт Booking для копіювання
     */
    Booking(const Booking& other);
}

```

```

* @brief Переміщувальний конструктор
* @param other Інший об'єкт Booking для переміщення
*/
Booking(Booking&& other) noexcept;

/**
* @brief Деструктор
*/
~Booking();

/**
* @brief Оператор присвоєння копіюванням
* @param other Інший об'єкт Booking
* @return Посилання на поточний об'єкт
*/
Booking& operator=(const Booking& other);

/**
* @brief Оператор присвоєння переміщенням
* @param other Інший об'єкт Booking
* @return Посилання на поточний об'єкт
*/
Booking& operator=(Booking&& other) noexcept;

// Геттери
/**
* @brief Отримати ID бронювання
* @return ID бронювання
*/
int getId() const;

/**
* @brief Отримати ID гостя
* @return ID гостя
*/
int getGuestId() const;

/**
* @brief Отримати ID номера
* @return ID номера
*/
int getRoomId() const;

/**
* @brief Отримати дату заїзду
* @return Дата заїзду
*/
std::string getCheckInDate() const;

/**
* @brief Отримати дату виїзду
* @return Дата виїзду
*/
std::string getCheckOutDate() const;

// Сеттери
/**
* @brief Встановити ID бронювання
* @param id Новий ID
*/
void setId(int id);

/**
* @brief Встановити ID гостя
* @param guestId Новий ID гостя
*/
void setGuestId(int guestId);

/**
* @brief Встановити ID номера

```

```

    * @param roomId Новий ID номера
    */
void setRoomId(int roomId);

/**
 * @brief Встановити дату заїзду
 * @param checkInDate Нова дата заїзду
 */
void setCheckInDate(const std::string& checkInDate);

/**
 * @brief Встановити дату виїзду
 * @param checkOutDate Нова дата виїзду
 */
void setCheckOutDate(const std::string& checkOutDate);

// Методи
/**
 * @brief Вивести інформацію про бронювання
 */
void display() const;

/**
 * @brief Перевірити валідність даних бронювання
 * @return true якщо дані валідні, false інакше
 */
bool validate() const;

/**
 * @brief Перевірити валідність дати
 * @param date Дата у форматі YYYY-MM-DD
 * @return true якщо дата валідна, false інакше
 */
static bool isValidDate(const std::string& date);

/**
 * @brief Перевірити, чи дата заїзду раніше дати виїзду
 * @return true якщо порядок дат правильний
 */
bool areDatesValid() const;

/**
 * @brief Розрахувати кількість днів проживання
 * @return Кількість днів
 */
int calculateDays() const;

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 * @return Рядок у форматі CSV
 */
std::string toCSV() const;

/**
 * @brief Створити об'єкт Booking з рядка CSV формату
 * @param csvLine Рядок у форматі CSV
 * @return Об'єкт Booking
 */
static Booking fromCSV(const std::string& csvLine);

/**
 * @brief Оновити наступний доступний ID
 * @param id Значення ID
 */
static void updateNextId(int id);

/**
 * @brief Порівняти дві дати
 * @param date1 Перша дата
 * @param date2 Друга дата

```

```

    * @return -1 якщо date1 < date2, 0 якщо рівні, 1 якщо date1 > date2
    */
    static int compareDates(const std::string& date1, const std::string& date2);
};

#endif // BOOKING_H

```

– Вміст Booking.cpp

```

/**
 * @file Booking.cpp
 * @brief Реалізація класу Booking для системи бронювання готелів
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Booking.h"
#include "../include/Constants.h"
#include <sstream>
#include <stdexcept>
#include <iomanip>
#include <algorithm>

// Ініціалізація статичної змінної
int Booking::nextId = 1;

/**
 * @brief Конструктор за замовчуванням
 */
Booking::Booking() : id(0), guestId(0), roomId(0), checkInDate(""), checkOutDate("") {}

/**
 * @brief Конструктор з параметрами
 */
Booking::Booking(int guestId, int roomId, const std::string& checkInDate,
                 const std::string& checkOutDate)
    : id(nextId++), guestId(guestId), roomId(roomId),
      checkInDate(checkInDate), checkOutDate(checkOutDate) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані бронювання");
    }
}

/**
 * @brief Конструктор з усіма параметрами
 */
Booking::Booking(int id, int guestId, int roomId, const std::string& checkInDate,
                 const std::string& checkOutDate)
    : id(id), guestId(guestId), roomId(roomId),
      checkInDate(checkInDate), checkOutDate(checkOutDate) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані бронювання");
    }
}

/**
 * @brief Копіювальний конструктор
 */
Booking::Booking(const Booking& other)
    : id(other.id), guestId(other.guestId), roomId(other.roomId),
      checkInDate(other.checkInDate), checkOutDate(other.checkOutDate) {}

/**
 * @brief Переміщувальний конструктор
 */
Booking::Booking(Booking&& other) noexcept
    : id(other.id), guestId(other.guestId), roomId(other.roomId),
      checkInDate(std::move(other.checkInDate)),

```

```

        checkOutDate(std::move(other.checkOutDate)) {
            other.id = 0;
            other.guestId = 0;
            other.roomId = 0;
        }

/**
 * @brief Деструктор
 */
Booking::~Booking() {
    // Деструктор
    if (id != 0) {
        // std::cout << "Бронювання ID: " << id << " знищено\n";
    }
}

/**
 * @brief Оператор присвоєння копіюванням
 */
Booking& Booking::operator=(const Booking& other) {
    if (this != &other) {
        id = other.id;
        guestId = other.guestId;
        roomId = other.roomId;
        checkInDate = other.checkInDate;
        checkOutDate = other.checkOutDate;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
Booking& Booking::operator=(Booking&& other) noexcept {
    if (this != &other) {
        id = other.id;
        guestId = other.guestId;
        roomId = other.roomId;
        checkInDate = std::move(other.checkInDate);
        checkOutDate = std::move(other.checkOutDate);

        other.id = 0;
        other.guestId = 0;
        other.roomId = 0;
    }
    return *this;
}

// Геттери
int Booking::getId() const {
    return id;
}

int Booking::getGuestId() const {
    return guestId;
}

int Booking::getRoomId() const {
    return roomId;
}

std::string Booking::getCheckInDate() const {
    return checkInDate;
}

std::string Booking::getCheckOutDate() const {
    return checkOutDate;
}

// Сеттери

```



```

void Booking::setId(int id) {
    this->id = id;
}

void Booking::setGuestId(int guestId) {
    if (guestId <= 0) {
        throw std::invalid_argument("ID гостя має бути додатнім");
    }
    this->guestId = guestId;
}

void Booking::setRoomId(int roomId) {
    if (roomId <= 0) {
        throw std::invalid_argument("ID номера має бути додатнім");
    }
    this->roomId = roomId;
}

void Booking::setCheckInDate(const std::string& checkInDate) {
    if (!isValidDate(checkInDate)) {
        throw std::invalid_argument("Некоректна дата заїзду");
    }
    this->checkInDate = checkInDate;
}

void Booking::setCheckOutDate(const std::string& checkOutDate) {
    if (!isValidDate(checkOutDate)) {
        throw std::invalid_argument("Некоректна дата виїзду");
    }
    this->checkOutDate = checkOutDate;
}

/**
 * @brief Вивести інформацію про бронювання
 */
void Booking::display() const {
    std::cout << "
    std::cout << "      ІНФОРМАЦІЯ ПРО БРОНЮВАННЯ      \n";
    std::cout << "
    std::cout << "ID бронювання: " << std::left << std::setw(23) << id << "\n";
    std::cout << "ID гостя: " << std::left << std::setw(30) << guestId << "\n";
    std::cout << "ID номера: " << std::left << std::setw(29) << roomId << "\n";
    std::cout << "Дата заїзду: " << std::left << std::setw(27) << checkInDate << "\n";
    std::cout << "Дата виїзду: " << std::left << std::setw(27) << checkOutDate << "\n";
    std::cout << "Кількість днів: " << std::left << std::setw(24) << calculateDays() << "\n";
    std::cout << "
}

/**
 * @brief Перевірити валідність даних бронювання
 */
bool Booking::validate() const {
    if (guestId <= 0 || roomId <= 0) {
        return false;
    }
    if (!isValidDate(checkInDate) || !isValidDate(checkOutDate)) {
        return false;
    }
    if (!areDatesValid()) {
        return false;
    }
    return true;
}

/**
 * @brief Перевірити валідність дати
 */
bool Booking::isValidDate(const std::string& date) {
    if (date.length() != Constants::Limits::DATE_LENGTH) {
        return false;
    }
}

```

```

}

// Перевіряємо формат YYYY-MM-DD
if (date[4] != '-' || date[7] != '-') {
    return false;
}

try {
    int year = std::stoi(date.substr(0, 4));
    int month = std::stoi(date.substr(5, 2));
    int day = std::stoi(date.substr(8, 2));

    // Перевірка діапазонів
    if (year < 2020 || year > 2100) return false;
    if (month < 1 || month > 12) return false;
    if (day < 1 || day > 31) return false;

    // Перевірка днів у місяці
    if (month == 2) {
        bool isLeap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
        if (day > (isLeap ? 29 : 28)) return false;
    } else if (month == 4 || month == 6 || month == 9 || month == 11) {
        if (day > 30) return false;
    }

    return true;
} catch (...) {
    return false;
}
}

/**
 * @brief Перевірити, чи дата заїзду раніше дати виїзду
 */
bool Booking::areDatesValid() const {
    return compareDates(checkInDate, checkOutDate) < 0;
}

/**
 * @brief Розрахувати кількість днів проживання
 */
int Booking::calculateDays() const {
    try {
        int year1 = std::stoi(checkInDate.substr(0, 4));
        int month1 = std::stoi(checkInDate.substr(5, 2));
        int day1 = std::stoi(checkInDate.substr(8, 2));

        int year2 = std::stoi(checkOutDate.substr(0, 4));
        int month2 = std::stoi(checkOutDate.substr(5, 2));
        int day2 = std::stoi(checkOutDate.substr(8, 2));

        // Спрощений розрахунок (не враховує всі нюанси)
        int days1 = year1 * 365 + month1 * 30 + day1;
        int days2 = year2 * 365 + month2 * 30 + day2;

        return days2 - days1;
    } catch (...) {
        return 0;
    }
}

/**
 * @brief Конвертувати об'єкт у рядок CSV формату
 */
std::string Booking::toCSV() const {
    std::ostringstream oss;
    oss << id << "," << guestId << "," << roomId << ","
        << checkInDate << "," << checkOutDate;
    return oss.str();
}

```

```

/**
 * @brief Створити об'єкт Booking з рядка CSV формату
 */
Booking Booking::fromCSV(const std::string& csvLine) {
    std::istringstream iss(csvLine);
    std::string token;
    int id, guestId, roomId;
    std::string checkInDate, checkOutDate;

    try {
        // Читаємо ID
        std::getline(iss, token, ',');
        id = std::stoi(token);

        // Читаємо ID гостя
        std::getline(iss, token, ',');
        guestId = std::stoi(token);

        // Читаємо ID номера
        std::getline(iss, token, ',');
        roomId = std::stoi(token);

        // Читаємо дату заїзду
        std::getline(iss, checkInDate, ',');

        // Читаємо дату виїзду
        std::getline(iss, checkOutDate, ',');

        return Booking(id, guestId, roomId, checkInDate, checkOutDate);
    } catch (const std::exception& e) {
        throw std::runtime_error("Помилка парсингу CSV: " + std::string(e.what()));
    }
}

/**
 * @brief Оновити наступний доступний ID
 */
void Booking::updateNextId(int id) {
    if (id >= nextId) {
        nextId = id + 1;
    }
}

/**
 * @brief Порівняти дві дати
 */
int Booking::compareDates(const std::string& date1, const std::string& date2) {
    if (date1 < date2) return -1;
    if (date1 > date2) return 1;
    return 0;
}

```

Програмний модуль «User»

– Вміст User.h

```

/**
 * @file User.h
 * @brief Заголовковий файл класу User для системи авторизації
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef USER_H
#define USER_H

#include <string>
#include <iostream>

```

```

/**
 * @class User
 * @brief Клас для представлення користувача системи
 *
 * Цей клас зберігає інформацію про користувача: логін та пароль.
 * Підтримує систему авторизації та адміністрування користувачів.
 */
class User {
private:
    std::string username;    ///< Логін користувача
    std::string password;    ///< Пароль користувача
    bool isAdmin;           ///< Чи є користувач адміністратором

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    User();

    /**
     * @brief Конструктор з параметрами
     * @param username Логін користувача
     * @param password Пароль користувача
     * @param isAdmin Чи є користувач адміністратором
     */
    User(const std::string& username, const std::string& password,
          bool isAdmin = false);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт User для копіювання
     */
    User(const User& other);

    /**
     * @brief Переміщувальний конструктор
     * @param other Інший об'єкт User для переміщення
     */
    User(User&& other) noexcept;

    /**
     * @brief Деструктор
     */
    ~User();

    /**
     * @brief Оператор присвоєння копіюванням
     * @param other Інший об'єкт User
     * @return Посилання на поточний об'єкт
     */
    User& operator=(const User& other);

    /**
     * @brief Оператор присвоєння переміщенням
     * @param other Інший об'єкт User
     * @return Посилання на поточний об'єкт
     */
    User& operator=(User&& other) noexcept;

    // Getter
    /**
     * @brief Отримати логін користувача
     * @return Логін користувача
     */
    std::string getUsername() const;

    /**
     * @brief Отримати пароль користувача
     * @return Пароль користувача
     */

```

```

std::string getPassword() const;

/**
 * @brief Перевірити, чи є користувач адміністратором
 * @return true якщо адміністратор, false інакше
 */
bool getIsAdmin() const;

// Сеттери
/**
 * @brief Встановити логін користувача
 * @param username Новий логін
 */
void setUsername(const std::string& username);

/**
 * @brief Встановити пароль користувача
 * @param password Новий пароль
 */
void setPassword(const std::string& password);

/**
 * @brief Встановити статус адміністратора
 * @param isAdmin Статус адміністратора
 */
void setIsAdmin(bool isAdmin);

// Методи
/**
 * @brief Вивести інформацію про користувача
 */
void display() const;

/**
 * @brief Перевірити валідність даних користувача
 * @return true якщо дані валідні, false інакше
 */
bool validate() const;

/**
 * @brief Перевірити валідність логіну
 * @param username Логін для перевірки
 * @return true якщо логін валідний, false інакше
 */
static bool isValidUsername(const std::string& username);

/**
 * @brief Перевірити валідність пароля
 * @param password Пароль для перевірки
 * @return true якщо пароль валідний, false інакше
 */
static bool isValidPassword(const std::string& password);

/**
 * @brief Перевірити, чи співпадає пароль
 * @param password Пароль для порівняння
 * @return true якщо пароль співпадає
 */
bool checkPassword(const std::string& password) const;

/**
 * @brief Конвертувати об'єкт у рядок для збереження у файл
 * @return Рядок у форматі username:password:isAdmin
 */
std::string toString() const;

/**
 * @brief Створити об'єкт User з рядка
 * @param line Рядок у форматі username:password:isAdmin
 * @return Об'єкт User

```

```

    */
    static User fromString(const std::string& line);

    /**
     * @brief Оператор порівняння на рівність
     * @param other Інший користувач
     * @return true якщо користувачі рівні
     */
    bool operator==(const User& other) const;
};

#endif // USER_H

```

– Вміст User.cpp

```

/**
 * @file User.cpp
 * @brief Реалізація класу User для системи авторизації
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/User.h"
#include "../include/Constants.h"
#include <sstream>
#include <string>
#include <iomanip>
#include <algorithm>

/**
 * @brief Конструктор за замовчуванням
 */
User::User() : username(""), password(""), isAdmin(false) {}

/**
 * @brief Конструктор з параметрами
 */
User::User(const std::string& username, const std::string& password, bool isAdmin)
    : username(username), password(password), isAdmin(isAdmin) {
    if (!validate()) {
        throw std::invalid_argument("Некоректні дані користувача");
    }
}

/**
 * @brief Копіювальний конструктор
 */
User::User(const User& other)
    : username(other.username), password(other.password), isAdmin(other.isAdmin) {}

/**
 * @brief Переміщувальний конструктор
 */
User::User(User&& other) noexcept
    : username(std::move(other.username)),
      password(std::move(other.password)),
      isAdmin(other.isAdmin) {
    other.isAdmin = false;
}

/**
 * @brief Деструктор
 */
User::~User() {
    if (!username.empty()) {}
}

```

```

}

/**
 * @brief Оператор присвоєння копіюванням
 */
User& User::operator=(const User& other) {
    if (this != &other) {
        username = other.username;
        password = other.password;
        isAdmin = other.isAdmin;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
User& User::operator=(User&& other) noexcept {
    if (this != &other) {
        username = std::move(other.username);
        password = std::move(other.password);
        isAdmin = other.isAdmin;

        other.isAdmin = false;
    }
    return *this;
}

// Геттери
std::string User::getUsername() const {
    return username;
}

std::string User::getPassword() const {
    return password;
}

bool User::getIsAdmin() const {
    return isAdmin;
}

// Сеттери
void User::setUsername(const std::string& username) {
    if (!isValidUsername(username)) {
        throw std::invalid_argument("Некоректний логін");
    }
    this->username = username;
}

void User::setPassword(const std::string& password) {
    if (!isValidPassword(password)) {
        throw std::invalid_argument("Некоректний пароль");
    }
    this->password = password;
}

void User::setIsAdmin(bool isAdmin) {
    this->isAdmin = isAdmin;
}

/**
 * @brief Вивести інформацію про користувача
 */
void User::display() const {
    std::cout << "
    std::cout << "      ІНФОРМАЦІЯ ПРО КОРИСТУВАЧА      \n";
    std::cout << "
    std::cout << " Логін: " << std::left << std::setw(33) << username << "\n";
    std::cout << " Роль: " << std::left << std::setw(34)
        << (isAdmin ? "Адміністратор" : "Користувач") << "\n";

```

```

        std::cout << "_____|\n";
    }

    /**
     * @brief Перевірити валідність даних користувача
     */
    bool User::validate() const {
        if (!IsValidUsername(username)) {
            return false;
        }
        if (!IsValidPassword(password)) {
            return false;
        }
        return true;
    }

    /**
     * @brief Перевірити валідність логіну
     */
    bool User::IsValidUsername(const std::string& username) {
        if (username.empty() || username.length() < Constants::Limits::MIN_USERNAME_LEN) {
            return false;
        }

        // Логін може містити лише літери, цифри та підкреслення
        for (char c : username) {
            if (!std::isalnum(c) && c != '_') {
                return false;
            }
        }

        return true;
    }

    /**
     * @brief Перевірити валідність пароля
     */
    bool User::IsValidPassword(const std::string& password) {
        // Пароль має бути довжиною мінімум 4 символи
        if (password.empty() || password.length() < Constants::Limits::MIN_PASSWORD_LEN) {
            return false;
        }

        return true;
    }

    /**
     * @brief Перевірити, чи співпадає пароль
     */
    bool User::checkPassword(const std::string& password) const {
        return this->password == password;
    }

    /**
     * @brief Конвертувати об'єкт у рядок для збереження у файл
     */
    std::string User::toString() const {
        std::ostringstream oss;
        oss << username << ":" << password << ":" << (isAdmin ? "1" : "0");
        return oss.str();
    }

    /**
     * @brief Створити об'єкт User з рядка
     */
    User User::fromString(const std::string& line) {
        std::istringstream iss(line);
        std::string username, password, adminStr;

        try {

```



```

// Читаємо логін
std::getline(iss, username, ':');

// Читаємо пароль
std::getline(iss, password, ':');

// Читаємо статус адміністратора
std::getline(iss, adminStr, ':');
bool isAdmin = (adminStr == "1");

return User(username, password, isAdmin);
} catch (const std::exception& e) {
    throw std::runtime_error("Помилка парсингу рядка користувача: " +
        std::string(e.what()));
}
}

/**
 * @brief Оператор порівняння на рівність
 */
bool User::operator==(const User& other) const {
    return username == other.username;
}

```

Програмний модуль «Managers»

– Вміст HotelManager.h

```

/**
 * @file HotelManager.h
 * @brief Заголовковий файл класу HotelManager для керування готелями
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef HOTEL_MANAGER_H
#define HOTEL_MANAGER_H

#include "Hotel.h"
#include <vector>
#include <string>
#include <memory>

/**
 * @class HotelManager
 * @brief Клас для управління колекцією готелів
 *
 * Цей клас забезпечує функціонал для роботи з базою готелів:
 * додавання, редагування, видалення, пошук, сортування.
 * Також відповідає за завантаження та збереження даних у файл.
 */
class HotelManager {
private:
    std::vector<Hotel> hotels;    ///< Колекція готелів
    std::string filename;        ///< Ім'я файлу для збереження даних

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    HotelManager();

    /**
     * @brief Конструктор з параметром
     * @param filename Ім'я файлу для збереження даних
     */
    HotelManager(const std::string& filename);
}

```

```

* @brief Копіювальний конструктор
* @param other Інший об'єкт HotelManager
*/
HotelManager(const HotelManager& other);

/**
* @brief Переміщувальний конструктор
* @param other Інший об'єкт HotelManager
*/
HotelManager(HotelManager&& other) noexcept;

/**
* @brief Деструктор
*/
~HotelManager();

/**
* @brief Оператор присвоєння копіюванням
*/
HotelManager& operator=(const HotelManager& other);

/**
* @brief Оператор присвоєння переміщенням
*/
HotelManager& operator=(HotelManager&& other) noexcept;

/**
* @brief Додати новий готель
* @param hotel Готель для додавання
* @return true якщо успішно додано
*/
bool addHotel(const Hotel& hotel);

/**
* @brief Редагувати готель за ID
* @param id ID готелю
* @param newHotel Нові дані готелю
* @return true якщо успішно відредаговано
*/
bool editHotel(int id, const Hotel& newHotel);

/**
* @brief Видалити готель за ID
* @param id ID готелю
* @return true якщо успішно видалено
*/
bool deleteHotel(int id);

/**
* @brief Знайти готель за ID
* @param id ID готелю
* @return Вказівник на готель або nullptr
*/
Hotel* findHotelById(int id);

/**
* @brief Пошук готелів за назвою
* @param name Назва або частина назви
* @return Вектор знайдених готелів
*/
std::vector<Hotel> searchByName(const std::string& name) const;

/**
* @brief Пошук готелів за містом
* @param city Назва міста
* @return Вектор знайдених готелів
*/
std::vector<Hotel> searchByCity(const std::string& city) const;

/**

```

```

    * @brief Пошук готелів за кількістю зірок
    * @param stars Кількість зірок
    * @return Вектор знайдених готелів
    */
std::vector<Hotel> searchByStars(int stars) const;

/**
 * @brief Отримати всі готелі
 * @return Вектор всіх готелів
 */
std::vector<Hotel> getAllHotels() const;

/**
 * @brief Отримати кількість готелів
 * @return Кількість готелів
 */
size_t getHotelCount() const;

/**
 * @brief Перевірити, чи існує готель з певним ID
 * @param id ID готелю
 * @return true якщо існує
 */
bool hotelExists(int id) const;

/**
 * @brief Сортувати готелі за назвою
 */
void sortByName();

/**
 * @brief Сортувати готелі за містом
 */
void sortByCity();

/**
 * @brief Сортувати готелі за кількістю зірок
 */
void sortByStars();

/**
 * @brief Вивести всі готелі
 */
void displayAll() const;

/**
 * @brief Завантажити дані з файлу
 * @return true якщо успішно завантажено
 */
bool loadFromFile();

/**
 * @brief Зберегти дані у файл
 * @return true якщо успішно збережено
 */
bool saveToFile() const;

/**
 * @brief Очистити всі дані
 */
void clear();
};

```

```
#endif // HOTEL_MANAGER_H
```

— Вміст HotelManager.cpp

```

/**
 * @file HotelManager.cpp
 * @brief Реалізація класу HotelManager для керування готелями
 * @author Дем'янчук Анастасія

```

```

* @date 2025
*/

#include "../include/HotelManager.h"
#include "../include/Constants.h"
#include <fstream>
#include <algorithm>
#include <iostream>

/**
 * @brief Конструктор за замовчуванням
 */
HotelManager::HotelManager() : filename(Constants::Files::HOTELS) {
}

/**
 * @brief Конструктор з параметром
 */
HotelManager::HotelManager(const std::string& filename) : filename(filename) {
}

/**
 * @brief Копіювальний конструктор
 */
HotelManager::HotelManager(const HotelManager& other)
    : hotels(other.hotels), filename(other.filename) {
}

/**
 * @brief Переміщувальний конструктор
 */
HotelManager::HotelManager(HotelManager&& other) noexcept
    : hotels(std::move(other.hotels)), filename(std::move(other.filename)) {
}

/**
 * @brief Деструктор
 */
HotelManager::~HotelManager() {
    // Деструктор
}

/**
 * @brief Оператор присвоєння копіюванням
 */
HotelManager& HotelManager::operator=(const HotelManager& other) {
    if (this != &other) {
        hotels = other.hotels;
        filename = other.filename;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
HotelManager& HotelManager::operator=(HotelManager&& other) noexcept {
    if (this != &other) {
        hotels = std::move(other.hotels);
        filename = std::move(other.filename);
    }
    return *this;
}

/**
 * @brief Додати новий готель
 */
bool HotelManager::addHotel(const Hotel& hotel) {
    try {
        hotels.push_back(hotel);
    }
}

```

```

        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при додаванні готелю: " << e.what() << std::endl;
        return false;
    }
}

/**
 * @brief Редагувати готель за ID
 */
bool HotelManager::editHotel(int id, const Hotel& newHotel) {
    for (auto& hotel : hotels) {
        if (hotel.getId() == id) {
            hotel = newHotel;
            hotel.setId(id); // Зберігаємо оригінальний ID
            return true;
        }
    }
    return false;
}

/**
 * @brief Видалити готель за ID
 */
bool HotelManager::deleteHotel(int id) {
    auto it = std::remove_if(hotels.begin(), hotels.end(),
        [id](const Hotel& h) { return h.getId() == id; });

    if (it != hotels.end()) {
        hotels.erase(it, hotels.end());
        return true;
    }
    return false;
}

/**
 * @brief Знайти готель за ID
 */
Hotel* HotelManager::findHotelById(int id) {
    for (auto& hotel : hotels) {
        if (hotel.getId() == id) {
            return &hotel;
        }
    }
    return nullptr;
}

/**
 * @brief Пошук готелів за назвою
 */
std::vector<Hotel> HotelManager::searchByName(const std::string& name) const {
    std::vector<Hotel> result;
    std::string lowerName = name;
    std::transform(lowerName.begin(), lowerName.end(), lowerName.begin(), ::tolower);

    for (const auto& hotel : hotels) {
        std::string hotelName = hotel.getName();
        std::transform(hotelName.begin(), hotelName.end(), hotelName.begin(), ::tolower);

        if (hotelName.find(lowerName) != std::string::npos) {
            result.push_back(hotel);
        }
    }
    return result;
}

/**
 * @brief Пошук готелів за містом
 */
std::vector<Hotel> HotelManager::searchByCity(const std::string& city) const {

```

```

std::vector<Hotel> result;
std::string lowerCity = city;
std::transform(lowerCity.begin(), lowerCity.end(), lowerCity.begin(), ::tolower);

for (const auto& hotel : hotels) {
    std::string hotelCity = hotel.getCity();
    std::transform(hotelCity.begin(), hotelCity.end(), hotelCity.begin(), ::tolower);

    if (hotelCity.find(lowerCity) != std::string::npos) {
        result.push_back(hotel);
    }
}
return result;
}

/**
 * @brief Пошук готелів за кількістю зірок
 */
std::vector<Hotel> HotelManager::searchByStars(int stars) const {
    std::vector<Hotel> result;
    for (const auto& hotel : hotels) {
        if (hotel.getStars() == stars) {
            result.push_back(hotel);
        }
    }
    return result;
}

/**
 * @brief Отримати всі готелі
 */
std::vector<Hotel> HotelManager::getAllHotels() const {
    return hotels;
}

/**
 * @brief Отримати кількість готелів
 */
size_t HotelManager::getHotelCount() const {
    return hotels.size();
}

/**
 * @brief Перевірити, чи існує готель з певним ID
 */
bool HotelManager::hotelExists(int id) const {
    return std::any_of(hotels.begin(), hotels.end(),
        [id](const Hotel& h) { return h.getId() == id; });
}

/**
 * @brief Сортувати готелі за назвою
 */
void HotelManager::sortByName() {
    std::sort(hotels.begin(), hotels.end(),
        [](const Hotel& a, const Hotel& b) {
            return a.getName() < b.getName();
        });
}

/**
 * @brief Сортувати готелі за містом
 */
void HotelManager::sortByCity() {
    std::sort(hotels.begin(), hotels.end(),
        [](const Hotel& a, const Hotel& b) {
            return a.getCity() < b.getCity();
        });
}

```

```

/**
 * @brief Сортувати готелі за кількістю зірок
 */
void HotelManager::sortByStars() {
    std::sort(hotels.begin(), hotels.end(),
        [](const Hotel& a, const Hotel& b) {
            return a.getStars() > b.getStars(); // За спаданням
        });
}

/**
 * @brief Вивести всі готелі
 */
void HotelManager::displayAll() const {
    if (hotels.empty()) {
        std::cout << "\n📭 Список готелів порожній.\n" << std::endl;
        return;
    }

    std::cout << "\n" << std::string(50, '=') << std::endl;
    std::cout << "📋 СПИСОК ГОТЕЛІВ (Всього: " << hotels.size() << ")" << std::endl;
    std::cout << std::string(50, '=') << "\n" << std::endl;

    for (const auto& hotel : hotels) {
        hotel.display();
        std::cout << std::endl;
    }
}

/**
 * @brief Завантажити дані з файлу
 */
bool HotelManager::loadFromFile() {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не вдалося відкрити файл: " << filename << std::endl;
        return false;
    }

    hotels.clear();
    std::string line;

    try {
        while (std::getline(file, line)) {
            if (line.empty()) continue;

            Hotel hotel = Hotel::fromCSV(line);
            hotels.push_back(hotel);
            Hotel::updateNextId(hotel.getId());
        }
        file.close();
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при завантаженні даних: " << e.what() << std::endl;
        file.close();
        return false;
    }
}

/**
 * @brief Зберегти дані у файл
 */
bool HotelManager::saveToFile() const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не вдалося відкрити файл для запису: " << filename << std::endl;
        return false;
    }

    try {

```

```

    for (const auto& hotel : hotels) {
        file << hotel.toCSV() << std::endl;
    }
    file.close();
    return true;
} catch (const std::exception& e) {
    std::cerr << "Помилка при збереженні даних: " << e.what() << std::endl;
    file.close();
    return false;
}
}

/**
 * @brief Очистити всі дані
 */
void HotelManager::clear() {
    hotels.clear();
}

```

– Вміст RoomManager.h

```

/**
 * @file RoomManager.h
 * @brief Заголовковий файл класу RoomManager для керування номерами
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef ROOM_MANAGER_H
#define ROOM_MANAGER_H

#include "Room.h"
#include <vector>
#include <string>
#include <memory>

/**
 * @class RoomManager
 * @brief Клас для управління колекцією номерів
 *
 * Цей клас забезпечує функціонал для роботи з базою номерів:
 * додавання, редагування, видалення, пошук, сортування.
 * Також відповідає за завантаження та збереження даних у файл.
 */
class RoomManager {
private:
    std::vector<Room*> rooms;    ///< Колекція вказівників на номери
    std::string filename;       ///< Ім'я файлу для збереження даних

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    RoomManager();

    /**
     * @brief Конструктор з параметром
     * @param filename Ім'я файлу для збереження даних
     */
    RoomManager(const std::string& filename);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт RoomManager
     */
    RoomManager(const RoomManager& other);

    /**
     * @brief Переміщувальний конструктор
     * @param other Інший об'єкт RoomManager
     */

```



```

*/
RoomManager(RoomManager&& other) noexcept;

/**
 * @brief Деструктор
 */
~RoomManager();

/**
 * @brief Оператор присвоєння копіюванням
 */
RoomManager& operator=(const RoomManager& other);

/**
 * @brief Оператор присвоєння переміщенням
 */
RoomManager& operator=(RoomManager&& other) noexcept;

/**
 * @brief Додати новий номер
 * @param room Вказівник на номер для додавання
 * @return true якщо успішно додано
 */
bool addRoom(Room* room);

/**
 * @brief Редагувати номер за ID
 * @param id ID номера
 * @param newRoom Вказівник на новий номер
 * @return true якщо успішно відредаговано
 */
bool editRoom(int id, Room* newRoom);

/**
 * @brief Видалити номер за ID
 * @param id ID номера
 * @return true якщо успішно видалено
 */
bool deleteRoom(int id);

/**
 * @brief Видалити всі номери готелю
 * @param hotelId ID готелю
 * @return Кількість видалених номерів
 */
int deleteRoomsByHotelId(int hotelId);

/**
 * @brief Знайти номер за ID
 * @param id ID номера
 * @return Вказівник на номер або nullptr
 */
Room* findRoomById(int id);

/**
 * @brief Пошук номерів за ID готелю
 * @param hotelId ID готелю
 * @return Вектор знайдених номерів
 */
std::vector<Room*> searchByHotelId(int hotelId) const;

/**
 * @brief Пошук номерів за типом
 * @param type Тип номера
 * @return Вектор знайдених номерів
 */
std::vector<Room*> searchByType(RoomType type) const;

/**
 * @brief Пошук номерів за кількістю місць

```

```

* @param capacity Кількість місць
* @return Вектор знайдених номерів
*/
std::vector<Room*> searchByCapacity(int capacity) const;

/**
* @brief Пошук вільних номерів
* @return Вектор вільних номерів
*/
std::vector<Room*> searchFreeRooms() const;

/**
* @brief Пошук вільних номерів у готелі
* @param hotelId ID готелю
* @return Вектор вільних номерів
*/
std::vector<Room*> searchFreeRoomsByHotelId(int hotelId) const;

/**
* @brief Отримати всі номери
* @return Вектор всіх номерів
*/
std::vector<Room*> getAllRooms() const;

/**
* @brief Отримати кількість номерів
* @return Кількість номерів
*/
size_t getRoomCount() const;

/**
* @brief Отримати кількість номерів у готелі
* @param hotelId ID готелю
* @return Кількість номерів
*/
size_t getRoomCountByHotelId(int hotelId) const;

/**
* @brief Отримати кількість вільних номерів у готелі
* @param hotelId ID готелю
* @return Кількість вільних номерів
*/
size_t getFreeRoomCountByHotelId(int hotelId) const;

/**
* @brief Перевірити, чи існує номер з певним ID
* @param id ID номера
* @return true якщо існує
*/
bool roomExists(int id) const;

/**
* @brief Сортувати номери за ID готелю
*/
void sortByHotelId();

/**
* @brief Сортувати номери за типом
*/
void sortByType();

/**
* @brief Сортувати номери за кількістю місць
*/
void sortByCapacity();

/**
* @brief Вивести всі номери
*/
void displayAll() const;

```

```

/**
 * @brief Вивести номери готелю
 * @param hotelId ID готелю
 */
void displayByHotelId(int hotelId) const;

/**
 * @brief Завантажити дані з файлу
 * @return true якщо успішно завантажено
 */
bool loadFromFile();

/**
 * @brief Зберегти дані у файл
 * @return true якщо успішно збережено
 */
bool saveToFile() const;

/**
 * @brief Очистити всі дані
 */
void clear();
};

#endif // ROOM_MANAGER_H

```

– Вміст RoomManager.cpp

```

/**
 * @file RoomManager.cpp
 * @brief Реалізація класу RoomManager для керування номерами
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/RoomManager.h"
#include "../include/Constants.h"
#include <fstream>
#include <algorithm>
#include <iostream>

/**
 * @brief Конструктор за замовчуванням
 */
RoomManager::RoomManager() : filename(Constants::Files::ROOMS) {
}

/**
 * @brief Конструктор з параметром
 */
RoomManager::RoomManager(const std::string& filename) : filename(filename) {
}

/**
 * @brief Копіювальний конструктор
 */
RoomManager::RoomManager(const RoomManager& other) : filename(other.filename) {
    for (const auto& room : other.rooms) {
        if (room->getType() == RoomType::LUXURY) {
            rooms.push_back(new LuxuryRoom(*dynamic_cast<LuxuryRoom*>(room)));
        } else {
            rooms.push_back(new StandardRoom(*dynamic_cast<StandardRoom*>(room)));
        }
    }
}

/**

```

```

    * @brief Переміщувальний конструктор
    */
RoomManager::RoomManager(RoomManager&& other) noexcept
    : rooms(std::move(other.rooms)), filename(std::move(other.filename)) {
}

/**
 * @brief Деструктор
 */
RoomManager::~RoomManager() {
    clear();
}

/**
 * @brief Оператор присвоєння копіюванням
 */
RoomManager& RoomManager::operator=(const RoomManager& other) {
    if (this != &other) {
        clear();
        filename = other.filename;
        for (const auto& room : other.rooms) {
            if (room->getType() == RoomType::LUXURY) {
                rooms.push_back(new LuxuryRoom(*dynamic_cast<LuxuryRoom*>(room)));
            } else {
                rooms.push_back(new StandardRoom(*dynamic_cast<StandardRoom*>(room)));
            }
        }
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
RoomManager& RoomManager::operator=(RoomManager&& other) noexcept {
    if (this != &other) {
        clear();
        rooms = std::move(other.rooms);
        filename = std::move(other.filename);
    }
    return *this;
}

/**
 * @brief Додати новий номер
 */
bool RoomManager::addRoom(Room* room) {
    try {
        if (room != nullptr) {
            rooms.push_back(room);
            return true;
        }
        return false;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при додаванні номера: " << e.what() << std::endl;
        return false;
    }
}

/**
 * @brief Редагувати номер за ID
 */
bool RoomManager::editRoom(int id, Room* newRoom) {
    for (size_t i = 0; i < rooms.size(); i++) {
        if (rooms[i]->getId() == id) {
            delete rooms[i];
            rooms[i] = newRoom;
            rooms[i]->setId(id); // Зберігаємо оригінальний ID
            return true;
        }
    }
}

```

```

    }
    return false;
}

/**
 * @brief Вудалити номер за ID
 */
bool RoomManager::deleteRoom(int id) {
    for (auto it = rooms.begin(); it != rooms.end(); ++it) {
        if ((*it)->getId() == id) {
            delete *it;
            rooms.erase(it);
            return true;
        }
    }
    return false;
}

/**
 * @brief Вудалити всі номери готелю
 */
int RoomManager::deleteRoomsByHotelId(int hotelId) {
    int count = 0;
    auto it = rooms.begin();
    while (it != rooms.end()) {
        if ((*it)->getHotelId() == hotelId) {
            delete *it;
            it = rooms.erase(it);
            count++;
        } else {
            ++it;
        }
    }
    return count;
}

/**
 * @brief Знайти номер за ID
 */
Room* RoomManager::findRoomById(int id) {
    for (auto& room : rooms) {
        if (room->getId() == id) {
            return room;
        }
    }
    return nullptr;
}

/**
 * @brief Пошук номерів за ID готелю
 */
std::vector<Room*> RoomManager::searchByHotelId(int hotelId) const {
    std::vector<Room*> result;
    for (const auto& room : rooms) {
        if (room->getHotelId() == hotelId) {
            result.push_back(room);
        }
    }
    return result;
}

/**
 * @brief Пошук номерів за типом
 */
std::vector<Room*> RoomManager::searchByType(RoomType type) const {
    std::vector<Room*> result;
    for (const auto& room : rooms) {
        if (room->getType() == type) {
            result.push_back(room);
        }
    }
}

```

```

    }
    return result;
}

/**
 * @brief Пошук номерів за кількістю місць
 */
std::vector<Room*> RoomManager::searchByCapacity(int capacity) const {
    std::vector<Room*> result;
    for (const auto& room : rooms) {
        if (room->getCapacity() == capacity) {
            result.push_back(room);
        }
    }
    return result;
}

/**
 * @brief Пошук вільних номерів
 */
std::vector<Room*> RoomManager::searchFreeRooms() const {
    std::vector<Room*> result;
    for (const auto& room : rooms) {
        if (!room->getIsOccupied()) {
            result.push_back(room);
        }
    }
    return result;
}

/**
 * @brief Пошук вільних номерів у готелі
 */
std::vector<Room*> RoomManager::searchFreeRoomsByHotelId(int hotelId) const {
    std::vector<Room*> result;
    for (const auto& room : rooms) {
        if (room->getHotelId() == hotelId && !room->getIsOccupied()) {
            result.push_back(room);
        }
    }
    return result;
}

/**
 * @brief Отримати всі номери
 */
std::vector<Room*> RoomManager::getAllRooms() const {
    return rooms;
}

/**
 * @brief Отримати кількість номерів
 */
size_t RoomManager::getRoomCount() const {
    return rooms.size();
}

/**
 * @brief Отримати кількість номерів у готелі
 */
size_t RoomManager::getRoomCountByHotelId(int hotelId) const {
    return std::count_if(rooms.begin(), rooms.end(),
        [hotelId](const Room* r) { return r->getHotelId() == hotelId; });
}

/**
 * @brief Отримати кількість вільних номерів у готелі
 */
size_t RoomManager::getFreeRoomCountByHotelId(int hotelId) const {
    return std::count_if(rooms.begin(), rooms.end(),

```

```

        [hotelId](const Room* r) {
            return r->getHotelId() == hotelId && !r->getIsOccupied();
        });
    }

    /**
     * @brief Перевірити, чи існує номер з певним ID
     */
    bool RoomManager::roomExists(int id) const {
        return std::any_of(rooms.begin(), rooms.end(),
            [id](const Room* r) { return r->getId() == id; });
    }

    /**
     * @brief Сортувати номери за ID готелю
     */
    void RoomManager::sortByHotelId() {
        std::sort(rooms.begin(), rooms.end(),
            [](const Room* a, const Room* b) {
                return a->getHotelId() < b->getHotelId();
            });
    }

    /**
     * @brief Сортувати номери за типом
     */
    void RoomManager::sortByType() {
        std::sort(rooms.begin(), rooms.end(),
            [](const Room* a, const Room* b) {
                return a->getType() < b->getType();
            });
    }

    /**
     * @brief Сортувати номери за кількістю місць
     */
    void RoomManager::sortByCapacity() {
        std::sort(rooms.begin(), rooms.end(),
            [](const Room* a, const Room* b) {
                return a->getCapacity() > b->getCapacity();
            });
    }

    /**
     * @brief Вивести всі номери
     */
    void RoomManager::displayAll() const {
        if (rooms.empty()) {
            std::cout << "\nСписок номерів порожній.\n" << std::endl;
            return;
        }

        std::cout << "\n" << std::string(50, '=') << std::endl;
        std::cout << "СПИСОК номерів (Всього: " << rooms.size() << ")" << std::endl;
        std::cout << std::string(50, '=') << "\n" << std::endl;

        for (const auto& room : rooms) {
            room->display();
            std::cout << std::endl;
        }
    }

    /**
     * @brief Вивести номери готелю
     */
    void RoomManager::displayByHotelId(int hotelId) const {
        auto hotelRooms = searchByHotelId(hotelId);

        if (hotelRooms.empty()) {
            std::cout << "\nУ цьому готелі немає номерів.\n" << std::endl;
        }
    }

```

```

        return;
    }

    std::cout << "\n" << std::string(50, '=') << std::endl;
    std::cout << "ГОМЕРНИ ГОТЕЛЮ (ID: " << hotelId << ", Всього: "
        << hotelRooms.size() << ")" << std::endl;
    std::cout << std::string(50, '=') << "\n" << std::endl;

    for (const auto& room : hotelRooms) {
        room->display();
        std::cout << std::endl;
    }
}

/**
 * @brief Завантажити дані з файлу
 */
bool RoomManager::loadFromFile() {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << Constants::Messages::ERR_FILE_OPEN << filename << std::endl;
        return false;
    }

    clear();
    std::string line;

    try {
        while (std::getline(file, line)) {
            if (line.empty()) continue;

            Room* room = Room::fromCSV(line);
            rooms.push_back(room);
            Room::updateNextId(room->getId());
        }
        file.close();
        return true;
    } catch (const std::exception& e) {
        std::cerr << Constants::Messages::ERR_LOAD_DATA << e.what() << std::endl;
        file.close();
        return false;
    }
}

/**
 * @brief Зберегти дані у файл
 */
bool RoomManager::saveToFile() const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << Constants::Messages::ERR_FILE_SAVE << filename << std::endl;
        return false;
    }

    try {
        for (const auto& room : rooms) {
            file << room->toCSV() << std::endl;
        }
        file.close();
        return true;
    } catch (const std::exception& e) {
        std::cerr << Constants::Messages::ERR_SAVE_DATA << e.what() << std::endl;
        file.close();
        return false;
    }
}

/**
 * @brief Очистити всі дані
 */

```



```
void RoomManager::clear() {
    for (auto& room : rooms) {
        delete room;
    }
    rooms.clear();
}
```

– Вміст GuestManager.h

```
/**
 * @file GuestManager.h
 * @brief Заголовковий файл класу GuestManager для керування гостями
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef GUEST_MANAGER_H
#define GUEST_MANAGER_H

#include "Guest.h"
#include <vector>
#include <string>

/**
 * @class GuestManager
 * @brief Клас для управління колекцією гостей
 *
 * Цей клас забезпечує функціонал для роботи з базою гостей:
 * додавання, редагування, видалення, пошук, сортування.
 * Також відповідає за завантаження та збереження даних у файл.
 */
class GuestManager {
private:
    std::vector<Guest> guests;    ///< Колекція гостей
    std::string filename;        ///< Ім'я файлу для збереження даних

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    GuestManager();

    /**
     * @brief Конструктор з параметром
     * @param filename Ім'я файлу для збереження даних
     */
    GuestManager(const std::string& filename);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт GuestManager
     */
    GuestManager(const GuestManager& other);

    /**
     * @brief Переміщувальний конструктор
     * @param other Інший об'єкт GuestManager
     */
    GuestManager(GuestManager&& other) noexcept;

    /**
     * @brief Деструктор
     */
    ~GuestManager();

    /**
     * @brief Оператор присвоєння копіюванням
     */
    GuestManager& operator=(const GuestManager& other);
```

```

/**
 * @brief Оператор присвоєння переміщенням
 */
GuestManager& operator=(GuestManager&& other) noexcept;

/**
 * @brief Додати нового гостя
 * @param guest Гість для додавання
 * @return true якщо успішно додано
 */
bool addGuest(const Guest& guest);

/**
 * @brief Редагувати гостя за ID
 * @param id ID гостя
 * @param newGuest Нові дані гостя
 * @return true якщо успішно відредаговано
 */
bool editGuest(int id, const Guest& newGuest);

/**
 * @brief Видалити гостя за ID
 * @param id ID гостя
 * @return true якщо успішно видалено
 */
bool deleteGuest(int id);

/**
 * @brief Знайти гостя за ID
 * @param id ID гостя
 * @return Вказівник на гостя або nullptr
 */
Guest* findGuestById(int id);

/**
 * @brief Пошук гостей за прізвищем
 * @param lastName Прізвище або частина прізвища
 * @return Вектор знайдених гостей
 */
std::vector<Guest> searchByLastName(const std::string& lastName) const;

/**
 * @brief Пошук гостей за ім'ям
 * @param firstName Ім'я або частина імені
 * @return Вектор знайдених гостей
 */
std::vector<Guest> searchByFirstName(const std::string& firstName) const;

/**
 * @brief Пошук гостя за номером телефону
 * @param phone Номер телефону
 * @return Вектор знайдених гостей
 */
std::vector<Guest> searchByPhone(const std::string& phone) const;

/**
 * @brief Отримати всіх гостей
 * @return Вектор всіх гостей
 */
std::vector<Guest> getAllGuests() const;

/**
 * @brief Отримати кількість гостей
 * @return Кількість гостей
 */
size_t getGuestCount() const;

/**
 * @brief Перевірити, чи існує гість з певним ID
 * @param id ID гостя

```

```

    * @return true якщо існує
    */
    bool guestExists(int id) const;

    /**
    * @brief Сортувати гостей за прізвищем
    */
    void sortByLastName();

    /**
    * @brief Сортувати гостей за ім'ям
    */
    void sortByFirstName();

    /**
    * @brief Сортувати гостей за повним ім'ям (прізвище + ім'я)
    */
    void sortByFullName();

    /**
    * @brief Вивести всіх гостей
    */
    void displayAll() const;

    /**
    * @brief Завантажити дані з файлу
    * @return true якщо успішно завантажено
    */
    bool loadFromFile();

    /**
    * @brief Зберегти дані у файл
    * @return true якщо успішно збережено
    */
    bool saveToFile() const;

    /**
    * @brief Очистити всі дані
    */
    void clear();
};

#endif // GUEST_MANAGER_H

```

— Вміст GuestManager.cpp

```

/**
 * @file GuestManager.cpp
 * @brief Реалізація класу GuestManager для керування гостями
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/GuestManager.h"
#include "../include/Constants.h"
#include <fstream>
#include <algorithm>
#include <iostream>

/**
 * @brief Конструктор за замовчуванням
 */
GuestManager::GuestManager() : filename(Constants::Files::GUESTS) {
}

/**
 * @brief Конструктор з параметром
 */
GuestManager::GuestManager(const std::string& filename) : filename(filename) {
}

```

```

/**
 * @brief Копіювальний конструктор
 */
GuestManager::GuestManager(const GuestManager& other)
    : guests(other.guests), filename(other.filename) {
}

/**
 * @brief Переміщувальний конструктор
 */
GuestManager::GuestManager(GuestManager&& other) noexcept
    : guests(std::move(other.guests)), filename(std::move(other.filename)) {
}

/**
 * @brief Деструктор
 */
GuestManager::~~GuestManager() {
    // Деструктор
}

/**
 * @brief Оператор присвоєння копіюванням
 */
GuestManager& GuestManager::operator=(const GuestManager& other) {
    if (this != &other) {
        guests = other.guests;
        filename = other.filename;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
GuestManager& GuestManager::operator=(GuestManager&& other) noexcept {
    if (this != &other) {
        guests = std::move(other.guests);
        filename = std::move(other.filename);
    }
    return *this;
}

/**
 * @brief Додати нового гостя
 */
bool GuestManager::addGuest(const Guest& guest) {
    try {
        guests.push_back(guest);
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при додаванні гостя: " << e.what() << std::endl;
        return false;
    }
}

/**
 * @brief Редагувати гостя за ID
 */
bool GuestManager::editGuest(int id, const Guest& newGuest) {
    for (auto& guest : guests) {
        if (guest.getId() == id) {
            guest = newGuest;
            guest.setId(id); // Зберігаємо оригінальний ID
            return true;
        }
    }
    return false;
}

```

```

/**
 * @brief Видалити гостя за ID
 */
bool GuestManager::deleteGuest(int id) {
    auto it = std::remove_if(guests.begin(), guests.end(),
        [id](const Guest& g) { return g.getId() == id; });

    if (it != guests.end()) {
        guests.erase(it, guests.end());
        return true;
    }
    return false;
}

/**
 * @brief Знайти гостя за ID
 */
Guest* GuestManager::findGuestById(int id) {
    for (auto& guest : guests) {
        if (guest.getId() == id) {
            return &guest;
        }
    }
    return nullptr;
}

/**
 * @brief Пошук гостей за прізвищем
 */
std::vector<Guest> GuestManager::searchByLastName(const std::string& lastName) const {
    std::vector<Guest> result;
    std::string lowerLastName = lastName;
    std::transform(lowerLastName.begin(), lowerLastName.end(),
        lowerLastName.begin(), ::tolower);

    for (const auto& guest : guests) {
        std::string guestLastName = guest.getLastName();
        std::transform(guestLastName.begin(), guestLastName.end(),
            guestLastName.begin(), ::tolower);

        if (guestLastName.find(lowerLastName) != std::string::npos) {
            result.push_back(guest);
        }
    }
    return result;
}

/**
 * @brief Пошук гостей за ім'ям
 */
std::vector<Guest> GuestManager::searchByFirstName(const std::string& firstName) const {
    std::vector<Guest> result;
    std::string lowerFirstName = firstName;
    std::transform(lowerFirstName.begin(), lowerFirstName.end(),
        lowerFirstName.begin(), ::tolower);

    for (const auto& guest : guests) {
        std::string guestFirstName = guest.getFirstName();
        std::transform(guestFirstName.begin(), guestFirstName.end(),
            guestFirstName.begin(), ::tolower);

        if (guestFirstName.find(lowerFirstName) != std::string::npos) {
            result.push_back(guest);
        }
    }
    return result;
}

/**

```



```

        return;
    }

    std::cout << "\n" << std::string(50, '=') << std::endl;
    std::cout << "СПИСОК ГОСТЕЙ (Всього: " << guests.size() << ")" << std::endl;
    std::cout << std::string(50, '=') << "\n" << std::endl;

    for (const auto& guest : guests) {
        guest.display();
        std::cout << std::endl;
    }
}

/**
 * @brief Завантажити дані з файлу
 */
bool GuestManager::loadFromFile() {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не вдалося відкрити файл: " << filename << std::endl;
        return false;
    }

    guests.clear();
    std::string line;

    try {
        while (std::getline(file, line)) {
            if (line.empty()) continue;

            Guest guest = Guest::fromCSV(line);
            guests.push_back(guest);
            Guest::updateNextId(guest.getId());
        }
        file.close();
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при завантаженні даних: " << e.what() << std::endl;
        file.close();
        return false;
    }
}

/**
 * @brief Зберегти дані у файл
 */
bool GuestManager::saveToFile() const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не вдалося відкрити файл для запису: " << filename << std::endl;
        return false;
    }

    try {
        for (const auto& guest : guests) {
            file << guest.toCSV() << std::endl;
        }
        file.close();
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при збереженні даних: " << e.what() << std::endl;
        file.close();
        return false;
    }
}

/**
 * @brief Очистити всі дані
 */
void GuestManager::clear() {

```

```

    guests.clear();
}

```

– Вміст BookingManager.h

```

/**
 * @file BookingManager.h
 * @brief Заголовковий файл класу BookingManager для керування бронюваннями
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef BOOKING_MANAGER_H
#define BOOKING_MANAGER_H

#include "Booking.h"
#include <vector>
#include <string>

/**
 * @class BookingManager
 * @brief Клас для управління колекцією бронювань
 *
 * Цей клас забезпечує функціонал для роботи з базою бронювань:
 * додавання, редагування, видалення, пошук, фільтрування.
 * Також відповідає за завантаження та збереження даних у файл.
 */
class BookingManager {
private:
    std::vector<Booking> bookings;    ///< Колекція бронювань
    std::string filename;             ///< Ім'я файлу для збереження даних

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    BookingManager();

    /**
     * @brief Конструктор з параметром
     * @param filename Ім'я файлу для збереження даних
     */
    BookingManager(const std::string& filename);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт BookingManager
     */
    BookingManager(const BookingManager& other);

    /**
     * @brief Переміщувальний конструктор
     * @param other Інший об'єкт BookingManager
     */
    BookingManager(BookingManager&& other) noexcept;

    /**
     * @brief Деструктор
     */
    ~BookingManager();

    /**
     * @brief Оператор присвоєння копіюванням
     */
    BookingManager& operator=(const BookingManager& other);

    /**
     * @brief Оператор присвоєння переміщенням

```



```

*/
BookingManager& operator=(BookingManager&& other) noexcept;

/**
 * @brief Додати нове бронювання
 * @param booking Бронювання для додавання
 * @return true якщо успішно додано
 */
bool addBooking(const Booking& booking);

/**
 * @brief Редагувати бронювання за ID
 * @param id ID бронювання
 * @param newBooking Нові дані бронювання
 * @return true якщо успішно відредаговано
 */
bool editBooking(int id, const Booking& newBooking);

/**
 * @brief Видалити бронювання за ID
 * @param id ID бронювання
 * @return true якщо успішно видалено
 */
bool deleteBooking(int id);

/**
 * @brief Видалити всі бронювання гостя
 * @param guestId ID гостя
 * @return Кількість видалених бронювань
 */
int deleteBookingsByGuestId(int guestId);

/**
 * @brief Видалити всі бронювання номера
 * @param roomId ID номера
 * @return Кількість видалених бронювань
 */
int deleteBookingsByRoomId(int roomId);

/**
 * @brief Знайти бронювання за ID
 * @param id ID бронювання
 * @return Вказівник на бронювання або nullptr
 */
Booking* findBookingById(int id);

/**
 * @brief Пошук бронювань за ID гостя
 * @param guestId ID гостя
 * @return Вектор знайдених бронювань
 */
std::vector<Booking> searchByGuestId(int guestId) const;

/**
 * @brief Пошук бронювань за ID номера
 * @param roomId ID номера
 * @return Вектор знайдених бронювань
 */
std::vector<Booking> searchByRoomId(int roomId) const;

/**
 * @brief Пошук бронювань за датою заїзду
 * @param checkInDate Дата заїзду
 * @return Вектор знайдених бронювань
 */
std::vector<Booking> searchByCheckInDate(const std::string& checkInDate) const;

/**
 * @brief Пошук бронювань за датою виїзду
 * @param checkOutDate Дата виїзду

```

```

* @return Вектор знайдених бронювань
*/
std::vector<Booking> searchByCheckOutDate(const std::string& checkOutDate) const;

/**
* @brief Перевірити чи номер зайнятий в певний період
* @param roomId ID номера
* @param checkInDate Дата заїзду
* @param checkOutDate Дата виїзду
* @return true якщо номер зайнятий
*/
bool isRoomOccupied(int roomId, const std::string& checkInDate,
                    const std::string& checkOutDate) const;

/**
* @brief Отримати всі бронювання
* @return Вектор всіх бронювань
*/
std::vector<Booking> getAllBookings() const;

/**
* @brief Отримати кількість бронювань
* @return Кількість бронювань
*/
size_t getBookingCount() const;

/**
* @brief Перевірити, чи існує бронювання з певним ID
* @param id ID бронювання
* @return true якщо існує
*/
bool bookingExists(int id) const;

/**
* @brief Сортувати бронювання за датою заїзду
*/
void sortByCheckInDate();

/**
* @brief Сортувати бронювання за датою виїзду
*/
void sortByCheckOutDate();

/**
* @brief Сортувати бронювання за ID гостя
*/
void sortByGuestId();

/**
* @brief Вивести всі бронювання
*/
void displayAll() const;

/**
* @brief Вивести бронювання гостя
* @param guestId ID гостя
*/
void displayByGuestId(int guestId) const;

/**
* @brief Вивести бронювання номера
* @param roomId ID номера
*/
void displayByRoomId(int roomId) const;

/**
* @brief Завантажити дані з файлу
* @return true якщо успішно завантажено
*/
bool loadFromFile();

```

```

/**
 * @brief Зберегти дані у файл
 * @return true якщо успішно збережено
 */
bool saveToFile() const;

/**
 * @brief Очистити всі дані
 */
void clear();
};

#endif // BOOKING_MANAGER_H

```

– Вміст BookingManager.cpp

```

/**
 * @file BookingManager.cpp
 * @brief Реалізація класу BookingManager для керування бронюваннями
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/BookingManager.h"
#include "../include/Constants.h"
#include <fstream>
#include <algorithm>
#include <iostream>

/**
 * @brief Конструктор за замовчуванням
 */
BookingManager::BookingManager() : filename(Constants::Files::BOOKINGS) {
}

/**
 * @brief Конструктор з параметром
 */
BookingManager::BookingManager(const std::string& filename) : filename(filename) {
}

/**
 * @brief Копіювальний конструктор
 */
BookingManager::BookingManager(const BookingManager& other)
    : bookings(other.bookings), filename(other.filename) {
}

/**
 * @brief Переміщувальний конструктор
 */
BookingManager::BookingManager(BookingManager&& other) noexcept
    : bookings(std::move(other.bookings)), filename(std::move(other.filename)) {
}

/**
 * @brief Деструктор
 */
BookingManager::~BookingManager() {
    // Деструктор
}

/**
 * @brief Оператор присвоєння копіюванням
 */
BookingManager& BookingManager::operator=(const BookingManager& other) {
    if (this != &other) {
        bookings = other.bookings;
        filename = other.filename;
    }
}

```

```

    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
BookingManager& BookingManager::operator=(BookingManager&& other) noexcept {
    if (this != &other) {
        bookings = std::move(other.bookings);
        filename = std::move(other.filename);
    }
    return *this;
}

/**
 * @brief Додати нове бронювання
 */
bool BookingManager::addBooking(const Booking& booking) {
    try {
        bookings.push_back(booking);
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при додаванні бронювання: " << e.what() << std::endl;
        return false;
    }
}

/**
 * @brief Редагувати бронювання за ID
 */
bool BookingManager::editBooking(int id, const Booking& newBooking) {
    for (auto& booking : bookings) {
        if (booking.getId() == id) {
            booking = newBooking;
            booking.setId(id); // Зберігаємо оригінальний ID
            return true;
        }
    }
    return false;
}

/**
 * @brief Вудалити бронювання за ID
 */
bool BookingManager::deleteBooking(int id) {
    auto it = std::remove_if(bookings.begin(), bookings.end(),
        [id](const Booking& b) { return b.getId() == id; });

    if (it != bookings.end()) {
        bookings.erase(it, bookings.end());
        return true;
    }
    return false;
}

/**
 * @brief Вудалити всі бронювання гостя
 */
int BookingManager::deleteBookingsByGuestId(int guestId) {
    auto it = std::remove_if(bookings.begin(), bookings.end(),
        [guestId](const Booking& b) { return b.getGuestId() == guestId; });

    int count = std::distance(it, bookings.end());
    bookings.erase(it, bookings.end());
    return count;
}

/**
 * @brief Вудалити всі бронювання номера
 */

```

```

*/
int BookingManager::deleteBookingsByRoomId(int roomId) {
    auto it = std::remove_if(bookings.begin(), bookings.end(),
        [roomId](const Booking& b) { return b.getRoomId() == roomId; });

    int count = std::distance(it, bookings.end());
    bookings.erase(it, bookings.end());
    return count;
}

/**
 * @brief Знайти бронювання за ID
 */
Booking* BookingManager::findBookingById(int id) {
    for (auto& booking : bookings) {
        if (booking.getId() == id) {
            return &booking;
        }
    }
    return nullptr;
}

/**
 * @brief Пошук бронювань за ID гостя
 */
std::vector<Booking> BookingManager::searchByGuestId(int guestId) const {
    std::vector<Booking> result;
    for (const auto& booking : bookings) {
        if (booking.getGuestId() == guestId) {
            result.push_back(booking);
        }
    }
    return result;
}

/**
 * @brief Пошук бронювань за ID номера
 */
std::vector<Booking> BookingManager::searchByRoomId(int roomId) const {
    std::vector<Booking> result;
    for (const auto& booking : bookings) {
        if (booking.getRoomId() == roomId) {
            result.push_back(booking);
        }
    }
    return result;
}

/**
 * @brief Пошук бронювань за датою заїзду
 */
std::vector<Booking> BookingManager::searchByCheckInDate(const std::string& checkInDate) const {
    std::vector<Booking> result;
    for (const auto& booking : bookings) {
        if (booking.getCheckInDate() == checkInDate) {
            result.push_back(booking);
        }
    }
    return result;
}

/**
 * @brief Пошук бронювань за датою виїзду
 */
std::vector<Booking> BookingManager::searchByCheckOutDate(const std::string& checkOutDate) const {
    std::vector<Booking> result;
    for (const auto& booking : bookings) {
        if (booking.getCheckOutDate() == checkOutDate) {
            result.push_back(booking);
        }
    }
}

```

```

    }
    return result;
}

/**
 * @brief Перевірити чи номер зайнятий в певний період
 */
bool BookingManager::isRoomOccupied(int roomId, const std::string& checkInDate,
                                     const std::string& checkOutDate) const {
    for (const auto& booking : bookings) {
        if (booking.getRoomId() == roomId) {
            // Перевіряємо перетин дат
            // Період зайнятий, якщо:
            // - новий заїзд раніше старого виїзду АБО
            // - новий виїзд пізніше старого заїзду
            if (!(checkOutDate <= booking.getCheckInDate() ||
                  checkInDate >= booking.getCheckOutDate())) {
                return true;
            }
        }
    }
    return false;
}

/**
 * @brief Отримати всі бронювання
 */
std::vector<Booking> BookingManager::getAllBookings() const {
    return bookings;
}

/**
 * @brief Отримати кількість бронювань
 */
size_t BookingManager::getBookingCount() const {
    return bookings.size();
}

/**
 * @brief Перевірити, чи існує бронювання з певним ID
 */
bool BookingManager::bookingExists(int id) const {
    return std::any_of(bookings.begin(), bookings.end(),
                       [id](const Booking& b) { return b.getId() == id; });
}

/**
 * @brief Сортувати бронювання за датою заїзду
 */
void BookingManager::sortByCheckInDate() {
    std::sort(bookings.begin(), bookings.end(),
              [](const Booking& a, const Booking& b) {
                  return a.getCheckInDate() < b.getCheckInDate();
              });
}

/**
 * @brief Сортувати бронювання за датою виїзду
 */
void BookingManager::sortByCheckOutDate() {
    std::sort(bookings.begin(), bookings.end(),
              [](const Booking& a, const Booking& b) {
                  return a.getCheckOutDate() < b.getCheckOutDate();
              });
}

/**
 * @brief Сортувати бронювання за ID гостя
 */
void BookingManager::sortByGuestId() {

```

```

std::sort(bookings.begin(), bookings.end(),
    [](const Booking& a, const Booking& b) {
        return a.getGuestId() < b.getGuestId();
    });
}

/**
 * @brief Вивести всі бронювання
 */
void BookingManager::displayAll() const {
    if (bookings.empty()) {
        std::cout << "nСписок бронювань порожній.\n" << std::endl;
        return;
    }

    std::cout << "n" << std::string(50, '=') << std::endl;
    std::cout << "СПИСОК БРОНЮВАНЬ (Всього: " << bookings.size() << ") " << std::endl;
    std::cout << std::string(50, '=') << "n" << std::endl;

    for (const auto& booking : bookings) {
        booking.display();
        std::cout << std::endl;
    }
}

/**
 * @brief Вивести бронювання гостя
 */
void BookingManager::displayByGuestId(int guestId) const {
    auto guestBookings = searchByGuestId(guestId);

    if (guestBookings.empty()) {
        std::cout << "nУ цього гостя немає бронювань.\n" << std::endl;
        return;
    }

    std::cout << "n" << std::string(50, '=') << std::endl;
    std::cout << "БРОНЮВАННЯ ГОСТЯ (ID: " << guestId << ", Всього: "
        << guestBookings.size() << ") " << std::endl;
    std::cout << std::string(50, '=') << "n" << std::endl;

    for (const auto& booking : guestBookings) {
        booking.display();
        std::cout << std::endl;
    }
}

/**
 * @brief Вивести бронювання номера
 */
void BookingManager::displayByRoomId(int roomId) const {
    auto roomBookings = searchByRoomId(roomId);

    if (roomBookings.empty()) {
        std::cout << "nДля цього номера немає бронювань.\n" << std::endl;
        return;
    }

    std::cout << "n" << std::string(50, '=') << std::endl;
    std::cout << "БРОНЮВАННЯ НОМЕРА (ID: " << roomId << ", Всього: "
        << roomBookings.size() << ") " << std::endl;
    std::cout << std::string(50, '=') << "n" << std::endl;

    for (const auto& booking : roomBookings) {
        booking.display();
        std::cout << std::endl;
    }
}

/**

```



```

#include "User.h"
#include <vector>
#include <string>

/**
 * @class UserManager
 * @brief Клас для управління колекцією користувачів системи
 *
 * Цей клас забезпечує функціонал для роботи з користувачами:
 * авторизація, додавання, видалення користувачів.
 * Також відповідає за завантаження та збереження даних у файл.
 */
class UserManager {
private:
    std::vector<User> users;    ///< Колекція користувачів
    std::string filename;      ///< Ім'я файлу для збереження даних
    User* currentUser;         ///< Поточний авторизований користувач

public:
    /**
     * @brief Конструктор за замовчуванням
     */
    UserManager();

    /**
     * @brief Конструктор з параметром
     * @param filename Ім'я файлу для збереження даних
     */
    UserManager(const std::string& filename);

    /**
     * @brief Копіювальний конструктор
     * @param other Інший об'єкт UserManager
     */
    UserManager(const UserManager& other);

    /**
     * @brief Переміщувальний конструктор
     * @param other Інший об'єкт UserManager
     */
    UserManager(UserManager&& other) noexcept;

    /**
     * @brief Деструктор
     */
    ~UserManager();

    /**
     * @brief Оператор присвоєння копіюванням
     */
    UserManager& operator=(const UserManager& other);

    /**
     * @brief Оператор присвоєння переміщенням
     */
    UserManager& operator=(UserManager&& other) noexcept;

    /**
     * @brief Ініціалізувати систему з адміністратором за замовчуванням
     */
    void initializeDefaultAdmin();

    /**
     * @brief Авторизувати користувача
     * @param username Логін
     * @param password Пароль
     * @return true якщо авторизація успішна
     */
    bool login(const std::string& username, const std::string& password);

```

```

/**
 * @brief Вийти з системи
 */
void logout();

/**
 * @brief Отримати поточного користувача
 * @return Вказівник на поточного користувача або nullptr
 */
User* getCurrentUser();

/**
 * @brief Перевірити, чи користувач авторизований
 * @return true якщо авторизований
 */
bool isLoggedIn() const;

/**
 * @brief Перевірити, чи поточний користувач адміністратор
 * @return true якщо адміністратор
 */
bool isCurrentUserAdmin() const;

/**
 * @brief Додати нового користувача (тільки для адміністратора)
 * @param user Користувач для додавання
 * @return true якщо успішно додано
 */
bool addUser(const User& user);

/**
 * @brief Видалити користувача (тільки для адміністратора)
 * @param username Логін користувача для видалення
 * @return true якщо успішно видалено
 */
bool deleteUser(const std::string& username);

/**
 * @brief Знайти користувача за логіном
 * @param username Логін користувача
 * @return Вказівник на користувача або nullptr
 */
User* findUserByUsername(const std::string& username);

/**
 * @brief Перевірити, чи існує користувач з таким логіном
 * @param username Логін користувача
 * @return true якщо існує
 */
bool userExists(const std::string& username) const;

/**
 * @brief Отримати всіх користувачів
 * @return Вектор всіх користувачів
 */
std::vector<User> getAllUsers() const;

/**
 * @brief Отримати кількість користувачів
 * @return Кількість користувачів
 */
size_t getUserCount() const;

/**
 * @brief Вивести всіх користувачів (для адміністратора)
 */
void displayAll() const;

```

```

    * @brief Завантажити дані з файлу
    * @return true якщо успішно завантажено
    */
    bool loadFromFile();

    /**
    * @brief Зберегти дані у файл
    * @return true якщо успішно збережено
    */
    bool saveToFile() const;

    /**
    * @brief Очистити всі дані
    */
    void clear();
};

#endif // USER_MANAGER_H

```

– Вміст UserManager.cpp

```

/**
 * @file UserManager.cpp
 * @brief Реалізація класу UserManager для керування користувачами
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/UserManager.h"
#include "../include/Constants.h"
#include <fstream>
#include <algorithm>
#include <iostream>

/**
 * @brief Конструктор за замовчуванням
 */
UserManager::UserManager() : filename(Constants::Files::USERS), currentUser(nullptr) {
}

/**
 * @brief Конструктор з параметром
 */
UserManager::UserManager(const std::string& filename)
    : filename(filename), currentUser(nullptr) {
    initializeDefaultAdmin();
}

/**
 * @brief Копіювальний конструктор
 */
UserManager::UserManager(const UserManager& other)
    : users(other.users), filename(other.filename), currentUser(nullptr) {
    // currentUser не копіюємо - кожен екземпляр має свою сесію
}

/**
 * @brief Переміщувальний конструктор
 */
UserManager::UserManager(UserManager&& other) noexcept
    : users(std::move(other.users)), filename(std::move(other.filename)),
      currentUser(other.currentUser) {
    other.currentUser = nullptr;
}

/**
 * @brief Деструктор
 */
UserManager::~UserManager() {
    currentUser = nullptr;
}

```

```

}

/**
 * @brief Оператор присвоєння копіюванням
 */
userManager& userManager::operator=(const userManager& other) {
    if (this != &other) {
        users = other.users;
        filename = other.filename;
        currentUser = nullptr;
    }
    return *this;
}

/**
 * @brief Оператор присвоєння переміщенням
 */
userManager& userManager::operator=(userManager&& other) noexcept {
    if (this != &other) {
        users = std::move(other.users);
        filename = std::move(other.filename);
        currentUser = other.currentUser;
        other.currentUser = nullptr;
    }
    return *this;
}

/**
 * @brief Ініціалізувати систему з адміністратором за замовчуванням
 */
void userManager::initializeDefaultAdmin() {
    // Перевіряємо, чи вже є адміністратор
    if (!userExists(Constants::DefaultUser::ADMIN_LOGIN)) {
        try {
            User admin(Constants::DefaultUser::ADMIN_LOGIN,
                Constants::DefaultUser::ADMIN_PASSWORD, true);
            users.push_back(admin);
        } catch (const std::exception& e) {
            std::cerr << "Помилка створення адміністратора: " << e.what() << std::endl;
        }
    }
}

/**
 * @brief Авторизувати користувача
 */
bool userManager::login(const std::string& username, const std::string& password) {
    for (auto& user : users) {
        if (user.getUsername() == username && user.checkPassword(password)) {
            currentUser = &user;
            return true;
        }
    }
    return false;
}

/**
 * @brief Вийти з системи
 */
void userManager::logout() {
    currentUser = nullptr;
}

/**
 * @brief Отримати поточного користувача
 */
User* userManager::getCurrentUser() {
    return currentUser;
}

```

```

/**
 * @brief Переверити, чи користувач авторизований
 */
bool UserManager::isLoggedIn() const {
    return currentUser != nullptr;
}

/**
 * @brief Переверити, чи поточний користувач адміністратор
 */
bool UserManager::isCurrentUserAdmin() const {
    return currentUser != nullptr && currentUser->getIsAdmin();
}

/**
 * @brief Додати нового користувача
 */
bool UserManager::addUser(const User& user) {
    try {
        // Переверяємо чи не існує вже такий користувач
        if (userExists(user.getUsername())) {
            std::cerr << "Користувач з таким логіном вже існує!" << std::endl;
            return false;
        }

        users.push_back(user);
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при додаванні користувача: " << e.what() << std::endl;
        return false;
    }
}

/**
 * @brief Видалити користувача
 */
bool UserManager::deleteUser(const std::string& username) {
    // Не можна видалити admin
    if (username == Constants::DefaultUser::ADMIN_LOGIN) {
        std::cerr << "Не можна видалити адміністратора!" << std::endl;
        return false;
    }

    // Не можна видалити себе
    if (currentUser && currentUser->getUsername() == username) {
        std::cerr << "Не можна видалити себе!" << std::endl;
        return false;
    }

    auto it = std::remove_if(users.begin(), users.end(),
        [&username](const User& u) { return u.getUsername() == username; });

    if (it != users.end()) {
        users.erase(it, users.end());
        return true;
    }
    return false;
}

/**
 * @brief Знайти користувача за логіном
 */
User* UserManager::findUserByUsername(const std::string& username) {
    for (auto& user : users) {
        if (user.getUsername() == username) {
            return &user;
        }
    }
    return nullptr;
}

```

```

/**
 * @brief Перевірити, чи існує користувач з таким логіном
 */
bool UserManager::userExists(const std::string& username) const {
    return std::any_of(users.begin(), users.end(),
        [&username](const User& u) { return u.getUsername() == username; });
}

/**
 * @brief Отримати всіх користувачів
 */
std::vector<User> UserManager::getAllUsers() const {
    return users;
}

/**
 * @brief Отримати кількість користувачів
 */
size_t UserManager::getUserCount() const {
    return users.size();
}

/**
 * @brief Вивести всіх користувачів
 */
void UserManager::displayAll() const {
    if (users.empty()) {
        std::cout << "\nСписок користувачів порожній.\n" << std::endl;
        return;
    }

    std::cout << "\n" << std::string(50, '=') << std::endl;
    std::cout << "СПИСОК КОРИСТУВАЧІВ (Всього: " << users.size() << ")" << std::endl;
    std::cout << std::string(50, '=') << "\n" << std::endl;

    for (const auto& user : users) {
        user.display();
        std::cout << std::endl;
    }
}

/**
 * @brief Завантажити дані з файлу
 */
bool UserManager::loadFromFile() {
    std::ifstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не вдалося відкрити файл: " << filename << std::endl;
        // Якщо файл не існує, створюємо адміністратора за замовчуванням
        initializeDefaultAdmin();
        return false;
    }

    users.clear();
    std::string line;

    try {
        while (std::getline(file, line)) {
            if (line.empty()) continue;

            User user = User::fromString(line);
            users.push_back(user);
        }
        file.close();

        // Якщо після завантаження немає адміністратора, створюємо його
        if (!userExists(Constants::DefaultUser::ADMIN_LOGIN)) {
            initializeDefaultAdmin();
        }
    }
}

```

```

        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при завантаженні даних: " << e.what() << std::endl;
        file.close();

        // У разі помилки створюємо адміністратора за замовчуванням
        users.clear();
        initializeDefaultAdmin();
        return false;
    }
}

/**
 * @brief Зберегти дані у файл
 */
bool UserManager::saveToFile() const {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "Не вдалося відкрити файл для запису: " << filename << std::endl;
        return false;
    }

    try {
        for (const auto& user : users) {
            file << user.toString() << std::endl;
        }
        file.close();
        return true;
    } catch (const std::exception& e) {
        std::cerr << "Помилка при збереженні даних: " << e.what() << std::endl;
        file.close();
        return false;
    }
}

/**
 * @brief Очистити всі дані
 */
void UserManager::clear() {
    users.clear();
    currentUser = nullptr;
    initializeDefaultAdmin();
}

```

Програмний модуль «Menu»

— Вміст Menu.h

```

/**
 * @file Menu.h
 * @brief Заголовковий файл класу Menu для інтерфейсу системи
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#ifndef MENU_H
#define MENU_H

#include "HotelManager.h"
#include "RoomManager.h"
#include "GuestManager.h"
#include "BookingManager.h"
#include "UserManager.h"

/**
 * @class Menu
 * @brief Клас для управління інтерфейсом системи бронювання готелів
 *
 * Цей клас відповідає за всю взаємодію з користувачем:

```

```

* меню, введення/виведення даних, навігацію.
*/
class Menu {
private:
    HotelManager& hotelManager;    ///< Менеджер готелів
    RoomManager& roomManager;      ///< Менеджер номерів
    GuestManager& guestManager;    ///< Менеджер гостей
    BookingManager& bookingManager; ///< Менеджер бронювань
    UserManager& userManager;      ///< Менеджер користувачів

    bool isRunning;                ///< Прапорець роботи програми

public:
    /**
     * @brief Конструктор
     * @param hotelMgr Менеджер готелів
     * @param roomMgr Менеджер номерів
     * @param guestMgr Менеджер гостей
     * @param bookingMgr Менеджер бронювань
     * @param userMgr Менеджер користувачів
     */
    Menu(HotelManager& hotelMgr, RoomManager& roomMgr, GuestManager& guestMgr,
        BookingManager& bookingMgr, UserManager& userManager);

    /**
     * @brief Деструктор
     */
    ~Menu();

    /**
     * @brief Запустити головне меню системи
     */
    void run();

private:
    // ===== АВТОРИЗАЦІЯ =====

    /**
     * @brief Показати екран авторизації
     * @return true якщо авторизація успішна
     */
    bool showLoginScreen();

    // ===== ГОЛОВНЕ МЕНЮ =====

    /**
     * @brief Показати головне меню
     */
    void showMainMenu();

    // ===== 1. КЕРУВАННЯ ГОТЕЛЯМИ =====

    /**
     * @brief Меню керування готелями
     */
    void hotelMenu();

    /**
     * @brief Додати готель
     */
    void addHotel();

    /**
     * @brief Редагувати готель
     */
    void editHotel();

    /**
     * @brief Видалити готель
     */

```



```

void deleteHotel();

/**
 * @brief Переглянути всі готелі
 */
void viewAllHotels();

/**
 * @brief Пошук готелів
 */
void searchHotels();

/**
 * @brief Сорткування готелів
 */
void sortHotels();

// ===== 2. КЕРУВАННЯ НОМЕРАМИ =====

/**
 * @brief Меню керування номерами
 */
void roomMenu();

/**
 * @brief Додати номер
 */
void addRoom();

/**
 * @brief Редагувати номер
 */
void editRoom();

/**
 * @brief Видалити номер
 */
void deleteRoom();

/**
 * @brief Переглянути всі номери
 */
void viewAllRooms();

/**
 * @brief Переглянути номери готелю
 */
void viewRoomsByHotel();

/**
 * @brief Пошук вільних номерів
 */
void searchFreeRooms();

/**
 * @brief Сорткування номерів
 */
void sortRooms();

// ===== 3. КЕРУВАННЯ ГОСТЯМИ =====

/**
 * @brief Меню керування гостями
 */
void guestMenu();

/**
 * @brief Додати гостя
 */
void addGuest();

```

```

/**
 * @brief Редагувати гостя
 */
void editGuest();

/**
 * @brief Видалити гостя
 */
void deleteGuest();

/**
 * @brief Переглянути всіх гостей
 */
void viewAllGuests();

/**
 * @brief Пошук гостей
 */
void searchGuests();

/**
 * @brief Сортювання гостей
 */
void sortGuests();

// ===== 4. КЕРУВАННЯ БРОНЮВАННЯМИ =====

/**
 * @brief Меню керування бронюваннями
 */
void bookingMenu();

/**
 * @brief Додати бронювання
 */
void addBooking();

/**
 * @brief Редагувати бронювання
 */
void editBooking();

/**
 * @brief Видалити бронювання
 */
void deleteBooking();

/**
 * @brief Переглянути всі бронювання
 */
void viewAllBookings();

/**
 * @brief Пошук бронювань
 */
void searchBookings();

/**
 * @brief Сортювання бронювань
 */
void sortBookings();

// ===== 7. ДОПОМОГА =====

/**
 * @brief Показати довідку
 */
void showHelp();

```

```
// ===== 8. АДМІНІСТРУВАННЯ =====

/**
 * @brief Меню адміністрування (тільки для адміна)
 */
void adminMenu();

/**
 * @brief Додати користувача
 */
void addUser();

/**
 * @brief Вудалити користувача
 */
void deleteUser();

/**
 * @brief Переглянути всіх користувачів
 */
void viewAllUsers();

// ===== ДОПОМІЖНІ МЕТОДИ =====

/**
 * @brief Зберегти всі дані
 */
void saveAllData();

/**
 * @brief Завантажити всі дані
 */
void loadAllData();

/**
 * @brief Вибрати готель зі списку
 * @return ID обраного готелю або -1
 */
int selectHotel();

/**
 * @brief Вибрати номер зі списку
 * @return ID обраного номера або -1
 */
int selectRoom();

/**
 * @brief Вибрати гостя зі списку
 * @return ID обраного гостя або -1
 */
int selectGuest();
};

#endif // MENU_H
```

— Вміст Menu.cpp

```
/**
 * @file Menu.cpp
 * @brief Реалізація класу Меню для інтерфейсу системи
 * @author Дем'янчук Анастасія
 * @date 2025
 */

#include "../include/Menu.h"
#include "../include/Utils.h"
```

```

#include "../include/Room.h"
#include "../include/Constants.h"
#include <iostream>
#include <iomanip>

/**
 * @brief Конструктор
 */
Menu::Menu(HotelManager& hotelMgr, RoomManager& roomMgr, GuestManager& guestMgr,
           BookingManager& bookingMgr, UserManager& userMgr)
    : hotelManager(hotelMgr), roomManager(roomMgr), guestManager(guestMgr),
      bookingManager(bookingMgr), userManager(userMgr), isRunning(false) {
}

/**
 * @brief Деструктор
 */
Menu::~Menu() {
}

/**
 * @brief Запустити головне меню системи
 */
void Menu::run() {
    Utils::clearScreen();
    Utils::printLogo();

    // Завантажити дані
    loadAllData();

    // Авторизація
    if (!showLoginScreen()) {
        Utils::printError("Не вдалося авторизуватися!");
        return;
    }

    isRunning = true;

    while (isRunning) {
        showMainMenu();
    }

    // Зберегти дані перед виходом
    saveAllData();

    Utils::clearScreen();
    Utils::printSuccess("До побачення! Дані збережено.");
}

```

```

}

// ===== АВТОРИЗАЦІЯ =====

/**
 * @brief Показати екран авторизації
 */
bool Menu::showLoginScreen() {
    Utils::printHeader("АВТОРИЗАЦІЯ");

    int attempts = 3;
    while (attempts > 0) {
        std::string username = Utils::getStringInput("Логін: ");
        std::string password = Utils::getStringInput("Пароль: ");

        if (userManager.login(username, password)) {
            Utils::printSuccess("Ви успішно увійшли в систему!");
            User* currentUser = userManager.getCurrentUser();
            if (currentUser) {
                std::cout << "Вітаємо, " << currentUser->getUsername() << "!" << std::endl;
                if (currentUser->getIsAdmin()) {
                    std::cout << "Роль: Адміністратор" << std::endl;
                }
            }
            Utils::pause();
            return true;
        } else {
            attempts--;
            Utils::printError("Невірний логін або пароль! Залишилось спроб: " +
                             std::to_string(attempts));
        }
    }

    return false;
}

// ===== ГОЛОВНЕ МЕНЮ =====

/**
 * @brief Показати головне меню
 */
void Menu::showMainMenu() {
    Utils::clearScreen();

    std::vector<std::string> options = {
        "Керування готелями",
        "Керування номерами",
    }

```

```

        "Керування гостями",
        "Керування бронюваннями",
        "Пошук бронювань",
        "Допомога",
        "Адміністрування"
    };

    int choice = Utils::displayMenu("ГОЛОВНЕ МЕНЮ", options);

    switch (choice) {
        case 1: hotelMenu(); break;
        case 2: roomMenu(); break;
        case 3: guestMenu(); break;
        case 4: bookingMenu(); break;
        case 5: searchBookings(); break;
        case 6: showHelp(); break;
        case 7: adminMenu(); break;
        case 0:
            saveAllData();
            isRunning = false;
            break;
        default:
            Utils::printError("Невірний вибір!");
            Utils::pause();
    }
}

// ===== 1. КЕРУВАННЯ ГОТЕЛЯМИ =====

/**
 * @brief Меню керування готелями
 */
void Menu::hotelMenu() {
    while (true) {
        Utils::clearScreen();

        std::vector<std::string> options = {
            "Додати готель",
            "Редагувати готель",
            "Видалити готель",
            "Переглянути всі готелі",
            "Пошук готелів",
            "Сортування готелів"
        };

        int choice = Utils::displayMenu("КЕРУВАННЯ ГОТЕЛЯМИ", options);
    }
}

```

```

switch (choice) {
    case 1: addHotel(); break;
    case 2: editHotel(); break;
    case 3: deleteHotel(); break;
    case 4: viewAllHotels(); break;
    case 5: searchHotels(); break;
    case 6: sortHotels(); break;
    case 0: return;
    default:
        Utils::printError("Невірний вибір!");
        Utils::pause();
    }
}

/**
 * @brief Додати готель
 */
void Menu::addHotel() {
    Utils::clearScreen();
    Utils::printHeader("ДОДАТИ ГОТЕЛЬ");

    try {
        std::string name = Utils::getStringInput("Назва готелю: ");
        std::string city = Utils::getStringInput("Місто: ");
        int stars = Utils::getIntInput("Кількість зірок (1-5): ", 1, 5);

        Hotel hotel(name, city, stars);

        if (hotelManager.addHotel(hotel)) {
            Utils::printSuccess("Готель успішно додано!");
        } else {
            Utils::printError("Не вдалося додати готель!");
        }
    } catch (const std::exception& e) {
        Utils::printError(std::string("Помилка: ") + e.what());
    }

    Utils::pause();
}

/**
 * @brief Редагувати готель
 */
void Menu::editHotel() {
    Utils::clearScreen();
    Utils::printHeader("РЕДАГУВАТИ ГОТЕЛЬ");

```

```

int hotelId = selectHotel();
if (hotelId == -1) return;

Hotel* hotel = hotelManager.findHotelById(hotelId);
if (!hotel) {
    Utils::printError("Готель не знайдено!");
    Utils::pause();
    return;
}

std::cout << "\nПоточні дані готелю:\n";
hotel->display();

std::cout << "\nЩо редагувати?\n";
std::cout << "1. Назву\n";
std::cout << "2. Місто\n";
std::cout << "3. Кількість зірок\n";
std::cout << "4. Все\n";
std::cout << "0. Скасувати\n";

int choice = Utils::getIntInput("Ваш вибір: ", 0, 4);

try {
    std::string name = hotel->getName();
    std::string city = hotel->getCity();
    int stars = hotel->getStars();

    switch (choice) {
        case 1:
            name = Utils::getStringInput("Нова назва (або Enter щоб залишити): ", true);
            if (name.empty()) name = hotel->getName();
            break;
        case 2:
            city = Utils::getStringInput("Нове місто (або Enter щоб залишити): ", true);
            if (city.empty()) city = hotel->getCity();
            break;
        case 3:
            stars = Utils::getIntInput("Нова кількість зірок (1-5): ", 1, 5);
            break;
        case 4:
            name = Utils::getStringInput("Нова назва: ");
            city = Utils::getStringInput("Нове місто: ");
            stars = Utils::getIntInput("Нова кількість зірок (1-5): ", 1, 5);
            break;
        case 0:
            return;
    }
}

```



```

    }

    Hotel newHotel(name, city, stars);

    if (hotelManager.editHotel(hotelId, newHotel)) {
        Utils::printSuccess("Готель успішно відредаговано!");
    } else {
        Utils::printError("Не вдалося відредагувати готель!");
    }
} catch (const std::exception& e) {
    Utils::printError(std::string("Помилка: ") + e.what());
}

Utils::pause();
}

/**
 * @brief Видалити готель
 */
void Menu::deleteHotel() {
    Utils::clearScreen();
    Utils::printHeader("ВИДАЛИТИ ГОТЕЛЬ");

    int hotelId = selectHotel();
    if (hotelId == -1) return;

    Hotel* hotel = hotelManager.findHotelById(hotelId);
    if (!hotel) {
        Utils::printError("Готель не знайдено!");
        Utils::pause();
        return;
    }

    std::cout << "\nГотель, який буде видалено:\n";
    hotel->display();

    Utils::printWarning("При видаленні готелю будуть також видалені всі його номери та бронювання!");

    if (Utils::getConfirmation("Ви впевнені, що хочете видалити цей готель?")) {
        // Видаляємо всі номери цього готелю
        int deletedRooms = roomManager.deleteRoomsByHotelId(hotelId);

        // Видаляємо готель
        if (hotelManager.deleteHotel(hotelId)) {
            Utils::printSuccess("Готель успішно видалено! Також видалено номерів: " +
                                std::to_string(deletedRooms));
        } else {

```

```

        Utils::printError("Не вдалося видалити готель!");
    }
} else {
    Utils::printInfo("Видалення скасовано.");
}

Utils::pause();
}

/**
 * @brief Переглянути всі готелі
 */
void Menu::viewAllHotels() {
    Utils::clearScreen();
    hotelManager.displayAll();
    Utils::pause();
}

/**
 * @brief Пошук готелів
 */
void Menu::searchHotels() {
    Utils::clearScreen();
    Utils::printHeader("ПОШУК ГОТЕЛІВ");

    std::cout << "Шукати за:\n";
    std::cout << "1. Назвою\n";
    std::cout << "2. Містом\n";
    std::cout << "3. Кількістю зірок\n";
    std::cout << "0. Скасувати\n";

    int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

    std::vector<Hotel> results;

    switch (choice) {
        case 1: {
            std::string name = Utils::getStringInput("Введіть назву готелю: ");
            results = hotelManager.searchByName(name);
            break;
        }
        case 2: {
            std::string city = Utils::getStringInput("Введіть місто: ");
            results = hotelManager.searchByCity(city);
            break;
        }
        case 3: {

```

```

        int stars = Utils::getIntInput("Введіть кількість зірок (1-5): ", 1, 5);
        results = hotelManager.searchByStars(stars);
        break;
    }
    case 0:
        return;
    }

    Utils::clearScreen();
    if (results.empty()) {
        Utils::printInfo("Нічого не знайдено!");
    } else {
        Utils::printHeader("РЕЗУЛЬТАТИ ПОШУКУ (" + std::to_string(results.size()) + ")");
        for (const auto& hotel : results) {
            hotel.display();
            std::cout << std::endl;
        }
    }

    Utils::pause();
}

/**
 * @brief Сорткування готелів
 */
void Menu::sortHotels() {
    Utils::clearScreen();
    Utils::printHeader("СОРТУВАННЯ ГОТЕЛІВ");

    std::cout << "Сортувати за:\n";
    std::cout << "1. Назвою\n";
    std::cout << "2. Містом\n";
    std::cout << "3. Кількістю зірок\n";
    std::cout << "0. Скасувати\n";

    int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

    switch (choice) {
        case 1:
            hotelManager.sortByName();
            Utils::printSuccess("Готелі відсортовано за назвою!");
            break;
        case 2:
            hotelManager.sortByCity();
            Utils::printSuccess("Готелі відсортовано за містом!");
            break;
        case 3:

```

```

        hotelManager.sortByStars();
        Utils::printSuccess("Готелі відсортовано за кількістю зірок!");
        break;
    case 0:
        return;
    }

    viewAllHotels();
}

// ===== 2. КЕРУВАННЯ НОМЕРАМИ =====

/**
 * @brief Меню керування номерами
 */
void Menu::roomMenu() {
    while (true) {
        Utils::clearScreen();

        std::vector<std::string> options = {
            "Додати номер",
            "Редагувати номер",
            "Видалити номер",
            "Переглянути всі номери",
            "Переглянути номери готелю",
            "Пошук вільних номерів",
            "Сортування номерів"
        };

        int choice = Utils::displayMenu("КЕРУВАННЯ НОМЕРАМИ", options);

        switch (choice) {
            case 1: addRoom(); break;
            case 2: editRoom(); break;
            case 3: deleteRoom(); break;
            case 4: viewAllRooms(); break;
            case 5: viewRoomsByHotel(); break;
            case 6: searchFreeRooms(); break;
            case 7: sortRooms(); break;
            case 0: return;
            default:
                Utils::printError("Невірний вибір!");
                Utils::pause();
        }
    }
}

```

```

/**
 * @brief Додати номер
 */
void Menu::addRoom() {
    Utils::clearScreen();
    Utils::printHeader("ДОДАТИ НОМЕР");

    int hotelId = selectHotel();
    if (hotelId == -1) return;

    try {
        std::cout << "\nТип номера:\n";
        std::cout << "1. Люкс\n";
        std::cout << "2. Стандарт\n";
        int typeChoice = Utils::getIntInput("Оберіть тип: ", 1, 2);

        int capacity = Utils::getIntInput("Кількість місць (2 або 3): ", 2, 3);

        Room* room = nullptr;
        if (typeChoice == 1) {
            room = new LuxuryRoom(hotelId, capacity);
        } else {
            room = new StandardRoom(hotelId, capacity);
        }

        if (roomManager.addRoom(room)) {
            Utils::printSuccess("Номер успішно додано!");
        } else {
            delete room;
            Utils::printError("Не вдалося додати номер!");
        }
    } catch (const std::exception& e) {
        Utils::printError(std::string("Помилка: ") + e.what());
    }

    Utils::pause();
}

/**
 * @brief Редагувати номер
 */
void Menu::editRoom() {
    Utils::clearScreen();
    Utils::printHeader("РЕДАГУВАТИ НОМЕР");

    int roomId = selectRoom();
    if (roomId == -1) return;

```

```

Room* room = roomManager.findRoomById(roomId);
if (!room) {
    Utils::printError("Номер не знайдено!");
    Utils::pause();
    return;
}

std::cout << "\nПоточні дані номера:\n";
room->display();

std::cout << "\nЩо редагувати?\n";
std::cout << "1. Тип номера\n";
std::cout << "2. Кількість місць\n";
std::cout << "3. Все\n";
std::cout << "0. Скасувати\n";

int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

try {
    int hotelId = room->getHotelId();
    RoomType type = room->getType();
    int capacity = room->getCapacity();

    switch (choice) {
        case 1: {
            std::cout << "Новий тип:\n1. Люкс\n2. Стандарт\n";
            int typeChoice = Utils::getIntInput("Оберіть: ", 1, 2);
            type = (typeChoice == 1) ? RoomType::LUXURY : RoomType::STANDARD;
            break;
        }
        case 2:
            capacity = Utils::getIntInput("Нова кількість місць (2 або 3): ", 2, 3);
            break;
        case 3: {
            std::cout << "Новий тип:\n1. Люкс\n2. Стандарт\n";
            int typeChoice = Utils::getIntInput("Оберіть: ", 1, 2);
            type = (typeChoice == 1) ? RoomType::LUXURY : RoomType::STANDARD;
            capacity = Utils::getIntInput("Нова кількість місць (2 або 3): ", 2, 3);
            break;
        }
        case 0:
            return;
    }

    Room* newRoom = nullptr;
    if (type == RoomType::LUXURY) {

```

```

        newRoom = new LuxuryRoom(hotelId, capacity);
    } else {
        newRoom = new StandardRoom(hotelId, capacity);
    }

    if (roomManager.editRoom(roomId, newRoom)) {
        Utils::printSuccess("Номер успішно відредаговано!");
    } else {
        delete newRoom;
        Utils::printError("Не вдалося відредагувати номер!");
    }
} catch (const std::exception& e) {
    Utils::printError(std::string("Помилка: ") + e.what());
}

    Utils::pause();
}

/**
 * @brief Видалити номер
 */
void Menu::deleteRoom() {
    Utils::clearScreen();
    Utils::printHeader("ВИДАЛИТИ НОМЕР");

    int roomId = selectRoom();
    if (roomId == -1) return;

    Room* room = roomManager.findRoomById(roomId);
    if (!room) {
        Utils::printError("Номер не знайдено!");
        Utils::pause();
        return;
    }

    std::cout << "\nНомер, який буде видалено:\n";
    room->display();

    Utils::printWarning("При видаленні номера будуть також видалені всі його бронювання!");

    if (Utils::getConfirmation("Ви впевнені, що хочете видалити цей номер?")) {
        // Видаляємо всі бронювання цього номера
        int deletedBookings = bookingManager.deleteBookingsByRoomId(roomId);

        // Видаляємо номер
        if (roomManager.deleteRoom(roomId)) {
            Utils::printSuccess("Номер успішно видалено! Також видалено бронювань: " +

```

```

        std::to_string(deletedBookings));
    } else {
        Utils::printError("Не вдалося видалити номер!");
    }
} else {
    Utils::printInfo("Видалення скасовано.");
}

    Utils::pause();
}

/**
 * @brief Переглянути всі номери
 */
void Menu::viewAllRooms() {
    Utils::clearScreen();
    roomManager.displayAll();
    Utils::pause();
}

/**
 * @brief Переглянути номери готелю
 */
void Menu::viewRoomsByHotel() {
    Utils::clearScreen();
    Utils::printHeader("НОМЕРИ ГОТЕЛЮ");

    int hotelId = selectHotel();
    if (hotelId == -1) return;

    Utils::clearScreen();
    roomManager.displayByHotelId(hotelId);
    Utils::pause();
}

/**
 * @brief Пошук вільних номерів
 */
void Menu::searchFreeRooms() {
    Utils::clearScreen();
    Utils::printHeader("ПОШУК ВІЛЬНИХ НОМЕРІВ");

    std::cout << "Шукати в:\n";
    std::cout << "1. Всіх готелях\n";
    std::cout << "2. Конкретному готелі\n";
    std::cout << "0. Скасувати\n";

```



```

int choice = Utils::getIntInput("Ваш вибір: ", 0, 2);

std::vector<Room*> results;

switch (choice) {
    case 1:
        results = roomManager.searchFreeRooms();
        break;
    case 2: {
        int hotelId = selectHotel();
        if (hotelId == -1) return;
        results = roomManager.searchFreeRoomsByHotelId(hotelId);
        break;
    }
    case 0:
        return;
}

Utils::clearScreen();
if (results.empty()) {
    Utils::printInfo("Вільних номерів не знайдено!");
} else {
    Utils::printHeader("ВІЛЬНІ НОМЕРИ (" + std::to_string(results.size()) + ")");
    for (const auto& room : results) {
        room->display();
        std::cout << std::endl;
    }
}

Utils::pause();
}

/**
 * @brief Сортування номерів
 */
void Menu::sortRooms() {
    Utils::clearScreen();
    Utils::printHeader("СОРТУВАННЯ НОМЕРІВ");

    std::cout << "Сортувати за:\n";
    std::cout << "1. ID готелю\n";
    std::cout << "2. Типом\n";
    std::cout << "3. Кількістю місць\n";
    std::cout << "0. Скасувати\n";

    int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

```

```

switch (choice) {
    case 1:
        roomManager.sortByHotelId();
        Utils::printSuccess("Номери відсортовано за ID готелю!");
        break;
    case 2:
        roomManager.sortByType();
        Utils::printSuccess("Номери відсортовано за типом!");
        break;
    case 3:
        roomManager.sortByCapacity();
        Utils::printSuccess("Номери відсортовано за кількістю місць!");
        break;
    case 0:
        return;
}

viewAllRooms();
}

// ===== 3. КЕРУВАННЯ ГОСТЯМИ =====

/**
 * @brief Меню керування гостями
 */
void Menu::guestMenu() {
    while (true) {
        Utils::clearScreen();

        std::vector<std::string> options = {
            "Додати гостя",
            "Редагувати гостя",
            "Видалити гостя",
            "Переглянути всіх гостей",
            "Пошук гостей",
            "Сортування гостей"
        };

        int choice = Utils::displayMenu("КЕРУВАННЯ ГОСТЯМИ", options);

        switch (choice) {
            case 1: addGuest(); break;
            case 2: editGuest(); break;
            case 3: deleteGuest(); break;
            case 4: viewAllGuests(); break;
            case 5: searchGuests(); break;
            case 6: sortGuests(); break;

```

```

        case 0: return;
        default:
            Utils::printError("Невірний вибір!");
            Utils::pause();
    }
}

/**
 * @brief Додати гостя
 */
void Menu::addGuest() {
    Utils::clearScreen();
    Utils::printHeader("ДОДАТИ ГОСТЯ");

    try {
        std::string lastName = Utils::getStringInput("Прізвище: ");
        std::string firstName = Utils::getStringInput("Ім'я: ");
        std::string phone = Utils::getStringInput("Номер телефону: ");

        Guest guest(lastName, firstName, phone);

        if (guestManager.addGuest(guest)) {
            Utils::printSuccess("Гостя успішно додано!");
        } else {
            Utils::printError("Не вдалося додати гостя!");
        }
    } catch (const std::exception& e) {
        Utils::printError(std::string("Помилка: ") + e.what());
    }

    Utils::pause();
}

/**
 * @brief Редагувати гостя
 */
void Menu::editGuest() {
    Utils::clearScreen();
    Utils::printHeader("РЕДАГУВАТИ ГОСТЯ");

    int guestId = selectGuest();
    if (guestId == -1) return;

    Guest* guest = guestManager.findGuestById(guestId);
    if (!guest) {
        Utils::printError("Гостя не знайдено!");
    }
}

```

```

    Utils::pause();
    return;
}

std::cout << "\nПоточні дані гостя:\n";
guest->display();

std::cout << "\nЩо редагувати?\n";
std::cout << "1. Прізвище\n";
std::cout << "2. Ім'я\n";
std::cout << "3. Номер телефону\n";
std::cout << "4. Все\n";
std::cout << "0. Скасувати\n";

int choice = Utils::getIntInput("Ваш вибір: ", 0, 4);

try {
    std::string lastName = guest->getLastName();
    std::string firstName = guest->getFirstName();
    std::string phone = guest->getPhone();

    switch (choice) {
        case 1:
            lastName = Utils::getStringInput("Нове прізвище (або Enter щоб залишити): ", true);
            if (lastName.empty()) lastName = guest->getLastName();
            break;
        case 2:
            firstName = Utils::getStringInput("Нове ім'я (або Enter щоб залишити): ", true);
            if (firstName.empty()) firstName = guest->getFirstName();
            break;
        case 3:
            phone = Utils::getStringInput("Новий номер телефону (або Enter щоб залишити): ", true);
            if (phone.empty()) phone = guest->getPhone();
            break;
        case 4:
            lastName = Utils::getStringInput("Нове прізвище: ");
            firstName = Utils::getStringInput("Нове ім'я: ");
            phone = Utils::getStringInput("Новий номер телефону: ");
            break;
        case 0:
            return;
    }
}

Guest newGuest(lastName, firstName, phone);

if (guestManager.editGuest(guestId, newGuest)) {
    Utils::printSuccess("Гостя успішно відредаговано!");
}

```

```

    } else {
        Utils::printError("Не вдалося відредагувати гостя!");
    }
} catch (const std::exception& e) {
    Utils::printError(std::string("Помилка: ") + e.what());
}

Utils::pause();
}

/**
 * @brief Видалити гостя
 */
void Menu::deleteGuest() {
    Utils::clearScreen();
    Utils::printHeader("ВИДАЛИТИ ГОСТЯ");

    int guestId = selectGuest();
    if (guestId == -1) return;

    Guest* guest = guestManager.findGuestById(guestId);
    if (!guest) {
        Utils::printError("Гостя не знайдено!");
        Utils::pause();
        return;
    }

    std::cout << "\nГість, якого буде видалено:\n";
    guest->display();

    Utils::printWarning("При видаленні гостя будуть також видалені всі його бронювання!");

    if (Utils::getConfirmation("Ви впевнені, що хочете видалити цього гостя?")) {
        // Видаємо всі бронювання цього гостя
        int deletedBookings = bookingManager.deleteBookingsByGuestId(guestId);

        // Видаємо гостя
        if (guestManager.deleteGuest(guestId)) {
            Utils::printSuccess("Гостя успішно видалено! Також видалено бронювань: " +
                                std::to_string(deletedBookings));
        } else {
            Utils::printError("Не вдалося видалити гостя!");
        }
    } else {
        Utils::printInfo("Видалення скасовано.");
    }
}

```

```

        Utils::pause();
    }

    /**
     * @brief Переглянути всіх гостей
     */
    void Menu::viewAllGuests() {
        Utils::clearScreen();
        guestManager.displayAll();
        Utils::pause();
    }

    /**
     * @brief Пошук гостей
     */
    void Menu::searchGuests() {
        Utils::clearScreen();
        Utils::printHeader("ПОШУК ГОСТЕЙ");

        std::cout << "Шукати за:\n";
        std::cout << "1. Прізвищем\n";
        std::cout << "2. Ім'ям\n";
        std::cout << "3. Номером телефону\n";
        std::cout << "0. Скасувати\n";

        int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

        std::vector<Guest> results;

        switch (choice) {
            case 1: {
                std::string lastName = Utils::getStringInput("Введіть прізвище: ");
                results = guestManager.searchByLastName(lastName);
                break;
            }
            case 2: {
                std::string firstName = Utils::getStringInput("Введіть ім'я: ");
                results = guestManager.searchByFirstName(firstName);
                break;
            }
            case 3: {
                std::string phone = Utils::getStringInput("Введіть номер телефону: ");
                results = guestManager.searchByPhone(phone);
                break;
            }
            case 0:
                return;
        }
    }

```

```

}

Utils::clearScreen();
if (results.empty()) {
    Utils::printInfo("Нічого не знайдено!");
} else {
    Utils::printHeader("РЕЗУЛЬТАТИ ПОШУКУ (" + std::to_string(results.size()) + ")");
    for (const auto& guest : results) {
        guest.display();
        std::cout << std::endl;
    }
}

Utils::pause();
}

/**
 * @brief Сортювання гостей
 */
void Menu::sortGuests() {
    Utils::clearScreen();
    Utils::printHeader("СОРТУВАННЯ ГОСТЕЙ");

    std::cout << "Сортювати за:\n";
    std::cout << "1. Прізвищем\n";
    std::cout << "2. Ім'ям\n";
    std::cout << "3. Повним ім'ям\n";
    std::cout << "0. Скасувати\n";

    int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

    switch (choice) {
        case 1:
            guestManager.sortByLastName();
            Utils::printSuccess("Гостей відсортовано за прізвищем!");
            break;
        case 2:
            guestManager.sortByFirstName();
            Utils::printSuccess("Гостей відсортовано за ім'ям!");
            break;
        case 3:
            guestManager.sortByFullName();
            Utils::printSuccess("Гостей відсортовано за повним ім'ям!");
            break;
        case 0:
            return;
    }
}

```

```

    viewAllGuests();
}

// ===== 4. КЕРУВАННЯ БРОНЮВАННЯМИ =====

/**
 * @brief Меню керування бронюваннями
 */
void Menu::bookingMenu() {
    while (true) {
        Utils::clearScreen();

        std::vector<std::string> options = {
            "Додати бронювання",
            "Редагувати бронювання",
            "Видалити бронювання",
            "Переглянути всі бронювання",
            "Сортування бронювань"
        };

        int choice = Utils::displayMenu("КЕРУВАННЯ БРОНЮВАННЯМИ", options);

        switch (choice) {
            case 1: addBooking(); break;
            case 2: editBooking(); break;
            case 3: deleteBooking(); break;
            case 4: viewAllBookings(); break;
            case 5: sortBookings(); break;
            case 0: return;
            default:
                Utils::printError("Невірний вибір!");
                Utils::pause();
        }
    }
}

/**
 * @brief Додати бронювання
 */
void Menu::addBooking() {
    Utils::clearScreen();
    Utils::printHeader("ДОДАТИ БРОНЮВАННЯ");

    // Показуємо вільні готелі та їх номери
    std::cout << "\n=== ВІЛЬНІ ГОТЕЛІ ТА НОМЕРИ ===\n\n";
}

```



```

auto hotels = hotelManager.getAllHotels();
for (const auto& hotel : hotels) {
    size_t freeRooms = roomManager.getFreeRoomCountByHotelId(hotel.getId());
    if (freeRooms > 0) {
        std::cout << "🏨" << hotel.getName() << " (" << hotel.getCity()
            << ") - Вільних номерів: " << freeRooms << std::endl;
    }
}

std::cout << "\n";
Utils::printSeparator(50);

// Вибір гостя
int guestId = selectGuest();
if (guestId == -1) return;

// Вибір номера (тільки вільні)
Utils::clearScreen();
Utils::printHeader("ОБЕРІТЬ НОМЕР");

auto freeRooms = roomManager.searchFreeRooms();
if (freeRooms.empty()) {
    Utils::printError("Немає вільних номерів!");
    Utils::pause();
    return;
}

std::cout << "\nВільні номери:\n\n";
for (size_t i = 0; i < freeRooms.size(); i++) {
    std::cout << (i + 1) << ". ";
    freeRooms[i]->display();
    std::cout << std::endl;
}

int roomChoice = Utils::getIntInput("Оберіть номер (0 для скасування): ",
    0, freeRooms.size());
if (roomChoice == 0) return;

int roomId = freeRooms[roomChoice - 1]->getId();

// Введення дат
std::string checkInDate = Utils::getDateInput("Дата заїзду");
std::string checkOutDate = Utils::getDateInput("Дата виїзду");

// Перевірка, чи номер вільний в цей період
if (bookingManager.isRoomOccupied(roomId, checkInDate, checkOutDate)) {
    Utils::printError("Номер вже зайнятий в цей період!");
}

```

```

    Utils::pause();
    return;
}

try {
    Booking booking(guestId, roomId, checkInDate, checkOutDate);

    if (bookingManager.addBooking(booking)) {
        // Позначаємо номер як зайнятий
        Room* room = roomManager.findRoomById(roomId);
        if (room) {
            room->setIsOccupied(true);
        }

        Utils::printSuccess("Бронювання успішно створено!");

        // Показуємо деталі бронювання
        std::cout << "\n  Деталі бронювання:\n";
        booking.display();
    } else {
        Utils::printError("Не вдалося створити бронювання!");
    }
} catch (const std::exception& e) {
    Utils::printError(std::string("Помилка: ") + e.what());
}

Utils::pause();
}

/**
 * @brief Редагувати бронювання
 */
void Menu::editBooking() {
    Utils::clearScreen();
    Utils::printHeader("РЕДАГУВАТИ БРОНЮВАННЯ");

    if (bookingManager.getBookingCount() == 0) {
        Utils::printInfo("Немає бронювань для редагування!");
        Utils::pause();
        return;
    }

    // Показуємо всі бронювання
    bookingManager.displayAll();

    int bookingId = Utils::getIntInput("Введіть ID бронювання (0 для скасування): ", 0);
    if (bookingId == 0) return;
}

```

```

Booking* booking = bookingManager.findBookingById(bookingId);
if (!booking) {
    Utils::printError("Бронювання не знайдено!");
    Utils::pause();
    return;
}

std::cout << "\nПоточні дані бронювання:\n";
booking->display();

std::cout << "\nЩо редагувати?\n";
std::cout << "1. Гостя\n";
std::cout << "2. Номер\n";
std::cout << "3. Дату заїзду\n";
std::cout << "4. Дату виїзду\n";
std::cout << "5. Все\n";
std::cout << "0. Скасувати\n";

int choice = Utils::getIntInput("Ваш вибір: ", 0, 5);

try {
    int guestId = booking->getGuestId();
    int roomId = booking->getRoomId();
    std::string checkInDate = booking->getCheckInDate();
    std::string checkOutDate = booking->getCheckOutDate();

    switch (choice) {
        case 1:
            guestId = selectGuest();
            if (guestId == -1) return;
            break;
        case 2:
            roomId = selectRoom();
            if (roomId == -1) return;
            break;
        case 3:
            checkInDate = Utils::getDateInput("Нова дата заїзду");
            break;
        case 4:
            checkOutDate = Utils::getDateInput("Нова дата виїзду");
            break;
        case 5:
            guestId = selectGuest();
            if (guestId == -1) return;
            roomId = selectRoom();
            if (roomId == -1) return;
    }
}

```

```

        checkInDate = Utils::getDateInput("Нова дата заїзду");
        checkOutDate = Utils::getDateInput("Нова дата виїзду");
        break;
    case 0:
        return;
    }

    Booking newBooking(guestId, roomId, checkInDate, checkOutDate);

    if (bookingManager.editBooking(bookingId, newBooking)) {
        Utils::printSuccess("Бронювання успішно відредаговано!");
    } else {
        Utils::printError("Не вдалося відредагувати бронювання!");
    }
} catch (const std::exception& e) {
    Utils::printError(std::string("Помилка: ") + e.what());
}

    Utils::pause();
}

/**
 * @brief Видалити бронювання
 */
void Menu::deleteBooking() {
    Utils::clearScreen();
    Utils::printHeader("ВИДАЛИТИ БРОНЮВАННЯ");

    if (bookingManager.getBookingCount() == 0) {
        Utils::printInfo("Немає бронювань для видалення!");
        Utils::pause();
        return;
    }

    // Показуємо всі бронювання
    bookingManager.displayAll();

    int bookingId = Utils::getIntInput("Введіть ID бронювання (0 для скасування): ", 0);
    if (bookingId == 0) return;

    Booking* booking = bookingManager.findBookingById(bookingId);
    if (!booking) {
        Utils::printError("Бронювання не знайдено!");
        Utils::pause();
        return;
    }
}

```

```

std::cout << "\nБронювання, яке буде видалено:\n";
booking->display();

if (Utils::getConfirmation("Ви впевнені, що хочете видалити це бронювання?")) {
    int roomId = booking->getRoomId();

    if (bookingManager.deleteBooking(bookingId)) {
        // Звільняємо номер
        Room* room = roomManager.findRoomById(roomId);
        if (room) {
            room->setIsOccupied(false);
        }

        Utils::printSuccess("Бронювання успішно видалено! Номер звільнено.");
    } else {
        Utils::printError("Не вдалося видалити бронювання!");
    }
} else {
    Utils::printInfo("Видалення скасовано.");
}

Utils::pause();
}

/**
 * @brief Переглянути всі бронювання
 */
void Menu::viewAllBookings() {
    Utils::clearScreen();
    bookingManager.displayAll();
    Utils::pause();
}

/**
 * @brief Пошук бронювань
 */
void Menu::searchBookings() {
    Utils::clearScreen();
    Utils::printHeader("ПОШУК БРОНЮВАНЬ");

    std::cout << "Шукати за:\n";
    std::cout << "1. Гостем\n";
    std::cout << "2. Номером\n";
    std::cout << "3. Датою заїзду\n";
    std::cout << "4. Датою виїзду\n";
    std::cout << "0. Скасувати\n";

```

```

int choice = Utils::getIntInput("Ваш вибір: ", 0, 4);

std::vector<Booking> results;

switch (choice) {
    case 1: {
        int guestId = selectGuest();
        if (guestId == -1) return;
        results = bookingManager.searchByGuestId(guestId);
        break;
    }
    case 2: {
        int roomId = selectRoom();
        if (roomId == -1) return;
        results = bookingManager.searchByRoomId(roomId);
        break;
    }
    case 3: {
        std::string date = Utils::getDateInput("Дата заїзду");
        results = bookingManager.searchByCheckInDate(date);
        break;
    }
    case 4: {
        std::string date = Utils::getDateInput("Дата виїзду");
        results = bookingManager.searchByCheckOutDate(date);
        break;
    }
    case 0:
        return;
}

Utils::clearScreen();
if (results.empty()) {
    Utils::printInfo("Нічого не знайдено!");
} else {
    Utils::printHeader("РЕЗУЛЬТАТИ ПОШУКУ (" + std::to_string(results.size()) + ")");
    for (const auto& booking : results) {
        booking.display();
        std::cout << std::endl;
    }
}

Utils::pause();
}

/**
 * @brief Сортування бронювань

```

```

*/
void Menu::sortBookings() {
    Utils::clearScreen();
    Utils::printHeader("СОПТУВАННЯ БРОНІЮВАНЬ");

    std::cout << "Соптувати за:\n";
    std::cout << "1. Датою заїзду\n";
    std::cout << "2. Датою виїзду\n";
    std::cout << "3. ID гостя\n";
    std::cout << "0. Скасувати\n";

    int choice = Utils::getIntInput("Ваш вибір: ", 0, 3);

    switch (choice) {
        case 1:
            bookingManager.sortByCheckInDate();
            Utils::printSuccess("Бронювання відсортовано за датою заїзду!");
            break;
        case 2:
            bookingManager.sortByCheckOutDate();
            Utils::printSuccess("Бронювання відсортовано за датою виїзду!");
            break;
        case 3:
            bookingManager.sortByGuestId();
            Utils::printSuccess("Бронювання відсортовано за ID гостя!");
            break;
        case 0:
            return;
    }

    viewAllBookings();
}

// ===== 7. ДОПОМОГА =====

/**
 * @brief Показати довідку
 */
void Menu::showHelp() {
    Utils::clearScreen();
    Utils::printHeader("ДОВІДКА");

    std::cout << R"(
 ОПИС ФУНКЦІЙ ПРОГРАМИ:

 КЕРУВАННЯ ГОТЕЛЯМИ

```

- Додавання нових готелів до системи
- Редагування інформації про готелі (назва, місто, зірки)
- Видалення готелів (з автоматичним видаленням номерів)
- Перегляд всіх готелів
- Пошук готелів за назвою, містом або кількістю зірок
- Сорткування готелів

КЕРУВАННЯ НОМЕРАМИ

- Додавання номерів до готелів (Люкс або Стандарт, 2-3 місця)
- Редагування типу та кількості місць
- Видалення номерів (з автоматичним видаленням бронювань)
- Перегляд всіх номерів або номерів конкретного готелю
- Пошук вільних номерів
- Сорткування номерів

КЕРУВАННЯ ГОСТЯМИ

- Додавання нових гостей (ПІБ, телефон)
- Редагування інформації про гостей
- Видалення гостей (з автоматичним видаленням бронювань)
- Перегляд всіх гостей
- Пошук гостей за прізвищем, ім'ям або телефоном
- Сорткування гостей за іменем

КЕРУВАННЯ БРОНЮВАННЯМИ

- Створення нових бронювань з вибором вільних номерів
- Редагування бронювань (гість, номер, дати)
- Видалення бронювань (номер автоматично звільняється)
- Перегляд всіх бронювань
- Сорткування бронювань

ПОШУК БРОНЮВАНЬ

- Пошук за гостем, номером або датами

ДОПОМОГА

- Показує цю довідку

АДМІНІСТРУВАННЯ (тільки для адміністратора)

- Додавання нових користувачів
- Видалення користувачів
- Перегляд всіх користувачів системи

Всі дані автоматично зберігаються при виході з програми.

ВАЖЛИВО:

- При видаленні готелю видаляються всі його номери та бронювання
- При видаленні номера видаляються всі його бронювання

- При видаленні гостя видаляються всі його бронювання
- Номер може бути заброньований тільки якщо він вільний

```

)" << std::endl;

Utils::pause();
}

// ===== 8. АДМІНІСТРУВАННЯ =====

/**
 * @brief Меню адміністрування
 */
void Menu::adminMenu() {
    // Перевірка прав адміністратора
    if (!userManager.isCurrentUserAdmin()) {
        Utils::printError("Доступ заборонено! Тільки для адміністраторів.");
        Utils::pause();
        return;
    }

    while (true) {
        Utils::clearScreen();

        std::vector<std::string> options = {
            "Додати користувача",
            "Видалити користувача",
            "Переглянути всіх користувачів"
        };

        int choice = Utils::displayMenu("АДМІНІСТРУВАННЯ", options);

        switch (choice) {
            case 1: addUser(); break;
            case 2: deleteUser(); break;
            case 3: viewAllUsers(); break;
            case 0: return;
            default:
                Utils::printError("Невірний вибір!");
                Utils::pause();
        }
    }
}

/**
 * @brief Додати користувача
 */

```

```

void Menu::addUser() {
    Utils::clearScreen();
    Utils::printHeader("ДОДАТИ КОРИСТУВАЧА");

    try {
        std::string username = Utils::getStringInput("Логін: ");

        if (userManager.userExists(username)) {
            Utils::printError("Користувач з таким логіном вже існує!");
            Utils::pause();
            return;
        }

        std::string password = Utils::getStringInput("Пароль: ");

        bool isAdmin = Utils::getConfirmation("Зробити адміністратором?");

        User user(username, password, isAdmin);

        if (userManager.addUser(user)) {
            Utils::printSuccess("Користувача успішно додано!");
        } else {
            Utils::printError("Не вдалося додати користувача!");
        }
    } catch (const std::exception& e) {
        Utils::printError(std::string("Помилка: ") + e.what());
    }

    Utils::pause();
}

/**
 * @brief Видалити користувача
 */
void Menu::deleteUser() {
    Utils::clearScreen();
    Utils::printHeader("ВИДАЛИТИ КОРИСТУВАЧА");

    userManager.displayAll();

    std::string username = Utils::getStringInput("Логін користувача для видалення (або Enter для скасування):", true);
    if (username.empty()) return;

    if (Utils::getConfirmation("Ви впевнені, що хочете видалити користувача " + username + " ?")) {
        if (userManager.deleteUser(username)) {
            Utils::printSuccess("Користувача успішно видалено!");
        }
    }
}

```

```

    } else {
        Utils::printError("Не вдалося видалити користувача!");
    }
} else {
    Utils::printInfo("Видалення скасовано.");
}

Utils::pause();
}

/**
 * @brief Переглянути всіх користувачів
 */
void Menu::viewAllUsers() {
    Utils::clearScreen();
    userManager.displayAll();
    Utils::pause();
}

// ===== ДОПОМІЖНІ МЕТОДИ =====

/**
 * @brief Зберегти всі дані
 */
void Menu::saveAllData() {
    Utils::printInfo("Збереження даних...");

    bool success = true;

    if (!hotelManager.saveToFile()) {
        Utils::printError("Помилка збереження готелів!");
        success = false;
    }

    if (!roomManager.saveToFile()) {
        Utils::printError("Помилка збереження номерів!");
        success = false;
    }

    if (!guestManager.saveToFile()) {
        Utils::printError("Помилка збереження гостей!");
        success = false;
    }

    if (!bookingManager.saveToFile()) {
        Utils::printError("Помилка збереження бронювань!");
        success = false;
    }
}

```

```

    }

    if (!userManager.saveToFile()) {
        Utils::printError("Помилка збереження користувачів!");
        success = false;
    }

    if (success) {
        Utils::printSuccess("Всі дані успішно збережено!");
    }
}

/**
 * @brief Завантажити всі дані
 */
void Menu::loadAllData() {
    Utils::printInfo("Завантаження даних...");

    hotelManager.loadFromFile();
    roomManager.loadFromFile();
    guestManager.loadFromFile();
    bookingManager.loadFromFile();
    userManager.loadFromFile();

    Utils::printSuccess("Дані завантажено!");

    // Показуємо статистику
    std::cout << "\n Статистика системи:\n";
    std::cout << " Готелів: " << hotelManager.getHotelCount() << std::endl;
    std::cout << " Номерів: " << roomManager.getRoomCount() << std::endl;
    std::cout << " Гостей: " << guestManager.getGuestCount() << std::endl;
    std::cout << " Бронювань: " << bookingManager.getBookingCount() << std::endl;
    std::cout << " Користувачів: " << userManager.getUserCount() << std::endl;

    Utils::pause();
}

/**
 * @brief Вибрати готель зі списку
 */
int Menu::selectHotel() {
    auto hotels = hotelManager.getAllHotels();

    if (hotels.empty()) {
        Utils::printError("Немає готелів в системі! Спочатку додайте готель.");
        Utils::pause();
        return -1;
    }
}

```

```

}

Utils::clearScreen();
Utils::printHeader("ОБЕРІТЬ ГОТЕЛЬ");

std::cout << "\nCнucок готелів:\n\n";
for (size_t i = 0; i < hotels.size(); i++) {
    std::cout << (i + 1) << ". " << hotels[i].getName()
        << " (" << hotels[i].getCity() << ") - ";
    for (int j = 0; j < hotels[i].getStars(); j++) {
        std::cout << "★";
    }
    std::cout << std::endl;
}

int choice = Utils::getIntInput("\nОберіть готель (0 для скасування): ",
    0, hotels.size());

if (choice == 0) return -1;

return hotels[choice - 1].getId();
}

/**
 * @brief Вибрати номер зі списку
 */
int Menu::selectRoom() {
    auto rooms = roomManager.getAllRooms();

    if (rooms.empty()) {
        Utils::printError("Немає номерів в системі! Спочатку додайте номер.");
        Utils::pause();
        return -1;
    }

    Utils::clearScreen();
    Utils::printHeader("ОБЕРІТЬ НОМЕР");

    std::cout << "\nCписок номерів:\n\n";
    for (size_t i = 0; i < rooms.size(); i++) {
        std::cout << (i + 1) << ". ";
        std::cout << "ID: " << rooms[i]->getId()
            << " Готель ID: " << rooms[i]->getHotelId()
            << " Тип: " << rooms[i]->getTypeName()
            << " Місць: " << rooms[i]->getCapacity()
            << " " << (rooms[i]->getIsOccupied() ? "Зайнятий" : "Вільний")
            << std::endl;
    }
}

```

```

}

int choice = Utils::getIntInput("\nОберіть номер (0 для скасування): ",
                                0, rooms.size());

if (choice == 0) return -1;

return rooms[choice - 1]->getId();
}

/**
 * @brief Вибрати гостя зі списку
 */
int Menu::selectGuest() {
    auto guests = guestManager.getAllGuests();

    if (guests.empty()) {
        Utils::printError("Немає гостей в системі! Спочатку додайте гостя.");
        Utils::pause();
        return -1;
    }

    Utils::clearScreen();
    Utils::printHeader("ОБЕРІТЬ ГОСТЯ");

    std::cout << "\nЧиселок гостей:\n\n";
    for (size_t i = 0; i < guests.size(); i++) {
        std::cout << (i + 1) << ". " << guests[i].getFullName()
                  << " (Тел: " << guests[i].getPhone() << ")" << std::endl;
    }

    int choice = Utils::getIntInput("\nОберіть гостя (0 для скасування): ",
                                    0, guests.size());

    if (choice == 0) return -1;

    return guests[choice - 1].getId();
}

```